

OGC GeoSPARQL - A Geographic Query Language for RDF Data

Table of Contents

Abstract	1
Preface	2
Keywords	3
Clarifications	4
Changes to the OGC® Abstract Specification	5
Submitters	6
Submitting organizations	7
Normative references	8
1. Terms and definitions	9
1.1. Semantic Web	9
1.2. spatial	9
2. Conventions	11
2.1. Symbols and abbreviated terms	11
2.2. Namespaces	11
2.3. Placeholder IRIs	12
2.4. Data Types for Spatial Aggregate Functions	12
2.5. RDF Serializations	12
Introduction	13
RDF	13
SPARQL	13
GeoSPARQL Standard structure	14
3. Core	17
3.1. SPARQL	17
3.2. Classes	18
3.3. Standard Properties for geo:SpatialObject	20
3.4. Standard Properties for geo:Feature	25
4. Topology Vocabulary Extension	28
4.1. Parameters	28
4.2. Simple Features Relation Family	28
4.3. Egenhofer Relation Family	29
4.4. RCC8 and RCC*-9 Relation Family	30
4.5. Equivalent RCC8, Egenhofer and Simple Features Topological Relations	32
5. Geometry Extension	33
5.1. Rationale	35
5.2. GeoSPARQL and Simple Features (SFA-CA)	36
5.3. Recommendation for units of measure	36
5.4. Influence of Reference Systems on computations	36
5.5. Parameters	37

5.6. Geometry Class	37
5.7. Standard Properties for geo:Geometry	39
5.8. Geometry Serializations	44
5.9. Non-topological Query Functions	59
5.10. Non-topological Query Functions - 3D Extension	72
5.11. Spatial Aggregate Functions	73
6. Geometry Topology Extension	75
6.1. Parameters	75
6.2. Common Query Functions	76
6.3. Simple Features Relation Family	76
6.4. Egenhofer Relation Family	77
6.5. RCC8 Relation Family	78
6.6. RCC*-9 Relation Family	78
7. RDFS Entailment Extension	80
7.1. Parameters	80
7.2. Common Requirements	80
7.3. WKT Serialization	80
7.4. GML Serialization	81
Appendix A: Abstract Test Suite	83
A.1. Conformance Class: Core	83
A.2. Conformance Class: Topology Vocabulary Extension	86
A.3. Conformance Class: Geometry Extension	87
DGGS Conformance Class: Geometry Extension - DGGS	100
A.4. Conformance Class: Geometry Topology Extension	103
A.5. Conformance Class: RDFS Entailment Extension	105
A.6. Conformance Class: Query Rewrite Extension	107
Appendix B: Functions Summary	110
B.1. Functions Summary Table	110
B.2. GeoSPARQL to SFA Functions Mapping	115
Appendix C: GeoSPARQL Examples	118
C.1. RDF Examples	118
Example SPARQL Queries & Rules	131
Appendix D: CQL / GeoSPARQL Mapping	140
D.1. Accessing spatial Features in a SPARQL endpoint	140
D.2. Mappings from CQL2 statements to GeoSPARQL queries	141
D.3. Mappings from Simple Features for SQL	145
Bibliography	149

Abstract

GeoSPARQL contains a small spatial domain OWL ontology that allow literal representations of geometries to be associated with spatial features and for features to be associated with other features using spatial relations.

GeoSPARQL also contains SPARQL extension function definitions that can be used to calculate relations between spatial objects.

Several other supporting assets are also contained within GeoSPARQL such as vocabularies of Simple Feature types and data validators.

The namespace for the GeoSPARQL ontology is <http://www.opengis.net/ont/geosparql#>

The suggested prefix for this namespace is **geo**

The namespace for the GeoSPARQL functions is <http://www.opengis.net/def/function/geosparql/>

The suggested prefix for this namespace is **geof**

The namespace for the GeoSPARQL RIF rules is <http://www.opengis.net/def/rule/geosparql/>

The suggested prefix for this namespace is **geor**

Preface

The OGC GeoSPARQL Standard defines:

- A formal *profile*;
- **this document**;
- A core RDF/OWL ontology for geographic information representation;
- A set of SPARQL extension functions;
- A Functions & Rules vocabulary, derived from the ontology;
- A Simple Features geometry types vocabulary;
- SHACL shapes for RDF data validation.

This document authoritatively defines many of the Standard's elements, including the ontology classes and properties, SPARQL functions, and function and rule vocabulary concepts. Complete descriptions of the Standard's parts and their roles are given in the Introduction in the section [GeoSPARQL Standard structure](#).

Keywords

Open Geospatial Consortium, OGC, spatial, ontology, Knowledge Graph, Semantic Web, Linked Data, RDF, Resource Description Framework, Web Ontology Language, OWL, SPARQL, Simple Features, feature, geometry

Clarifications

- The terms [Spatial Reference System \(SRS\)](#) and [Coordinate Reference System \(CRS\)](#) are no longer interchangeable. Spatial Reference System is now taken to be a broader category than Coordinate Reference System. These are defined in the [Clause 1](#) section.
- Class definitions were updated to be more self-contained and easier to understand for people without a background in geoinformatics. The definitions are no longer dependent on other standards' definitions, only informed by them.
- A section was added on [the specification of units of measurement](#).
- A section was added on the [influence of Reference Systems on computations](#).

Changes to the OGC® Abstract Specification

The OGC® Abstract Specification does not require changes to accommodate this OGC® standard.

Submitters

All questions regarding this submission should be directed to the editor or the submitters:

Contact	Company
Simon J.D. Cox	CSIRO
Panagiotis (Peter) A. Vretanos	Cubewerx Inc.
Paul Cripps	DSTL
Linda van den Brink	Geonovum
Joseph Abhayaratna	Geoscape Australia
Irina Bastrakova	Geoscience Australia
Timo Homburg	Mainz University Of Applied Sciences
Matthew Perry	Oracle America
Nicholas J. Car	KurrawongAI

Submitting organizations

The following organizations submitted this Implementation Standard to the Open Geospatial Consortium Inc.:

- CSIRO
- Cubewerx Inc.
- Defence Science and Technology Laboratory (DSTL)
- Geonovum
- Geoscape Australia
- Geoscience Australia
- KurrawongAI
- Mainz University Of Applied Sciences
- Oracle America
- OSGeo

Normative references

The following normative documents contain provisions which, through reference in this text, constitute provisions of this document. For dated references, subsequent amendments to, or revisions of, any of these publications do not apply. However, parties to agreements based on this document are encouraged to investigate the possibility of applying the most recent editions of the normative documents indicated below. For undated references, the latest edition of the normative document referred to applies.

Items in this list are linked to their full citation [Bibliography](#).

Chapter 1. Terms and definitions

For the purposes of this document, the terms and definitions given in the above normative references apply, as well as those reproduced or created in this section.

1.1. Semantic Web

The following terms and their definitions relate to Semantic Web models, tools and methods.

1.1.1. RDF

The Resource Description Framework (RDF) is a framework for representing information in the Web. RDF graphs are sets of subject-predicate-object triples, where the elements may be IRIs, blank nodes, or datatyped literals. They are used to express descriptions of resources. [Section 1.1.1](#)

1.1.2. RDFS

RDF Schema provides a data-modelling vocabulary for RDF data. RDF Schema is an extension of the basic RDF vocabulary. [Section 1.1.2](#)

1.1.3. OWL

The OWL 2 Web Ontology Language, informally OWL 2, is an ontology language for the Semantic Web with formally defined meaning. OWL 2 ontologies provide classes, properties, individuals, and data values and are stored as Semantic Web documents. OWL 2 ontologies can be used along with information written in RDF, and OWL 2 ontologies themselves are primarily exchanged as RDF documents. [\[OWL2\]](#)

1.1.4. SPARQL

SPARQL is a query language for RDF. The results of SPARQL queries can be result sets or RDF graphs. [Section 1.1.4](#)

1.2. spatial

The following terms and their definitions relate to spatial science and data.

1.2.1. coordinate system

A coordinate system is a set of mathematical rules for specifying how coordinates are to be assigned to points.

1.2.2. coordinate reference system

A coordinate reference system (CRS) is a coordinate system that is related to an object by a datum.

1.2.3. datum

A datum is a parameter or set of parameters that define the position of the origin, the scale, and the orientation of a coordinate system.

1.2.4. discrete global grid system

A discrete global grid system (DGGS) is a spatial reference system that represents the Earth, or any other globe-like object, with a tessellation of nested cells. Generally, a DGGS will exhaustively partition the globe in closely packed hierarchical tessellations, each cell representing a homogenous value, with a unique identifier or indexing that allows for linear ordering, parent-child operations, and nearest neighbor algebraic operations.

1.2.5. spatial reference system

A spatial reference system (SRS) is a system for establishing spatial position. A spatial reference system can use geographic identifiers (place names, for example), coordinates (in which case it is a coordinate reference system), or identifiers with structured geometry (in which case it is a discrete global grid system).

Chapter 2. Conventions

2.1. Symbols and abbreviated terms

In this specification, the following common acronyms are used:

CRS	Coordinate Reference System
DGGS	Discrete Global Grid System
GeoJSON	Geographic JavaScript Object Notation
GFM	General Feature Model (as defined in ISO 19109)
GIS	Geographic Information System
GML	Geography Markup Language
IRI	Internationalized Resource Identifier
KML	Keyhole Markup Language
OWL	OWL 2 Web Ontology Language
RCC	Region Connection Calculus
RDF	Resource Description Framework
RDFS	RDF Schema
RIF	Rule Interchange Format
SPARQL	SPARQL Protocol and RDF Query Language
SQL	Structured Query Language
SRS	Spatial Reference System
URI	Universal Resource Identifier
WKT	Well Known Text (as defined by Simple Features or ISO 19125)
W3C	World Wide Web Consortium (https://www.w3.org)
XML	Extensible Markup Language

2.2. Namespaces

The following IRI namespace prefixes are used throughout this document:

dcap:	http://www.w3.org/ns/dcat#
dcterms:	http://purl.org/dc/terms/
ex:	http://example.com/
geo:	http://www.opengis.net/ont/geosparql#
geof:	http://www.opengis.net/def/function/geosparql/
geor:	http://www.opengis.net/def/rule/geosparql/

gml:	http://www.opengis.net/ont/gml#
my:	http://example.org/ApplicationSchema#
ogc:	http://www.opengis.net/
owl:	http://www.w3.org/2002/07/owl#
rdf:	http://www.w3.org/1999/02/22-rdf-syntax-ns#
rdfs:	http://www.w3.org/2000/01/rdf-schema#
sf:	http://www.opengis.net/ont/sf#
skos:	http://www.w3.org/2004/02/skos/core#
xsd:	http://www.w3.org/2001/XMLSchema#

2.3. Placeholder IRIs

All of these namespace prefixes in the previous section resolve to resources that contain their namespace content except for **eg:** (<http://example.com/>), which is used just for examples, and **ogc:** (<http://www.opengis.net/>), which is used in requirement specifications as a placeholder for the geometry literal serialization used in a fully-qualified conformance class, e.g. `<http://www.opengis.net/ont/geosparql#wktLiteral>`.

2.4. Data Types for Spatial Aggregate Functions

In this specification we use the placeholder URI `ogc:geomLiteral` to describe any geometry literal that is defined in this specification. To express a list of such literals, for example as input parameters for spatial aggregate functions, we use the notation `ogc:geomLiteral[]`. We do not define the order of `ogc:geomLiteral[]`. The order is to be determined by implementers.

2.5. RDF Serializations

Three RDF serializations are used in this document. Terse RDF Triple Language (turtle) [\[TURTLE\]](#) is used for RDF snippets placed within the main body of the document, and turtle, JSON-LD [\[JSON-LD\]](#) & RDF/XML [\[RDFXML\]](#) is used for the examples in [Appendix C](#).

Introduction

The W3C Semantic Web Activity defines a collection of technologies that enables a “web of data” where information is easily shared and reused across applications. Some key pieces of this technology stack are the Resource Description Framework (RDF) data model [Section 1.1.1](#), [Section 1.1.2](#), the OWL Web Ontology Language [\[OWL2\]](#) and the SPARQL Protocol and RDF Query Language [Section 1.1.4](#).

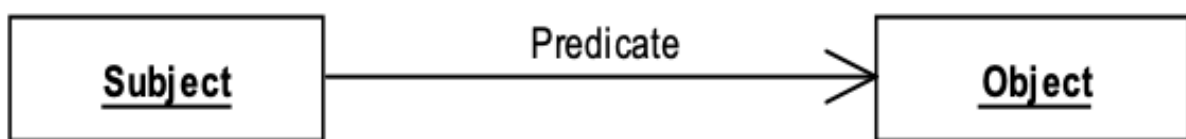
RDF

RDF is, among other things, a data model built on edge-node “graphs.” Each link in a graph consists of three elements (with many aliases depending on the mapping from other types of data models):

- Subject (start node, instance, entity, feature);
- Predicate (verb, property, attribute, relation, member, link, reference); and
- Object (value, end node, non-literal values can be used as a Subject).

Any of the three values in a triple can be represented with an Internationalized Resource Identifier (IRI) [\[IETF3987\]](#), which globally and uniquely identifies the resource referenced. IRIs are an extension to Uniform Resource Identifiers (URIs) that allow for non-ASCII characters. In addition to functioning as identifiers, IRIs are usually, but not necessarily, resolvable. This means a person or machine can “dereference” them (*click on them* or otherwise execute them) and be taken to more information about the resource, perhaps in a web browser.

Subjects and objects within an RDF triple are called nodes and can also be represented with a blank node (a local identifier without meaning outside the graph it is defined within). Objects can further be represented with a literal value. RDF uses the basic literal values from XML [\[XSD2\]](#) however the basic types can be extended for specialized purposes. Indeed, this document extends the basic types to include geometry data. The figure below shows a basic triple.



An RDF Triple

Note that the same node may be a subject in some triples, and an object in others.

Almost all data can be presented or represented in RDF. In particular, there are similarities to the (feature-instance-by-id, attribute, value) tuples of the General Feature Model [\[ISO19109\]](#), and to the relational model as well (table primary key, column, value).

SPARQL

From [Section 1.1.4](#):

SPARQL ... is a set of specifications that provide languages and protocols to

query and manipulate RDF graph content on the Web or in an RDF store.

From Wikipedia^[1]:

SPARQL (pronounced "sparkle", a recursive acronym for SPARQL Protocol and RDF Query Language) is an RDF query language - that is, a semantic query language for databases — able to retrieve and manipulate data stored in Resource Description Framework (RDF) format. It was made a standard by the RDF Data Access Working Group (DAWG) of the World Wide Web Consortium, and is recognized as one of the key technologies of the semantic web. On 15 January 2008, SPARQL 1.0 was acknowledged by W3C as an official recommendation, and SPARQL 1.1 in March, 2013.

SPARQL queries work on RDF representations of data by finding patterns that match templates in the query, in effect finding information graphs in the RDF data based on the templates and filters (constraints on nodes and edges) expressed in the query. This query template is represented in the SPARQL query by a set of parameterized “query variables” appearing in a sequence of RDF triples and filters. If the query processor finds a set of triples in the data (converted to an RDF graph in some predetermined standard manner) then the values that the “query variables” take on in those triples become a solution to the query request. The values of the variables are returned in the query result in a format based on the “SELECT” clause of the query (similar to SQL).

In addition to predicates defined in this manner, the SPARQL query may contain filter functions that can be used to further constrain the query. Several mechanisms are available to extend filter functions to allow for predicates calculated directly on data values. Section 17.6^[2] of the SPARQL specification [Section 1.1.4](#) describes the mechanism for invocation of such a filter function.

The OGC GeoSPARQL Standard supports representing and querying geospatial data on the Semantic Web. GeoSPARQL defines a vocabulary for representing geospatial data in RDF. It also defines extensions to the SPARQL query language for processing geospatial data.

GeoSPARQL does not directly provide support for temporality. Predicates for temporal relations may be used from the OWL Time Ontology [\[TIME\]](#), but query extension functions for spatiotemporal operations are not present in the GeoSPARQL standard.

GeoSPARQL Standard structure

The GeoSPARQL Standard consists of multiple parts, or *profile resources*. The comprehensive listing of these parts is given in the GeoSPARQL *profile definition*, (see <http://www.opengis.net/def/geosparql>). Below is an overview of the major parts:

1. *profile definition*

- <http://www.opengis.net/def/geosparql>
- Formally defined as an ontology, defined according to the *Profiles Vocabulary* [\[PROF\]](#);
- This relates the parts in the standard together, provides access to them, and declares

dependencies on other standards.

2. Standard document (this document)

- <http://www.opengis.net/doc/IS/geosparql/1.1>
- Defines many of the standard's parts;
- Includes normative RDF/OWL [Section 1.1.1,\[OWL2\]](#) ontology element definitions, conformance requirements and function signatures based on the General Feature Model [\[ISO19109\]](#), Simple Features [\[OGCSFACA\]](#) [\[ISO19125-1\]](#) and SQL MM [\[ISO13249\]](#);
- Also includes non-normative examples and mappings to other modelling and function systems.

3. Domain model RDF/OWL [Section 1.1.1,\[OWL2\]](#) ontology

- <http://www.opengis.net/ont/geosparql>;
- For geographic information representation;
- Based on the General Feature Model [\[ISO19109\]](#), Simple Features Access [\[OGCSFACA\]](#) [\[ISO19125-1\]](#), Geography Markup Language [\[GML\]](#) and SQL MM [\[ISO13249\]](#)
- Defined within the specification document and also delivered in RDF.

4. Functions & Rules vocabulary

- <http://www.opengis.net/def/geosparql/funcsrules>;
- Derived from the ontology;
- Presented as a [\[SKOS\]](#) taxonomy.

5. Simple Features vocabulary

- <http://www.opengis.net/ont/sf>;
- Derived from the class model defined in Simple Features Access [\[OGCSFACA\]](#) [\[ISO19125-1\]](#);
- Presented as an [\[OWL2\]](#) ontology.

6. [Section 1.1.4](#) extension functions defined within this document.

7. RDF data validator

- <http://www.opengis.net/def/geosparql/validator>;
- Defined using [\[SHACL\]](#);
- Presented within a single RDF file.

8. SPARQL 1.1 Service description for GeoSPARQL

- <http://www.opengis.net/def/geosparql/servicedescription>;
- Defined using [\[SPARQLSERVDESC\]](#).

9. Extended Examples

- Example data in RDF files too long for this document
- <https://github.com/opengeospatial/ogc-geosparql/tree/geosparql-1.1/examples>

This document follows a modular design and contains the following components:

- A *core* component defining the top-level RDFS/OWL classes for spatial objects.
- A *topology vocabulary* component defining the RDF properties for asserting and querying topological relationships between spatial objects.
- A *geometry* component defining RDFS data types for serializing geometry data, geometry-related RDF properties, and non-topological spatial query functions for geometry objects.
- A *geometry topology* component defining topological query functions.
- An *RDFS entailment* component defining mechanisms for matching implicit RDF triples that are derived based on RDF and RDFS semantics.
- A *query rewrite* component defining rules for transforming a simple triple pattern that tests a topological relationship between two features into an equivalent query involving concrete geometries and topological query functions.

Each of these components forms a set of *Requirements* known as a *GeoSPARQL Conformance Class*. Implementations can provide various levels of functionality by choosing which *Conformance Classes* to support. For example, a system based purely on qualitative spatial reasoning may support only the core and topology vocabulary Classes.

In addition, GeoSPARQL is designed to accommodate systems based on qualitative spatial reasoning and systems based on quantitative spatial computations. Systems based on qualitative spatial reasoning, (e.g., those based on Region Connection Calculus [QUAL], [LOGIC]) do not usually model explicit geometries, so queries in such systems will likely test for binary spatial relationships between features rather than between explicit geometries. To allow queries for spatial relationships between features in quantitative systems, GeoSPARQL defines a series of query transformation rules that expand a feature-only query into a geometry-based query. With these transformation rules, queries about spatial relationships between features will have the same specification in both qualitative systems and quantitative systems. The qualitative system will likely evaluate the query with a backward-chaining spatial “reasoner”, and the quantitative system can transform the query into a geometry-based query that can be evaluated with computational geometry.

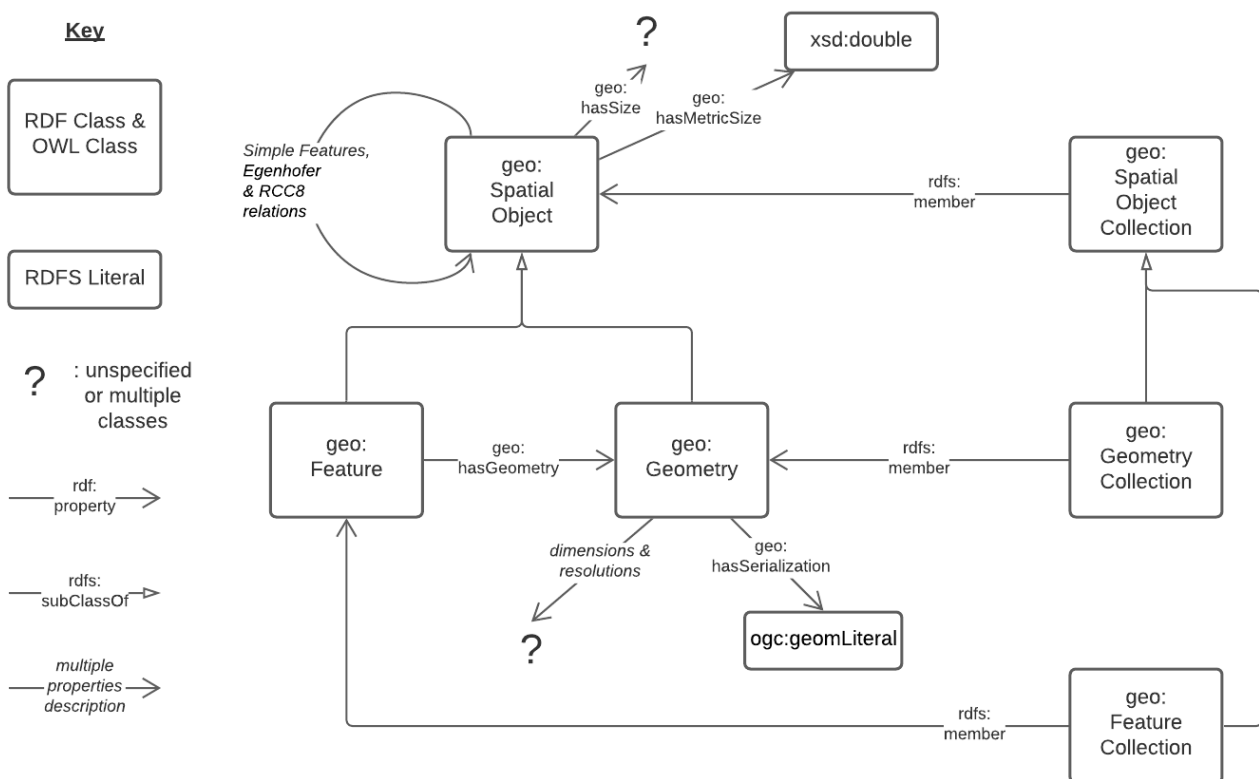
[1] <https://en.wikipedia.org/wiki/SPARQL>

[2] <https://www.w3.org/TR/sparql11-query/#extensionFunctions>

Chapter 3. Core

This clause establishes the **Core** Requirements class, with IRI [/req/core](#), which has a corresponding Conformance Class, **Core**, with IRI [/conf/core](#). These Requirements define a set of classes and properties for representing geospatial data. The resulting vocabulary - an ontology - can be used to construct SPARQL graph patterns for querying appropriately modeled geospatial data. The RDFS and OWL vocabularies have both been used so that the vocabulary can be understood by systems that support only RDFS entailment and by systems that support OWL-based reasoning.

The figure below gives an overview of the classes and properties defined by GeoSPARQL in the **Core**, **Topology Vocabulary Extension** and **Geometry Extension**, **Geometry Topology Extension** and **RDFS Entailment Extension** Conformance Classes.



An overview of the Classes and Properties defined in GeoSPARQL. Where specific Classes and Properties are indicated, the prefixed forms of their ontology identifiers (IRIs) are given. Where types or collections of properties are given, they are described in *italics*. Where unspecified Classes are given, they are represented with a question mark. For cardinalities and other ontology restrictions, see the ontology document. Subproperties of [geo:hasSize](#), its metric equivalent and [geo:hasSerialization](#) are not shown for clarity.

3.1. SPARQL

Example 1. SPARQL Protocol

Implementations shall support the SPARQL Query Language for RDF [Section 1.1.4](#), the SPARQL Protocol [\[SPARQLPROT\]](#) and the SPARQL Query Results XML [\[SPARQLRESX\]](#) and JSON [\[SPARQLRESJ\]](#) Formats.

3.2. Classes

Two main classes are defined: `geo:SpatialObject` and `Feature`.

Two container classes are defined: `Spatial Object Collection` and `Feature Collection`.

3.2.1. Class: `geo:SpatialObject`

The class `geo:SpatialObject` is defined by the following:

```
geo:SpatialObject
  a rdfs:Class, owl:Class ;
  rdfs:isDefinedBy geo: ;
  skos:prefLabel "Spatial Object"@en ;
  skos:definition "Anything spatial (being or having a shape, position or an
extent)."@en ;
  skos:note "Subclasses of this class are expected to be used for instance data."@en
;
```

Example 2. Spatial Object Class

Implementations shall allow the RDFS class `geo:SpatialObject` to be used in SPARQL graph patterns.

Example:

```
eg:x
  a geo:SpatialObject ;
  skos:prefLabel "Object X";
.
```

3.2.2. Class: `geo:Feature`

The class `geo:Feature` is equivalent to the class `GFI_Feature` [ISO19156] and is defined by the following:

```
geo:Feature
  a rdfs:Class, owl:Class ;
  rdfs:isDefinedBy geo: ;
  skos:prefLabel "Feature"@en ;
  rdfs:subClassOf geo:SpatialObject ;
  owl:disjointWith geo:Geometry ;
  skos:definition "A discrete spatial phenomenon in a universe of discourse."@en ;
  skos:note "A Feature represents a uniquely identifiable phenomenon, for example
a river or an apple. While such phenomena (and therefore the Features
```

used to represent them) are bounded, their boundaries may be crisp (e.g., the declared boundaries of a state), vague (e.g., the delineation of a valley versus its neighboring mountains), and change with time (e.g., a storm front). While discrete in nature, Features may be created from continuous observations, such as an isochrone that determines the region that can be reached by ambulance within 5 minutes."@en ;

Example 3. Feature Class

Implementations shall allow the RDFS class [Feature](#) to be used in SPARQL graph patterns.

3.2.3. Class: [geo:SpatialObjectCollection](#)

The class [Spatial Object Collection](#) is defined by the following:

```
geo:SpatialObjectCollection
  a owl:Class ;
  rdfs:isDefinedBy geo: ;
  skos:prefLabel "Spatial Object Collection" ;
  skos:definition "A collection of individual Spatial Objects."@en ;
  skos:note "This is the superclass of Feature Collection and Geometry
Collection."@en ;
  rdfs:subClassOf rdfs:Container ;
  rdfs:subClassOf [
    a owl:Restriction ;
    owl:allValuesFrom geo:SpatialObject ;
    owl:onProperty rdfs:member ;
  ] ;
.
```

Membership of the generic [rdfs:Container](#) that defines this class is restricted to instances of [Spatial Object](#). [Spatial Object Collection](#) members are to be indicated with the [rdfs:member](#) property.

Example 4. Spatial Object Collection Class

Implementations shall allow the RDFS class [Spatial Object Collection](#) to be used in SPARQL graph patterns.

3.2.4. Class: [geo:FeatureCollection](#)

The class [geo:FeatureCollection](#) is defined by the following:

```
geo:FeatureCollection
  a owl:Class ;
```

```

rdfs:isDefinedBy geo: ;
skos:prefLabel "Feature Collection" ;
skos:definition "A collection of individual Features."@en ;
rdfs:subClassOf geo:SpatialObjectCollection , geo:Feature ;
rdfs:subClassOf [
    a owl:Restriction ;
    owl:allValuesFrom :Feature ;
    owl:onProperty rdfs:member ;
] ;
.

```

This class is indicated to be a collection of spatial objects by being a subclass of the general [Spatial Object Collection](#). That it contains only [Feature](#) class instances is indicated by the restriction [rdfs:member](#) predicate. That it can be used as a Feature, for example as the domain value for the [has geometry](#) predicate, is indicated by it being a subclass of the [Geometry](#) class.

NOTE

Many spatial standards, such as [\[OGC17-069r3\]](#), contain a grouping mechanism for Features called *layers* or *feature collections* or *feature types*. These are commonly used to contain similar types of, or otherwise relates, Features. The Feature Collection class can be used for this purpose and, where it is, the bounding box geometry or other summary spatial projection of it can be indicated with a geometry directly. See

Example 5. Feature Collection Class

Implementations shall allow the RDFS class [Feature Collection](#) to be used in SPARQL graph patterns.

3.3. Standard Properties for geo:SpatialObject

Properties are defined for associating Spatial Objects with scalar spatial measurements (sizes) .

Example 6. Spatial Object Properties

Implementations shall allow the properties [geo:hasSize](#), [geo:hasMetricSize](#), [geo:hasLength](#), [geo:hasMetricLength](#), [geo:hasPerimeterLength](#), [geo:hasMetricPerimeterLength](#), [geo:hasArea](#), [geo:hasMetricArea](#), [geo:hasVolume](#) and [geo:hasMetricVolume](#) to be used in SPARQL graph patterns.

3.3.1. Property: geo:hasSize

The property [geo:hasSize](#) is the superproperty of all properties that can be used to indicate the size of a Spatial Object in case (only) metric units (meter, square meter or cubic meter) can not be used. If it is possible to express size in metric units, subproperties of [geo:hasMetricSize](#) should be used. This property has not range specification. This makes it possible to use other vocabularies for expressions of size, for example vocabularies for units of measurement or vocabularies for

specifying measurement quality.

GeoSPARQL 1.1 defines the following subproperties of this property: `geo:hasLength`, `geo:hasPerimeterLength`, `geo:hasArea` and `geo:hasVolume`.

```
geo:hasSize
  a rdf:Property, owl:ObjectProperty ;
  rdfs:isDefinedBy geo: ;
  rdfs:domain geo:SpatialObject ;
  skos:definition "Subproperties of this property are used to indicate the size of a
                  Spatial Object as a measurement or estimate of one or more
dimensions
                  of the Spatial Object's spatial presence."@en ;
  skos:prefLabel "has size"@en ;
  .
```

3.3.2. Property: `geo:hasMetricSize`

The property `geo:hasMetricSize` is the superproperty of all properties that can be used to indicate the size of a Spatial Object using metric units (meter, square meter or cubic meter). Using a subproperty of this property is the recommended way to specify size, because using a standard unit of length (meter) benefits data interoperability and simplicity. Subproperties of `geo:hasSize` can be used if more complex expressions are necessary, for example if the unit of length can not be converted to meter, or if additional data are needed to describe the measurement or estimate of size.

GeoSPARQL 1.1 defines the following subproperties of this property: `geo:hasMetricLength`, `geo:hasMetricPerimeterLength`, `geo:hasMetricArea` and `geo:hasMetricVolume`.

```
geo:hasMetricSize
  a rdf:Property, owl:DatatypeProperty ;
  rdfs:isDefinedBy geo: ;
  rdfs:domain geo:SpatialObject ;
  rdfs:range xsd:double ;
  skos:definition "Subproperties of this property are used to indicate the size of a
                  Spatial Object, as a measurement or estimate of one or more
dimensions
                  of the Spatial Object's spatial presence. Units are always metric
                  (meter, square meter or cubic meter)."@en ;
  skos:prefLabel "has metric size"@en ;
  .
```

3.3.3. Property: `geo:hasLength`

The property `geo:hasLength` can be used to indicate the length of a Spatial Object if it is not possible to use the property `geo:hasMetricLength`. It is a subproperty of `geo:hasSize`.


```

geo:hasLength
  a rdf:Property, owl:ObjectProperty ;
  rdfs:isDefinedBy geo: ;
  rdfs:subPropertyOf geo:hasSize ;
  rdfs:domain geo:SpatialObject ;
  skos:definition "The length of a Spatial Object."@en ;
  skos:prefLabel "has length"@en ;
.

```

3.3.4. Property: `geo:hasMetricLength`

The property `geo:hasMetricLength` can be used to indicate the length of a Spatial Object in meters (m). It is a subproperty of `geo:hasMetricSize`. This property can be used for Spatial Objects having one, two, or three dimensions.

```

geo:hasMetricLength
  a rdf:Property, owl:DatatypeProperty ;
  rdfs:isDefinedBy geo: ;
  rdfs:subPropertyOf geo:hasMetricSize ;
  rdfs:domain geo:SpatialObject ;
  rdfs:range xsd:double ;
  skos:definition "The length of a Spatial Object in meters."@en ;
  skos:prefLabel "has length in meters"@en ;
.

```

3.3.5. Property: `geo:hasPerimeterLength`

The property `geo:hasPerimeterLength` can be used to indicate the length of the outer boundary of a Spatial Object if it is not possible to use the property `geo:hasMetricPerimeterLength`. It is a subproperty of `geo:hasSize`.

```

geo:hasPerimeterLength
  a rdf:Property, owl:ObjectProperty ;
  rdfs:isDefinedBy geo: ;
  rdfs:subPropertyOf geo:hasSize ;
  skos:definition "The length of the perimeter of a Spatial Object."@en ;
  skos:prefLabel "has perimeter length"@en ;
.

```

3.3.6. Property: `geo:hasMetricPerimeterLength`

The property `geo:hasMetricPerimeterLength` can be used to indicate the length of the outer boundary of a Spatial Object in meters (m). It is a subproperty of `geo:hasMetricSize`. Circumference is considered a type of perimeter, so this property can be used for circular or curved objects too. This property can be used for Spatial Objects having two or three dimensions.

```

geo:hasMetricPerimeterLength
  a rdf:Property, owl:DatatypeProperty ;
  rdfs:isDefinedBy geo: ;
  rdfs:subPropertyOf geo:hasMetricSize ;
  rdfs:domain geo:SpatialObject ;
  rdfs:range xsd:double ;
  skos:definition "The length of the perimeter of a Spatial Object in meters."@en ;
  skos:prefLabel "has perimeter length in meters"@en ;
.

```

TIP

A consistency check can be applied to Geometry instances indicating both this property and the property `geo:dimension`: if supplied, the `geo:dimension` property's range value must be the literal integer 2 or 3. The following SPARQL query will return `true` if applied to a graph where this is not the case for all Geometries:

```

PREFIX geo: <http://www.opengis.net/ont/geosparql#>
ASK
WHERE {
  ?g geo:hasMetricPerimeterLength ?p ;
      geo:dimension ?d .

  FILTER (?d < 2)
}

```

3.3.7. Property: `geo:hasArea`

The property `geo:hasArea` can be used to indicate the area of a Spatial Object if it is not possible to use the property `geo:hasMetricArea`. It is a subproperty of `geo:hasSize`.

```

geo:hasArea
  a rdf:Property, owl:ObjectProperty ;
  rdfs:isDefinedBy geo: ;
  rdfs:subPropertyOf geo:hasSize ;
  rdfs:domain geo:SpatialObject ;
  skos:definition "The area of a Spatial Object."@en ;
  skos:prefLabel "has area"@en ;
.

```

3.3.8. Property: `geo:hasMetricArea`

The property `geo:hasMetricArea` can be used to indicate the area of a Spatial Object in square meters (m²). It is a subproperty of `geo:hasMetricSize`. This property can be used for Spatial Objects having two or three dimensions.

```

geo:hasMetricArea

```

```

a rdf:Property, owl:DatatypeProperty ;
rdfs:isDefinedBy geo: ;
rdfs:subPropertyOf geo:hasMetricSize ;
rdfs:domain geo:SpatialObject ;
rdfs:range xsd:double ;
skos:definition "The area of a Spatial Object in square meters."@en ;
skos:prefLabel "has area in meters"@en ;
.

```

TIP

A consistency check can be applied to Geometry instances indicating both this property and the property `geo:dimension`: if supplied, the `geo:dimension` property's range value must be the literal integer 2 or 3. The following SPARQL query will return `true` if applied to a graph where this is not the case for all Geometries:

```

PREFIX geo: <http://www.opengis.net/ont/geosparql#>

ASK
WHERE {
    ?g geo:hasMetricArea ?a ;
        geo:dimension ?d .

    FILTER (?d < 2)
}

```

3.3.9. Property: `geo:hasVolume`

The property `geo:hasVolume` can be used to indicate the volume of a Spatial Object if it is not possible to use the property `geo:hasMetricVolume`. It is a subproperty of `geo:hasSize`.

```

geo:hasVolume
a rdf:Property, owl:ObjectProperty ;
rdfs:isDefinedBy geo: ;
rdfs:subPropertyOf geo:hasSize ;
rdfs:domain geo:SpatialObject ;
skos:definition "The volume of a three-dimensional Spatial Object."@en ;
skos:prefLabel "has volume"@en ;
.

```

3.3.10. Property: `geo:hasMetricVolume`

The property `geo:hasMetricVolume` can be used to indicate the volume of a Spatial Object in cubic meters (m³). It is a subproperty of `geo:hasMetricSize`. This property can be used for Spatial Objects having three dimensions.

```

geo:hasMetricVolume
a rdf:Property, owl:DatatypeProperty ;

```

```
rdfs:isDefinedBy geo: ;
rdfs:subPropertyOf :hasMetricSize ;
rdfs:domain geo:SpatialObject ;
rdfs:range xsd:double ;
skos:definition "The volume of a Spatial Object in cubic meters."@en ;
skos:prefLabel "has area in meters"@en ;
```

TIP

A consistency check can be applied to Geometries indicating both this property and the property `geo:dimension`: if supplied, the property `geo:dimension` property's range value must be the literal integer 3. The following SPARQL query will return `true` if applied to a graph where this is not the case for all Geometries:

```
PREFIX geo: <http://www.opengis.net/ont/geosparql#>

ASK
WHERE {
    ?g geo:hasMetricVolume ?v ;
        geo:dimension ?d .

    FILTER (?d != 3)
}
```

3.4. Standard Properties for geo:Feature

Properties are defined for associating `Feature` instances with `geo:Geometry` instances.

Example 7. Feature Properties

Implementations shall allow the properties `geo:hasGeometry`, `geo:hasDefaultGeometry`, `geo:hasCentroid` and `geo:hasBoundingBox` to be used in SPARQL graph patterns.

3.4.1. Property: `geo:hasGeometry`

The property `geo:hasGeometry` is used to link a `Feature` with a `Geometry` that represents a spatial projection of it.

A `Feature` may be linked to `Geometry Collection` instead of a simple `Geometry`.

A `Feature` may also be associated with multiple individual `Geometries` or even multiple individual `Geometry Collections`, or any combinations of `Geometries` or `Geometry Collections`.

```
geo:hasGeometry
  a rdf:Property, owl:ObjectProperty ;
  rdfs:isDefinedBy geo: ;
  rdfs:domain geo:Feature ;
```

```
rdfs:range geo:Geometry ;
skos:prefLabel "has Geometry"@en ;
skos:definition "A spatial representation for a given Feature."@en ;
```

3.4.2. Property: `geo:hasDefaultGeometry`

The property `geo:hasDefaultGeometry` is used to link a Feature with its default Geometry. The default geometry is the Geometry that should be used for spatial calculations in the absence of a request for a specific geometry (e.g. in the case of query rewrite).

```
geo:hasDefaultGeometry
  a rdf:Property, owl:ObjectProperty ;
  rdfs:isDefinedBy geo: ;
  rdfs:domain geo:Feature ;
  rdfs:range geo:Geometry ;
  skos:prefLabel "has Default Geometry"@en ;
  skos:definition "The default geometry to be used in spatial calculations,
                  usually the most detailed geometry."@en ;
  rdfs:subPropertyOf geo:hasGeometry ;
```

GeoSPARQL does not restrict the cardinality of the `has default geometry` property. It is thus possible for a Feature to have more than one distinct default geometry or to have no default geometry. This situation does not result in a query processing error; SPARQL graph pattern matching simply proceeds as normal. Certain queries may, however, give logically inconsistent results. For example, if a Feature `my:f1` has two asserted default geometries, and those two geometries are disjoint polygons, the query below could return a non-zero count on a system supporting the GeoSPARQL Query Rewrite Extension (rule `geor:sfDisjoint`).

```
PREFIX geo: <http://www.opengis.net/ont/geosparql#>

SELECT (COUNT(*) AS ?cnt)
WHERE { :f1 geo:sfDisjoint :f1 }
```

Such cases are application-specific data modeling errors and are therefore outside of the scope of the GeoSPARQL specification., however it is recommended that multiple geometries indicated with `geo:hasDefaultGeometry` should be differentiated by `Geometry` class properties, perhaps relating to precision, SRS etc.

3.4.3. Property: `geo:hasBoundingBox`

The property `geo:hasBoundingBox` is used to link a Feature with a simplified geometry-representation corresponding to the envelope of the feature's geometry. Bounding-boxes are typically used in indexing and discovery.

```

geo:hasBoundingBox
  a rdf:Property, owl:ObjectProperty ;
  rdfs:isDefinedBy geo: ;
  rdfs:subPropertyOf geo:hasGeometry ;
  rdfs:domain geo:Feature ;
  rdfs:range geo:Geometry ;
  skos:prefLabel "has bounding box"@en ;
  skos:definition "The minimum or smallest bounding or enclosing box of a given
Feature."@en ;
  skos:scopeNote "The target is a geometry that defines a rectilinear region whose
edges are
                aligned with the axes of the coordinate reference system, which
exactly
                contains the geometry or Feature e.g. sf:Envelope"@en ;
.

```

GeoSPARQL does not restrict the cardinality of the `geo:hasBoundingBox` property. A Feature may be associated with more than one bounding-box, for example in different coordinate reference systems.

3.4.4. Property: `geo:hasCentroid`

The property `geo:hasCentroid` is used to link a Feature with a point geometry corresponding with the centroid of its geometry. The centroid is typically used to show location on a low-resolution image, and for some indexing and discovery functions.

```

geo:hasCentroid
  a rdf:Property, owl:ObjectProperty ;
  rdfs:isDefinedBy geo: ;
  rdfs:subPropertyOf geo:hasGeometry ;
  rdfs:domain geo:Feature ;
  rdfs:range geo:Geometry ;
  skos:prefLabel "has centroid"@en ;
  skos:definition "The arithmetic mean position of all the geometry points
of a given Feature."@en ;
  skos:scopeNote "The target geometry shall describe a point, e.g. sf:Point"@en ;
.

```

GeoSPARQL does not restrict the cardinality of the `geo:hasCentroid` property. A Feature may be associated with more than one centroid, for example computed using different rules or in different coordinate reference systems.

Chapter 4. Topology Vocabulary Extension

This clause establishes the *Topology Vocabulary Extension* parameterized Requirements class. The IRI base is [/req/topology-vocab-extension](#), which has a single corresponding Conformance Class *Topology Vocabulary Extension*, with IRI [/conf/topology-vocab-extension](#). This Requirements class defines a vocabulary for asserting and querying topological relations between spatial objects. The class is parameterized so that different families of topological relations may be used, such as RCC9 [\[rcc9\]](#) and Egenhofer. These relations are generalized so that they may connect features as well as geometries.

Example 8. Topology Vocabulary Extension

requirement

[/req/topology-vocab-extension/sf-spatial-relations](#)

requirement

[/req/topology-vocab-extension/eh-spatial-relations](#)

requirement

[/req/topology-vocab-extension/rcc8-spatial-relations](#)

A Dimensionally Extended 9-Intersection Model [\[DE-9IM\]](#) pattern, which specifies the spatial dimension of the intersections of the interiors, boundaries and exteriors of two geometric objects, is used to describe each spatial relation. Possible pattern values are **-1** (empty), **0**, **1**, **2**, **T** (true) = {**0**, **1**, **2**}, **F** (false) = {-1}, ***** (don't care) = {-1, **0**, **1**, **2**}. In the following descriptions, the notation **X/Y** is used to denote applying a spatial relation to geometry types **X** and **Y** (i.e., **x relation y** where **x** is of type **X** and **y** is of type **Y**). The symbol **P** is used for 0-dimensional geometries (e.g. points). The symbol **L** is used for 1-dimensional geometries (e.g. lines), and the symbol **A** is used for 2-dimensional geometries (e.g. polygons). Consult the Simple Features specification [\[OGCSFACA\]](#) [\[ISO19125-1\]](#) for a more detailed description of DE-9IM intersection patterns.

4.1. Parameters

The following parameter is defined for the *Topology Vocabulary Extension* Requirements.

relation_family: Specifies the set of topological spatial relations to support.

4.2. Simple Features Relation Family

This clause defines Requirements for the *Simple Features* relation family.

Example 9. Simple Feature Spatial Relations

Implementations shall allow the properties [geo:sfEquals](#), [geo:sfDisjoint](#), [geo:sfIntersects](#), [geo:sfTouches](#), [geo:sfCrosses](#), [geo:sfWithin](#), [geo:sfContains](#) and [geo:sfOverlaps](#) to be used in SPARQL graph patterns.

Topological relations in the *Simple Features* family are summarized in [Simple Features Topological Relations](#). Multi-row intersection patterns should be interpreted as a logical **OR** of each row.

Simple Features Topological Relations

Relation Name	Relation IRI	Domain/Range	Applies To Geometry Types	DE-9IM Intersection Pattern
equals	geo:sfEquals	geo:SpatialObject	All	(TFFFTFFFT)
disjoint	geo:sfDisjoint	geo:SpatialObject	All	(FF**FF****)
intersects	geo:sfIntersects	geo:SpatialObject	All	(T***** *T***** ***T***** ****T****)
touches	geo:sfTouches	geo:SpatialObject	All except P/P	(FT***** F**T***** F***T****)
within	geo:sfWithin	geo:SpatialObject	All	(T*F**F***)
contains	geo:sfContains	geo:SpatialObject	All	(T*****F*)
overlaps	geo:sfOverlaps	geo:SpatialObject	A/A, P/P, L/L	(T*T***T**) for A/A, P/P; (1*T***T**) for L/L
crosses	geo:sfCrosses	geo:SpatialObject	P/L, P/A, L/A, L/L	(T*T***T**) for P/L, P/A, L/A; (0*****F*) for L/L

4.3. Egenhofer Relation Family

This clause defines Requirements for the 9-intersection model for the binary topological relations (*Egenhofer*) relation family. The reader should consult references [\[FORMAL\]](#) and [\[CATEG\]](#) for a more detailed discussion of *Egenhofer* relations.

Example 10. Egenhofer Spatial Relations

Implementations shall allow the properties `geo:ehEquals`, `geo:ehDisjoint`, `geo:ehMeet`, `geo:ehOverlap`, `geo:ehCovers`, `geo:ehCoveredBy`, `geo:ehInside` and `geo:ehContains` to be used in SPARQL graph patterns.

Topological relations in the *Egenhofer* family are summarized in [Egenhofer Topological Relations](#). Multi-row intersection patterns should be interpreted as a logical **OR** of each row.

Egenhofer Topological Relations

Relation Name	Relation IRI	Domain/Range	Applies To Geometry Types	DE-9IM Intersection Pattern
equals	geo:ehEquals	geo:SpatialObject	All	(TFFFTFFFT)
disjoint	geo:ehDisjoint	geo:SpatialObject	All	(FF*FF****)
meet	geo:ehMeet	geo:SpatialObject	All except P/P	(FT***** F**T***** F***T****)
overlap	geo:ehOverlap	geo:SpatialObject	All	(T*T***T**)
covers	geo:ehCovers	geo:SpatialObject	A/A, A/L, L/L	(T*TFT*FF*)
covered by	geo:ehCoveredBy	geo:SpatialObject	A/A, L/A, L/L	(TFF*TFT**)
inside	geo:ehInside	geo:SpatialObject	All	(TFF*FFT**)
contains	geo:ehContains	geo:SpatialObject	All	(T*TFF*FF*)

4.4. RCC8 and RCC*-9 Relation Family

This clause defines Requirements for the region connection calculus basic 8 and 9 (*RCC8*, *RCC*-9*) relation family. The reader should consult references [\[QUAL\]](#) and [\[LOGIC\]](#) for a more detailed discussion of *RCC8* and [\[RCC9\]](#) for a discussion of *RCC*-9* relations.

Example 11. RCC8 Spatial Relations

Implementations shall allow the properties `geo:rcc8eq`, `geo:rcc8dc`, `geo:rcc8ec`, `geo:rcc8po`, `geo:rcc8tppi`, `geo:rcc8tpp`, `geo:rcc8ntpp`, `geo:rcc8ntppi` to be used in SPARQL graph patterns.

Topological relations in the *RCC8* family are summarized in [RCC8 Topological Relations](#).

Example 12. RCC*-9 Spatial Relations

Implementations shall allow the properties `geo:rcc9eq`, `geo:rcc9dc`, `geo:rcc9ec`, `geo:rcc9po`, `geo:rcc9tppi`, `geo:rcc9tpp`, `geo:rcc9ntpp`, `geo:rcc9ntppi`, `geo:rcc9cr`, to be used in SPARQL graph patterns.

Topological relations in the *RCC*-9* family are summarized in [RCC*-9 Topological Relations](#).

RCC8 Topological Relations

Relation Name	Relation IRI	Domain/Range	Applies To Geometry Types	DE-9IM Intersection Pattern
equals	geo:rcc8eq	geo:SpatialObject	A/A	(TFFFTFFFT)
disconnected	geo:rcc8dc	geo:SpatialObject	A/A	(FFTFFTTTT)

Relation Name	Relation IRI	Domain/Range	Applies To Geometry Types	DE-9IM Intersection Pattern
externally connected	geo:rcc8ec	geo:SpatialObject	A/A	(FFFTTTTT)
partially overlapping	geo:rcc8po	geo:SpatialObject	A/A	(TTTTTTTT)
tangential proper part inverse	geo:rcc8tpi	geo:SpatialObject	A/A	(TTFTTFFT)
tangential proper part	geo:rcc8tp	geo:SpatialObject	A/A	(TFFTFTTT)
non-tangential proper part	geo:rcc8ntpp	geo:SpatialObject	A/A	(TFFTFFTT)
non-tangential proper part inverse	geo:rcc8ntppi	geo:SpatialObject	A/A	(TTFFTFFT)

RCC-9 Topological Relations*

Relation Name	Relation IRI	Domain/Range	Applies To Geometry Types	DE-9IM Intersection Pattern
equals	geo:rcc9eq	geo:SpatialObject	A/A	(TTTT*FFFT)
disconnected	geo:rcc9dc	geo:SpatialObject	A/A	(FF*FF**)
externally connected	geo:rcc9ec	geo:SpatialObject	A/A	(F*TTT) v (FTTT) v (F*T*T*T*)
partially overlapping	geo:rcc9po	geo:SpatialObject	A/A	(TTTT) v (T*T*T*T**)
tangential proper part inverse	geo:rcc9tpi	geo:SpatialObject	A/A	(TFF) v (**T*FF*) v ¬(TFFTFFFT)
tangential proper part	geo:rcc9tp	geo:SpatialObject	A/A	(TFF) v (F*TF*) v ¬(TFFTFFFT)
non-tangential proper part	geo:rcc9ntpp	geo:SpatialObject	A/A	(FF*FFT*)
non-tangential proper part inverse	geo:rcc9ntppi	geo:SpatialObject	A/A	(*TFF*FF)
crosses	geo:rcc9cr	geo:SpatialObject	A/A	(TTTT*FFFT)

4.5. Equivalent RCC8, Egenhofer and Simple Features Topological Relations

[Equivalent Simple Features, RCC8 and Egenhofer relations](#) summarizes the equivalences between *Egenhofer*, *RCC8* and *Simple Features* spatial relations for closed, non-empty regions. The symbol + denotes logical OR, and the symbol ¬ denotes negation.

Equivalent Simple Features, RCC8 and Egenhofer relations

Simple Features	RCC9	Egenhofer
equals	equals	equals
disjoint	disconnected	disjoint
intersects	¬ disconnected	¬ disjoint
touches	externally connected	meet
within	non-tangential proper part + tangential proper part	inside + covered by
contains	non-tangential proper part inverse + tangential proper part inverse	contains + covers
overlaps	partially overlapping	overlap

Chapter 5. Geometry Extension

This clause defines the *Geometry Extension* parameterized Requirements class with the base IRI `/req/geometry-extension`. There is a single corresponding conformance class *Geometry Extension*, with the IRI `/conf/geometry-extension`. These Requirements define a vocabulary for asserting and querying information about geometry data, and define query functions for operating on geometry data.

Example 13. Geometry Extension

requirement

`/req/geometry-extension/geometry-class`

requirement

`/req/geometry-extension/geometry-collection-class`

requirement

`/req/geometry-extension/feature-properties`

requirement

`/req/geometry-extension/geometry-properties`

requirement

`/req/geometry-extension/query-functions`

requirement

`/req/geometry-extension/srid-function`

requirement

`/req/geometry-extension/query-functions-3D`

requirement

`/req/geometry-extension/sa-functions`

requirement

`/req/geometry-extension/wkt-literal`

requirement

`/req/geometry-extension/wkt-literal-default-srs`

requirement

`/req/geometry-extension/wkt-axis-order`

requirement

`/req/geometry-extension/wkt-literal-empty`

requirement

`/req/geometry-extension/geometry-as-wkt-literal`

requirement

/req/geometry-extension/asWKT-function

requirement

/req/geometry-extension/gml-literal

requirement

/req/geometry-extension/gml-literal-empty

requirement

/req/geometry-extension/gml-profile

requirement

/req/geometry-extension/geometry-as-gml-literal

requirement

/req/geometry-extension/asGML-function

requirement

/req/geometry-extension/geojson-literal

requirement

/req/geometry-extension/geojson-literal-srs

requirement

/req/geometry-extension/geojson-literal-empty

requirement

/req/geometry-extension/geometry-as-geojson-literal

requirement

/req/geometry-extension/asGeoJSON-function

requirement

/req/geometry-extension/kml-literal

requirement

/req/geometry-extension/kml-literal-srs

requirement

/req/geometry-extension/kml-literal-empty

requirement

/req/geometry-extension/geometry-as-kml-literal

requirement

/req/geometry-extension/asKML-function

requirement

/req/geometry-extension/geocode-literal

requirement

/req/geometry-extension/geocode-literal-srs

requirement

/req/geometry-extension/geocode-literal-empty

requirement

/req/geometry-extension/geometry-as-geocode-literal

requirement

/req/geometry-extension/asGeocode-function

As part of the vocabulary, RDFS datatypes are defined for encoding detailed geometry information as a literal value. A literal representation of a geometry is needed so that geometric values may be treated as a single unit. Such a representation allows geometries to be passed to external functions for computations and to be returned from a query.

5.1. Rationale

Other schemes for encoding simple geometry data in RDF have been implemented. The W3C Basic Geo vocabulary^[1] was an early (2003) RDF vocabulary for "representing lat(itude), long(itude) and other information about spatially-located things. Geo specifies WGS84 as the reference datum". Further, many widely used Semantic Web vocabularies contain some spatial data support. For example, *Dublin Core Terms* provides a *Location* class^[2] for "A spatial region or named place." and *schema.org* provides a number of spatial object and geometry classes, such as *GeoCoordinates*^[3] and *GeoShape*^[4].

Many vocabularies such as the above provide little specific support for detailed geometries and only specify using the WGS84 Coordinate Reference System (CRS).

Since the first version of GeoSPARQL, many ontologies have imported GeoSPARQL. For example, the *ISA Programme Location Core Vocabulary*^[5] whose usage notes provide examples containing GeoSPARQL literals and the use of GeoSPARQL's "geometry class". The W3C's more recent *Data Catalog Vocabulary, Version 2* (DCAT2) standard^[6] similarly contains usage notes for *geometry*, *bbox* and other properties that suggest the use of GeoSPARQL literals.

Some of the properties defined in these vocabularies, such as DCAT2's *dcat:spatialResolution* have motivated the inclusion of new properties in this version of GeoSPARQL. In this case the equivalent property is *geo:hasSpatialResolution*. The GeoSPARQL 1.1 Standards Working Group charter [CHARTER] contains references to a number of vocabularies/ontologies that were influential in the generation of this version of GeoSPARQL.

5.2. GeoSPARQL and Simple Features (SFA-CA)

The GeoSPARQL Geometry Extension is largely based on the ISO/OGC Simple Features Access - Common Architecture (SFA-CA) Standard [OGCSFACA]. Contrary to what the name may imply, SFA-CA is about Geometry and not about Features. SFA-CA describes simple geometry, meaning that geometric shapes are based on points and straight lines (linear interpolations) between points. Within a single Geometry, these lines may not cross.

Neither GeoSPARQL nor SFA-CA support full three-dimensional geometry. Coordinates may be three-dimensional, which means that points may have a Z-coordinate next to an X- and Y-coordinate. The Z-coordinate then holds the value of height or depth. However, lines or surfaces can only have one Z value for any explicit or interpolated X,Y pair. This approach is often referred to as 2.5 dimensional geometry. Geometric functions working with Geometries that have Z values will ignore Z values in calculations and first project geometry onto the Z=0 level.

SFA-CA also describes M coordinate values that may be part of geometry encodings. The M value represents a measure, a value that can be used in information systems that support linear referencing. GeoSPARQL at the moment does not support linear referencing. Like Z values in coordinates, M values are to be ignored.

SFA-CA specifies a class hierarchy for Geometry. Although these classes are not part of the GeoSPARQL ontology, the GeoSPARQL SWG does publish a vocabulary of Simple Features geometry: <http://www.opengis.net/ont/sf>. Geometry types defined in this vocabulary can be considered safe to use with GeoSPARQL. The two Geometry serializations that were specified in GeoSPARQL 1.0, WKT and GML, fully support all SFA-CA geometry types. However, the two Geometry serializations that were introduced in GeoSPARQL 1.1 do not. Some SFA-CA geometry types are not supported by either the OGC KML [OGC12-007r2] or the GeoJSON format. For example, neither KML nor GeoJSON support the Triangulated Integrated Network (TIN) or Triangle geometry types.

5.3. Recommendation for units of measure

For geometric data to be interpreted and used correctly, the units of measure should be known. Typically, the particular Spatial Reference System (SRS) that is associated with a Geometry instance will specify a unit of measurement. However, some elements of GeoSPARQL allow arbitrary units of distance to be used, for example the property `geo:hasSpatialResolution` or the function `geof:buffer`. In those cases it is advisable to make use of a well-known web vocabulary for units of measurement. Making the unit of measurement explicit will improve data interoperability. The recommended vocabulary for units of measurement for GeoSPARQL is the *Quantities, Units, Dimensions and Types (QUDT)* ontology^[7] but others may be used, as long as they are well-described.

5.4. Influence of Reference Systems on computations

A Geometry object consists of a set of coordinates and a specification on how the coordinates should be interpreted. This specification is known as a Spatial reference System (SRS). Taken together, coordinates and SRS allow performing computations on Geometry objects. For example, sizes can be calculated or new Geometry objects can be created. Some Spatial Reference Systems describe a two-dimensional flat space. In that case, coordinates are understood to be Cartesian, and

Cartesian geometric computations can be performed. But Spatial Reference Systems can describe other types of spaces, to which Cartesian computations are not applicable. For example, if CRS <http://www.opengis.net/def/crs/OGC/1.3/CRS84> is used, coordinates are to be interpreted as decimal degrees of latitude and longitude, designating positions on a spheroid. The distance between two points using this CRS is different from the distance between two points that have the same coordinates but are based on a Cartesian CRS or other SRS.

To avoid erroneous computations involving Geometry, data publishers are recommended to clearly indicate the type of space that is described by the SRS.

5.5. Parameters

The following parameters are defined for the *Geometry Extension* Requirements.

serialization

Specifies the serialization standard to use when generating geometry literals as well as the supported geometry types.

NOTE

A serialization strongly affects the geometry conceptualization. The WKT serialization aligns the geometry types with *ISO 19125 Simple Features* [\[OGCSFACA\]](#) [\[ISO19125-1\]](#); the GML serialization aligns the geometry types with *ISO 19107 Spatial Schema* [\[ISO19107\]](#).

version

Specifies the version of the serialization format used.

5.6. Geometry Class

A single root geometry class is defined: `geo:Geometry`. In addition, properties are defined for describing geometry data and for associating geometries with features.

One container class is defined: [Geometry Collection](#).

5.6.1. Class: `geo:Geometry`

The class `geo:Geometry` is conceptually derived from UML class `Geometry` in [\[ISO19107\]](#) which is that standard's "root class of the geometric object taxonomy and supports interfaces common to all geographically referenced geometric objects". `geo:Geometry` is defined by the following:

```
geo:Geometry
  a rdfs:Class, owl:Class ;
  rdfs:isDefinedBy geo: ;
  skos:prefLabel "Geometry"@en ;
  rdfs:subClassOf geo:SpatialObject ;
  owl:disjointWith geo:Feature;
  skos:definition "A coherent set of direct positions in space. The positions
                  are held within a Spatial Reference System (SRS)."@en ;
  skos:note "Geometry can be used as a representation of the shape, extent or
```


location of a Feature and may exist as a self-contained entity."@en ;

.

Example 14. Geometry Class

Implementations shall allow the RDFS class `geo:Geometry` to be used in SPARQL graph patterns.

5.6.2. Class: `geo:GeometryCollection`

The class `GeometryCollection` is defined by the following:

```
geo:GeometryCollection
  a owl:Class ;
  rdfs:isDefinedBy geo: ;
  skos:prefLabel "Geometry Collection"@en ;
  skos:definition "A collection of individual Geometries."@en ;
  rdfs:subClassOf geo:SpatialObjectCollection , geo:Geometry ;
  rdfs:subClassOf [
    a owl:Restriction ;
    owl:allValuesFrom geo:Geometry ;
    owl:onProperty rdfs:member ;
  ] ;
.
```

This class is indicated to be a collection of spatial objects by being a subclass of the general `Spatial Object Collection`. That it contains only `Geometry` class instances is indicated by the restriction `rdfs:member` predicate. That it can be used as a `Geometry`, for example as the range value for the `has geometry` predicate, is indicated by it being a subclass of the `Geometry` class.

There is no RDF/ontology relationship between this `geo:GeometryCollection` class and the Simple Features Vocabulary's `sf:GeometryCollection` class since the former is a collection of `geo:Geometry` objects and the latter is to be used for compound geometry literals.

`sf:GeometryCollection` instances can act as input or output of GeoSPARQL functions whereas `geo:GeometryCollection` instances are more likely to be used for grouping `geo:Geometry` objects for other purposes.

NOTE

Many geometry literal formats also have the ability to represent multiple geometries. Both the OGC Geography Markup Language (GML) and KML use a *MultiGeometry* type and Well Known Text (WKT) and GeoJSON use a *GeometryCollection* type. While the names of some of these objects are the same as this class' and all the concepts are similar, there is also no RDF/ontology relationship between this class and these literals. This class contains whole `geo:Geometry` instances, which may have more information within them than just a geometry serialization.

As per the expected use of `sf:GeometryCollection` instances mentioned above, the uses of multi-geometry literals and `geo:GeometryCollection` instances is expected to be different too.

Example 15. Geometry Collection Class

Implementations shall allow the RDFS class `geo:GeometryCollection` to be used in SPARQL graph patterns.

5.7. Standard Properties for geo:Geometry

Properties are defined for describing geometry metadata.

Example 16. Geometry Properties

Implementations shall allow the properties `geo:dimension`, `geo:coordinateDimension`, `geo:spatialDimension`, `geo:hasSpatialResolution`, `geo:hasMetricSpatialResolution`, `geo:hasSpatialAccuracy`, `geo:hasMetricSpatialAccuracy`, `geo:isClosed`, `geo:isEmpty`, `geo:isRing`, `geo:isSimple` and `geo:hasSerialization` to be used in SPARQL graph patterns.

5.7.1. Property: geo:dimension

The property `geo:dimension` is used to link a Geometry object to its topological dimension, which must be less than or equal to the coordinate dimension. In non-homogeneous collections, this will return the largest topological dimension of the contained objects.

```
geo:dimension
  a rdf:Property, owl:DatatypeProperty ;
  rdfs:isDefinedBy geo: ;
  skos:prefLabel "dimension"@en ;
  skos:definition "The topological dimension of this geometric object, which
                  must be less than or equal to the coordinate dimension. In
                  non-homogeneous collections, this is the largest
                  topological dimension of the contained objects."@en ;
  rdfs:domain geo:Geometry ;
  rdfs:range xsd:integer ;
  .
```

5.7.2. Property: geo:coordinateDimension

The property `geo:coordinateDimension` is defined to link a Geometry object to the dimension of direct positions (coordinate tuples) used in the Geometry's definition.

```
geo:coordinateDimension
  a rdf:Property, owl:DatatypeProperty;
  rdfs:isDefinedBy geo: ;
```

```

skos:prefLabel "coordinate dimension"@en ;
skos:definition "The number of measurements or axes needed to describe the
                  position of this Geometry in a coordinate system."@en ;
rdfs:domain geo:Geometry ;
rdfs:range xsd:integer ;
.

```

5.7.3. Property: `geo:spatialDimension`

The property `geo:spatialDimension` is defined to link a Geometry object to the dimension of the spatial portion of the direct positions (coordinate tuples) used in its serializations. If the direct positions do not carry a measure coordinate, this will be equal to the coordinate dimension.

```

geo:spatialDimension
  a rdf:Property, owl:DatatypeProperty;
  rdfs:isDefinedBy geo: ;
  skos:prefLabel "spatial dimension"@en ;
  skos:definition "The number of measurements or axes needed to describe the
                  spatial position of this Geometry in a coordinate system."@en ;
  rdfs:domain geo:Geometry ;
  rdfs:range xsd:integer ;
.

```

5.7.4. Property: `geo:hasSpatialResolution`

The property `geo:hasSpatialResolution` is defined to indicate the spatial resolution of the elements within a Geometry. Spatial resolution specifies the level of detail of a Geometry. It is the smallest distinguishable distance between adjacent coordinate sets. This property is not applicable to a point Geometry, because a point consists of a single coordinate set.

Since this property is defined for a `geo:Geometry`, all literal representations of that Geometry instance must have the same spatial resolution.

```

geo:hasSpatialResolution
  a rdf:Property, owl:ObjectProperty;
  rdfs:isDefinedBy geo: ;
  skos:prefLabel "has spatial resolution"@en ;
  skos:definition "The spatial resolution of a Geometry"@en ;
  rdfs:domain geo:Geometry ;
.

```

NOTE | See the [Section 5.3](#).

5.7.5. Property: `geo:hasMetricSpatialResolution`

The property `geo:hasMetricSpatialResolution` is similar to `geo:hasSpatialResolution`, except that the

unit of resolution is always meter (the standard distance unit of the International System of Units).

```
geo:hasMetricSpatialResolution
  a rdf:Property, owl:ObjectProperty;
  rdfs:isDefinedBy geo: ;
  skos:prefLabel "has spatial resolution in meters"@en ;
  skos:definition "The spatial resolution of a Geometry in meters."@en ;
  rdfs:domain geo:Geometry ;
  rdfs:range xsd:double ;
.
```

5.7.6. Property: `geo:hasSpatialAccuracy`

The property `geo:hasSpatialAccuracy` is applicable when a Geometry is used to represent a Feature. It is expressed as a distance that indicates the truthfulness of the positions (coordinates) that define the Geometry. In this case accuracy defines a zone surrounding each coordinate within which the real positions are known to be. The accuracy value defines this zone as a distance from the coordinate(s) in all directions (e.g. a line, a circle or a sphere, depending on spatial dimension).

```
geo:hasSpatialAccuracy
  a rdf:Property, owl:ObjectProperty;
  rdfs:isDefinedBy geo: ;
  skos:prefLabel "has spatial accuracy"@en ;
  skos:definition "The positional accuracy of the coordinates of a Geometry."@en ;
  rdfs:domain geo:Geometry ;
.
```

NOTE See the [Section 5.3](#).

5.7.7. Property: `geo:hasMetricSpatialAccuracy`

The property `geo:hasMetricSpatialAccuracy` is similar to `has spatial accuracy`, but is easier to specify and use because the unit of distance is always meter (the standard distance unit of the International System of Units).

```
geo:hasMetricSpatialAccuracy
  a rdf:Property, owl:ObjectProperty;
  rdfs:isDefinedBy geo: ;
  skos:prefLabel "has spatial accuracy in meters"@en ;
  skos:definition "The positional accuracy of the coordinates of a Geometry in meters."@en ;
  rdfs:domain geo:Geometry ;
  rdfs:range xsd:double ;
.
```

5.7.8. Property: `geo:isClosed`

The property `geo:isClosed` will indicate a Boolean object set to `true` if and only if the Geometry is closed, i.e. its start and end point are the same.

```
geo:isClosed
  a rdf:Property, owl:DatatypeProperty ;
  rdfs:isDefinedBy geo: ;
  skos:prefLabel "is closed"@en ;
  skos:definition "(true) if this geometric object is closed. If
                    true, then this geometric object's start point equals its end
point."@en ;
  rdfs:domain geo:Geometry ;
  rdfs:range xsd:boolean ;
  .
```

5.7.9. Property: `geo:isEmpty`

The property `geo:isEmpty` will indicate a Boolean object set to `true` if and only if the Geometry contains no information.

```
geo:isEmpty
  a rdf:Property, owl:DatatypeProperty ;
  rdfs:isDefinedBy geo: ;
  skos:prefLabel "is empty"@en ;
  skos:definition "(true) if this geometric object is the empty Geometry. If
                    true, then this geometric object represents the empty point
                    set for the coordinate space."@en ;
  rdfs:domain geo:Geometry ;
  rdfs:range xsd:boolean ;
  .
```

5.7.10. Property: `geo:isRing`

The property `geo:isRing` will indicate a Boolean object set to `true` if and only if the Geometry is closed and simple.

```
geo:isRing
  a rdf:Property, owl:DatatypeProperty ;
  rdfs:isDefinedBy geo: ;
  skos:prefLabel "is ring"@en ;
  skos:definition "(true) if this geometric object is a ring. If
                    true, then this geometric object's start point equals its end
point, i.e. it is closed and it is simple, i.e. is has no self-intersections."@en ;
  rdfs:domain geo:Geometry ;
  rdfs:range xsd:boolean ;
  .
```

5.7.11. Property: `geo:isSimple`

The property `geo:isSimple` will indicate a Boolean object set to `true` if and only if the Geometry contains no self-intersections, with the possible exception of its boundary.

```
geo:isSimple
  a rdf:Property, owl:DatatypeProperty ;
  rdfs:isDefinedBy geo: ;
  skos:prefLabel "is simple"@en ;
  skos:definition "(true) if this geometric object has no anomalous geometric
                  points, such as self intersection or self tangency."@en ;
  rdfs:domain geo:Geometry ;
  rdfs:range xsd:boolean ;
.
```

5.7.12. Property: `geo:hasSerialization`

The property `geo:hasSerialization` is defined to connect a Geometry with its text-based serialization.

It can be used to indicate a literal value, for example a GeoJSON `geo:geoJSONLiteral` or Well Known-Text `ge:wktLiteral` value, or it can be used to indicate a `dcat:Distribution` object which, as per that class' definition, is "A specific representation of a Dataset..." where, within GeoSPARQL, the Dataset is the Geometry.

```
geo:hasSerialization
  a rdf:Property, owl:DatatypeProperty ;
  rdfs:isDefinedBy geo: ;
  skos:prefLabel "has serialization"@en ;
  skos:definition "Connects a Geometry object with its text-based serialization."@en
;
  rdfs:domain geo:Geometry ;
  rdfs:range rdfs:Resource ;
.
```

This property is the generic property used to connect a Geometry with its serialization. GeoSPARQL also contains a number of sub properties of this property for connecting literal serializations of common types with geometries, for example as [GeoJSON](#) which can be used for [GeoJSON](#) literals.

NOTE

If used to indicate a literal value, the sub properties are preferred.

If used to indicate a `dcat:Distribution` object, the format and/or the conformance of that distribution to a data model or standard should be given as per DCAT Distribution norms, i.e. use of `dcterms:format` and `dcterms:conformsTo` predicates.

`dcat:accessURL` is the preferred predicate to indicate a file location or URL that links to the distribution data.

See the [Distributions](#) example.

5.8. Geometry Serializations

This section establishes the Requirements class for representing Geometry data in RDF literals, according to different non-RDF systems.

GeoSPARQL presents specializations of the `geo:hasSerialization` property for indicating particular serializations and specialized datatype literals for containing them. It does not provide comprehensive definitions of their content since these are given in standards external to GeoSPARQL, all of which are referenced.

GeoSPARQL does present some Requirements for literal structure which extend the serialization-defining standards, for example the requirement to allow indications of spatial reference systems within WKT geometry representations.

GeoSPARQL's expectation of RDF literal representations of geometry data is that it is related to the *Simple Features Access* (SFA) [\[OGCSFACA\]](#) [\[ISO19125-1\]](#) standard's conceptualization of geometry which defines classes such as `Point`, `Curve` and `Surface` and specialized variants of them which it presents in a hierarchy. All SFA classes are represented in OWL in the *Simple Features Vocabulary* presented within GeoSPARQL as an independent profile element, see [GeoSPARQL Standard structure](#).

Some geometry representation systems given here do not use the same terminology as SFA, in particular Discrete Global Grid Systems. To know the extent to which geometry literal representations listed here support SFA, or map to SFA, please see their definitions.

5.8.1. Well-Known Text

This section establishes the requirements for representing Geometry data in RDF based on Well-Known Text (WKT) as defined by *Simple Features Access* [\[OGCSFACA\]](#) [\[ISO19125-1\]](#). It defines one RDFS Datatype: `WKT Literal` and one property, as `WKT`.

5.8.1.1. RDFS Datatype: `geo:wktLiteral`

The datatype `geo:wktLiteral` is used to contain the Well-Known Text (WKT) serialization of a Geometry.

```
geo:wktLiteral
  a rdfs:Datatype ;
  rdfs:isDefinedBy geo: ;
  skos:prefLabel "Well-known Text literal"@en ;
  skos:definition "A Well-known Text serialization of a Geometry object."@en ;
  .
```

Example 17. WKT Literal

All RDFS Literals of type `geo:wktLiteral` shall consist of an optional IRI identifying the coordinate reference system and a required Well Known Text (WKT) description of a geometric value. Valid `geo:wktLiteral` instances are formed by either a WKT string as defined in [ISO13249] or by concatenating a valid absolute IRI, as defined in [IETF3987], enclosed in angled brackets (< & >) followed by whitespace as a separator, and a WKT string as defined in [ISO13249].

The following ABNF [IETF5234] syntax specification formally defines this literal:

```
wktLiteral ::= opt-iri-and-whitespace geometry-data  
  
opt-iri-and-space = "<" IRI ">" LWSP / ""
```

The token `opt-iri-and-whitespace` may be either an IRI and whitespace (spaces, tabs, newlines) or nothing (""), the token `IRI` (Internationalized Resource Identifier) is essentially a web address and is defined in [IETF3987] and the token `LWSP`, is one or more white space characters, as defined in [IETF5234]. `geometry-data` is the Well-Known Text representation of the Geometry, defined in [ISO13249].

In the absence of a leading spatial reference system IRI, the following spatial reference system IRI will be assumed: `<http://www.opengis.net/def/crs/OGC/1.3/CRS84>`. This IRI denotes WGS 84 longitude-latitude.

Example 18. WKT Literal Default SRS

The IRI `<http://www.opengis.net/def/crs/OGC/1.3/CRS84>` shall be assumed as the spatial reference system for `geo:wktLiteral` instances that do not specify an explicit spatial reference system IRI.

The OGC maintains a set of SRS IRIs under the `http://www.opengis.net/def/crs/` namespace and IRIs from this set are recommended for use. However others may also be used, as long as they are valid IRIs.

Example 19. WKT Literal Axis Order

Coordinate tuples within `geo:wktLiteral` shall be interpreted using the axis order defined in the spatial reference system used.

The example `WKT Literal` below encodes a point Geometry using the default WGS84 geodetic longitude-latitude spatial reference system:

```
"Point(-83.38 33.95)"^^<http://www.opengis.net/ont/geosparql#wktLiteral>
```


A second example below encodes the same point as encoded in the example above but using a SRS identified by <http://www.opengis.net/def/crs/EPSG/0/4326>: a WGS 84 geodetic latitude-longitude spatial reference system (note that this spatial reference system defines a different axis order):

```
"<http://www.opengis.net/def/crs/EPSG/0/4326> Point(33.95
-83.38)"^^<http://www.opengis.net/ont/geosparql#wktLiteral>
```

Example 20. Empty WKT Literal

An empty RDFS Literal of type `geo:wktLiteral` shall be interpreted as an empty Geometry.

5.8.1.2. Property: `geo:asWKT`

The property `geo:asWKT` is defined to link a Geometry with its WKT serialization.

Example 21. `asWKT` Property

Implementations shall allow the RDF property `geo:asWKT` to be used in SPARQL graph patterns.

```
geo:asWKT
  a rdf:Property, owl:DatatypeProperty ;
  rdfs:subPropertyOf geo:hasSerialization ;
  rdfs:isDefinedBy geo: ;
  skos:prefLabel "as WKT"@en ;
  skos:definition "The WKT serialization of a Geometry."@en ;
  rdfs:domain geo:Geometry ;
  rdfs:range geo:wktLiteral ;
.
```

5.8.1.3. Function: `geof:asWKT`

```
geof:asWKT (geom: ogc:geomLiteral): geo:wktLiteral
```

The function `geof:asWKT` converts `geom` to an equivalent WKT representation preserving the spatial reference system.

Example 22. `asWKT` Function

Implementations shall support `geof:asWKT` as a SPARQL extension function.

5.8.2. Extended Well-Known Binary

This section establishes the requirements for representing Geometry data in RDF based on Extended Well-Known Binary (EWKB) as defined by *Simple Features Access* [OGCSFACA] [ISO19125-

1]. It defines the RDFS Datatype: [EWKB Literal](#) and one property, [as EWKB](#). The literal definition in addition, allows for 3D types present in the Extended Well-Known Binary specification.

5.8.2.1. RDFS Datatypes: `geo:ewkbLiteral`

The datatype `geo:ewkbLiteral` is used to contain the Extended Well-Known Binary (EWKB) serialization of a Geometry.

```
geo:ewkbLiteral
  a rdfs:Datatype ;
  rdfs:isDefinedBy geo: ;
  skos:prefLabel "(Extended) Well-known Binary literal"@en ;
  skos:definition "An (Extended) Well-known Binary serialization of a Geometry object."@en ;
  .
```

Req 1 All RDFS Literals of type `geo:ewkbLiteral` shall consist of an optional IRI identifying the coordinate reference system and a required Extended Well-Known Binary (EWKB) description of a geometric value encoded in Base64. Valid `geo:ewkbLiteral` instances are formed by a EWKB string as defined in [\[ISO13249\]](#) or by concatenating a valid absolute IRI, as defined in [\[IETF3987\]](#), enclosed in angled brackets (`<` & `>`) followed by whitespace as a separator and a EWKB string as defined in [\[ISO13249\]](#). The Extended Well-Known Binary description shall allow the dimensionality flags defined in the Extended Well-Known Binary specification to represent 3D geometries.

<http://www.opengis.net/spec/geosparql/1.0/req/geometry-extension/ewkb-literal>

The following *ABNF* [\[IETF5234\]](#) syntax specification formally defines this literal:

```
ewkbLiteral ::= opt-iri-and-whitespace geometry-data

opt-iri-and-space = "<" IRI ">" LWSP / ""
```

The token `opt-iri-and-whitespace` may be either an IRI and whitespace (spaces, tabs, newlines) or nothing (`""`), the token `IRI` (Internationalized Resource Identifier) is essentially a web address and is defined in [\[IETF3987\]](#) and the token `LWSP`, a binary representation in EWKB serialized as Base64. `geometry-data` is the Extended Well-Known Binary representation of the Geometry, defined in [\[ISO13249\]](#).

In the absence of a leading spatial reference system IRI, the following spatial reference system IRI will be assumed: `<http://www.opengis.net/def/crs/OGC/1.3/CRS84>`. This IRI denotes WGS 84 longitude-latitude.

Req 2 The IRI `<http://www.opengis.net/def/crs/OGC/1.3/CRS84>` shall be assumed as the spatial reference system for `geo:ewkbLiteral` instances that do not specify an explicit spatial reference system IRI. IRIs defined as SRIDs in an Extended Well-Known Binary format shall be ignored.

<http://www.opengis.net/spec/geosparql/1.0/req/geometry-extension/ewkb-literal-default-srs>

The OGC maintains a set of SRS IRIs under the <http://www.opengis.net/def/crs/> namespace, and IRIs from this set are recommended for use. However, others may also be used as long as they are valid IRIs.

Req 3 Coordinate tuples within `geo:ewkbLiteral` shall be interpreted using the axis order defined in the spatial reference system used.

<http://www.opengis.net/spec/geosparql/1.0/req/geometry-extension/ewkb-axis-order>

The example `EWKB Literal` below encodes a point Geometry using the default WGS84 geodetic longitude-latitude spatial reference system:

```
"Point(-83.38 33.95)"^^<http://www.opengis.net/ont/geosparql#ewkbLiteral>
```

A second example below encodes the same point as encoded in the example above but using a SRS identified by <http://www.opengis.net/def/SRS/EPSG/0/4326>: a WGS 84 geodetic latitude-longitude spatial reference system (note that this spatial reference system defines a different axis order):

```
"<http://www.opengis.net/def/crs/EPSG/0/4326> Point(33.95  
-83.38)"^^<http://www.opengis.net/ont/geosparql#ewkbLiteral>
```

Req 18 An empty RDFS Literal of type `geo:ewkbLiteral` shall be interpreted as an empty Geometry.

<http://www.opengis.net/spec/geosparql/1.0/req/geometry-extension/ewkb-literal-empty>

5.8.2.2. Property: `geo:asEWKB`

The property `geo:asEWKB` is defined to link a Geometry with its EWKB serialization.

Req 4 Implementations shall allow the RDF property `geo:asEWKB` to be used in SPARQL graph patterns.

<http://www.opengis.net/spec/geosparql/1.0/req/geometry-extension/geometry-as-wkb-literal>

```
geo:asEWKB
  a rdf:Property, owl:DatatypeProperty ;
  rdfs:subPropertyOf geo:hasSerialization ;
  rdfs:isDefinedBy geo: ;
  skos:prefLabel "as EWKB"@en ;
  skos:definition "The EWKB serialization of a Geometry."@en ;
  rdfs:domain geo:Geometry ;
  rdfs:range geo:ewkbLiteral ;
  .
```

5.8.2.3. Function: `geof:asEWKB`

```
geof:asEWKB (geom: ogc:geomLiteral): geo:ewkbLiteral
```

The function **geof:asEWKB** converts **geom** to an equivalent EWKB representation preserving the spatial reference system.

Req 5 Implementations shall support **geo:asEWKB** as a SPARQL extension function.

<http://www.opengis.net/spec/geosparql/1.1/req/geometry-extension/asEWKB-function>

5.8.3. Geography Markup Language

This section establishes a Requirements class for representing Geometry data in RDF based on GML as defined by the Geography Markup Language Encoding Standard [OGC07-036]. It defines one RDFS Datatype: **GML Literal** and one property, **as GML**.

5.8.3.1. RDFS Datatype: geo:gmlLiteral

The datatype **geo:gmlLiteral** is used to contain the Geography Markup Language (GML) serialization of a Geometry.

```
geo:gmlLiteral
  a rdfs:Datatype ;
  rdfs:isDefinedBy geo: ;
  skos:prefLabel "GML literal"@en ;
  skos:definition "The datatype of GML literal values"@en ;
  .
```

Valid **GML Literal** instances are formed by encoding Geometry information as a valid element from the GML schema that implements a subtype of **GM_Object**. For example, in GML 3.2.1 this is every element directly or indirectly in the substitution group of the element **{<http://www.opengis.net/ont/gml/3.2>}AbstractGeometry**. In GML 3.1.1 and GML 2.1.2 this is every element directly or indirectly in the substitution group of the element **{<http://www.opengis.net/ont/gml>}_Geometry**.

Example 23. GML Literal

All **geo:gmlLiteral** instances shall consist of a valid element from the GML schema that implements a subtype of **GM_Object** as defined in [OGC07-036].

The example **GML Literal** below encodes a point Geometry in the WGS 84 geodetic longitude-latitude spatial reference system using GML version 3.2:

```
"""
<gml:Point
  srsName=\"http://www.opengis.net/def/crs/OGC/1.3/CRS84\"
```

```
xmlns:gml=\"http://www.opengis.net/gml/3.2\">
  <gml:pos>-83.38 33.95</gml:pos>
</gml:Point>
\"\"\"^^<http://www.opengis.net/ont/geosparql#gmlLiteral>
```

Example 24. Empty GML Literal

An empty **geo:gmlLiteral** shall be interpreted as an empty Geometry.

Example 25. GML Profile

Implementations shall document supported GML profiles.

5.8.3.2. Property: **geo:asGML**

The property **geo:asGML** is defined to link a Geometry with its GML serialization.

Example 26. asGML Property

Implementations shall allow the RDF property **geo:asGML** to be used in SPARQL graph patterns.

```
geo:asGML
  a rdf:Property ;
  rdfs:subPropertyOf geo:hasSerialization ;
  rdfs:isDefinedBy geo: ;
  skos:prefLabel "as GML"@en ;
  skos:definition "The GML serialization of a Geometry."@en ;
  rdfs:domain geo:Geometry ;
  rdfs:range geo:gmlLiteral ;
.
```

5.8.3.3. Function: **geof:asGML**

```
geof:asGML (geom: ogc:geomLiteral, gmlProfile: xsd:string): geo:gmlLiteral
```

The function **geof:asGML** converts **geom** to an equivalent GML representation defined by a **gmlProfile** version string preserving the coordinate reference system.

Example 27. asGML Function

Implementations shall support **geof:asGML** as a SPARQL extension function.

5.8.4. GeoJSON

This section establishes a Requirements class for representing Geometry data in RDF based on Geographic JavaScript Object Notation (GeoJSON) as defined by [\[GEOJSON\]](#). It defines one RDFS Datatype: [GeoJSON Literal](#) and one property, [as GeoJSON](#).

5.8.4.1. RDFS Datatype: `geo:geoJSONLiteral`

The datatype `geo:geoJSONLiteral` is used to contain the GeoJSON serialization of a Geometry.

```
geo:geoJSONLiteral a rdfs:Datatype ;  
  rdfs:isDefinedBy geo: ;  
  skos:prefLabel "GeoJSON Literal"@en ;  
  skos:definition "A GeoJSON serialization of a Geometry object."@en .
```

Valid [GeoJSON Literal](#) instances are formed by encoding Geometry information as a Geometry object as defined in the GeoJSON specification [\[GEOJSON\]](#).

Example 28. GeoJSON Literal

All `geo:geoJSONLiteral` instances shall consist of the Geometry objects as defined in the GeoJSON specification [\[GEOJSON\]](#).

Example 29. GeoJSON Literal SRS

RDFS Literals of type `geo:geoJSONLiteral` do not contain a SRS definition. All literals of this type shall, according to the GeoJSON specification, be encoded only in, and be assumed to use, the WGS84 geodetic longitude-latitude spatial reference system (<http://www.opengis.net/def/crs/OGC/1.3/CRS84>).

The example [GeoJSON Literal](#) below encodes a point Geometry using the default WGS84 geodetic longitude-latitude spatial reference system for Simple Features 1.0:

```
""  
{ "type": "Point", "coordinates": [-83.38, 33.95] }  
""^^<http://www.opengis.net/ont/geosparql#geoJSONLiteral>
```

Example 30. Empty GeoJSON Literal

An empty RDFS Literal of type `geo:geoJSONLiteral` shall be interpreted as an empty Geometry, i.e. `{ "geometry": null }` in GeoJSON.

5.8.4.2. Property: `geo:asGeoJSON`

The property `geo:asGeoJSON` is defined to link a Geometry with its GeoJSON serialization.

Example 31. asGeoJSON Property

Implementations shall allow the RDF property `geo:asGeoJSON` to be used in SPARQL graph patterns.

```
geo:asGeoJSON
  a rdf:Property, owl:DatatypeProperty ;
  rdfs:subPropertyOf geo:hasSerialization ;
  rdfs:isDefinedBy geo: ;
  skos:prefLabel "as GeoJSON"@en ;
  skos:definition "The GeoJSON serialization of a Geometry."@en ;
  rdfs:domain geo:Geometry ;
  rdfs:range geo:geoJSONLiteral ;
  .
```

5.8.4.3. Function: geof:asGeoJSON

```
geof:asGeoJSON (geom: ogc:geomLiteral): geo:geoJSONLiteral
```

The function `geof:asGeoJSON` converts `geom` to an equivalent GeoJSON representation. Coordinates are converted to the CRS84 coordinate system, the only valid coordinate system to be used in a GeoJSON literal.

Example 32. asGeoJSON Function

Implementations shall support `geof:asGeoJSON` as a SPARQL extension function.

5.8.5. Keyhole Markup Language

This section establishes the Requirements class for representing Geometry data in RDF based on KML as defined by [OGC12-007r2]. It defines one RDFS Datatype: `KML Literal` and one property, `as KML`.

5.8.5.1. RDFS Datatype: geo:kmlLiteral

The datatype `geo:kmlLiteral` is used to contain the Keyhole Markup Language (KML) serialization of a Geometry.

```
geo:kmlLiteral
  a rdfs:Datatype ;
  rdfs:isDefinedBy geo: ;
  skos:prefLabel "KML Literal"@en ;
  skos:definition "A KML serialization of a Geometry object."@en ;
  .
```

Valid [KML Literal](#) instances are formed by encoding Geometry information as a Geometry object as defined in the KML specification [\[OGC12-007r2\]](#).

Example 33. KML Literal

All `geo:kmlliteral` instances shall consist of the Geometry objects as defined in the KML specification [\[OGC12-007r2\]](#).

Example 34. KML Literal SRS

RDFS Literals of type `geo:kmlliteral` do not contain a SRS definition. All literals of this type shall according to the KML specification only be encoded in and assumed to use the WGS84 geodetic longitude-latitude spatial reference system (<http://www.opengis.net/def/crs/OGC/1.3/CRS84>).

The example [KML Literal](#) below encodes a point Geometry using the default WGS84 geodetic longitude-latitude spatial reference system for Simple Features 1.0:

```
"""
<Point xmlns="http://www.opengis.net/kml/2.2">
  <coordinates>-83.38,33.95</coordinates>
</Point>
""""^<http://www.opengis.net/ont/geosparql#kmlliteral>
```

Example 35. Empty KML Literal

An empty RDFS Literal of type `geo:kmlliteral` shall be interpreted as an empty Geometry.

5.8.5.2. Property: `geo:asKML`

The property `geo:asKML` is defined to link a Geometry with its KML serialization.

Example 36. asKML Property

Implementations shall allow the RDF property `geo:asKML` to be used in SPARQL graph patterns.

The property [as KML](#) is used to link a geometric element with its KML serialization.

```
geo:asKML
  a rdf:Property, owl:DatatypeProperty;
  rdfs:subPropertyOf geo:hasSerialization ;
  rdfs:isDefinedBy geo: ;
  skos:prefLabel "as KML"@en ;
  skos:definition "The KML serialization of a Geometry."@en ;
  rdfs:domain geo:Geometry ;
  rdfs:range geo:kmlliteral ;
```


.

5.8.5.3. Function: geof:asKML

```
geof:asKML (geom: ogc:geomLiteral): geo:kmlLiteral
```

The function `geof:asKML` converts `geom` to an equivalent KML representation. Coordinates are converted to the CRS84 coordinate system, the only valid coordinate system to be used in a KML literal.

Example 37. asKML Function

Implementations shall support `geof:asKML` as a SPARQL extension function.

5.8.6. Geocode Literal

This section establishes the Requirements class for representing geocode data in RDF based on a Literal type that can encompass different Geocodes. It defines one RDFS Datatype: [Geocode Literal](#) and one property, [as GeoCode](#).

5.8.6.1. RDFS Datatype: geo:geocodeLiteral

The datatype `geo:geocodeLiteral` is used to contain a geocode serialization of a Geometry.

```
geo:geocodeLiteral
  a rdfs:Datatype ;
  rdfs:isDefinedBy geo: ;
  skos:prefLabel "Geocode Literal"@en ;
  skos:definition "A Geocode serialization of a Geometry object."@en ;
.
```

The following *ABNF* [\[IETF5234\]](#) syntax specification formally defines this literal:

```
geocodeLiteral ::= opt-iri-and-whitespace geocode-data

opt-iri-and-space = "<" IRI ">" LWSP / ""
```

The token `opt-iri-and-whitespace` may be either an IRI and whitespace (spaces, tabs, newlines) or nothing (`""`), the token `IRI` (Internationalized Resource Identifier) is essentially a web address and is defined in [\[IETF3987\]](#) and the token `LWSP`, is one or more white space characters, as defined in [\[IETF5234\]](#). `geocode-data` is the Geocode representation of the Geometry, in the way it is defined in the definition of the respective Geocoding standard, as identified by its IRI.

Example 38. Geocode Literal Type

RDFS Literals of type `geo:geocodeLiteral` contain a Geocoding system definition in the form of an IRI.

The example [Geocode Literal](#) below encodes a point Geometry (point 27 19' 32S, 153 4' 59"E) in the GeoHash encoding:

```
"<https://w3id.org/geohashes/geohash>  
r7hue3x9jepv"^^<http://www.opengis.net/ont/geosparql#geocodeLiteral>
```

Example 39. Empty Geocode Literal

An empty RDFS Literal of type `geo:geocodeLiteral` shall be interpreted as an empty Geometry .

5.8.6.2. Property: `geo:asGeocode`

The property `geo:asGeocode` is defined to link a Geometry with a Geocode serialization.

Example 40. `asGeocode` Property

Implementations shall allow the RDF property `geo:asGeocode` to be used in SPARQL graph patterns.

The property [as Geocode](#) is used to link a geometric element with a Geocode serialization.

```
geo:asGeocode  
  a rdf:Property, owl:DatatypeProperty;  
  rdfs:subPropertyOf geo:hasSerialization ;  
  rdfs:isDefinedBy geo: ;  
  skos:prefLabel "as Geocode"@en ;  
  skos:definition "The Geocode serialization of a Geometry."@en ;  
  rdfs:domain geo:Geometry ;  
  rdfs:range geo:geocodeLiteral ;  
  .
```

5.8.6.3. Function: `geof:asGeocode`

```
geof:asGeocode (geom: ogc:geomLiteral, geocode: rdfs:Resource): geo:geocodeLiteral
```

The function `geof:asGeocode` converts `geom` to an equivalent Geocode representation in the Geocoding system as defined by the `geocode` parameter.

Implementations shall support `geof:asGeocode` as a SPARQL extension function.

5.8.7. Discrete Global Grid System

requirement

/req/geometry-extension-dggs/query-functions

requirement

/req/geometry-extension-dggs/query-functions-non-sf

requirement

/req/geometry-extension-dggs/srid-function

requirement

/req/geometry-extension-dggs/sa-functions

requirement

/req/geometry-extension-dggs/dggs-literal

requirement

/req/geometry-extension-dggs/dggs-literal-empty

requirement

/req/geometry-extension-dggs/geometry-as-dggs-literal

requirement

/req/geometry-extension-dggs/asDGGS-function

This section establishes the Requirements class for representing Discrete Global Grid System (DGGS) Geometry data as RDF literals. The form of geometry data representation is specific to individual DGGS implementations: known DGGSes are not compatible or even very similar.

The Requirements class defines one RDFS Datatype `http://www.opengis.net/ont/geosparql#dggsLiteral` and one property, `http://www.opengis.net/ont/geosparql#asDGGS`.

5.8.7.1. RDFS Datatype: `geo:dggsLiteral`

The datatype `geo:dggsLiteral` is used to contain the Discrete Global Grid System (DGGS) serialization of a Geometry.

```
geo:dggsLiteral
  a rdfs:Datatype ;
```

```
rdfs:isDefinedBy geo: ;
skos:prefLabel "DGGS Literal"@en ;
skos:definition "A textual serialization of a Discrete Global Grid System (DGGS)
Geometry object."@en
.
```

Valid **DGGS Literal** instances are formed by encoding Geometry information according to a specific DGGS implementation. The specific implementation should be indicated by use of a subclass of the **geo:dggsLiteral** datatype.

Example 43. DGGS Literal

All RDFS Literals of type **geo:dggsLiteral** shall consist of an IRI identifying the specific DGGS and a representation of the DGGS geometry data. The IRI shall be enclosed in angled brackets (< & >) followed by whitespace as a separator, and then the DGGS geometry data, formulated according to the identified DGGS.

The following *ABNF* [IETF5234] syntax specification formally defines this literal:

```
dggsLiteral ::= iri-and-whitespace dggs-geomety-data
iri-and-whitespace = "<" IRI ">" LWSP
```

The token **iri-and-whitespace** is an IRI and whitespace. The token **IRI** (Internationalized Resource Identifier) is essentially a web address and is defined in [IETF3987]. The token **LWSP** is one or more whitespace characters, as defined in [IETF5234]. **dggs-geomety-data** is geometry data formulated according to the DGGS identified by **IRI**.

An example of a DGGS literal for the AusPIX DGGS could be:

```
"<https://w3id.org/dggs/auspix> CELL (R3234)"^^geo:dggsLiteral
```

Where AusPIX is identified with the IRI **https://w3id.org/dggs/auspix** and **CELL (R3234)** is the representation of a geometry according to [AUSPIX].

NOTE	What R3234 means, or the meaning of any other element within a DGGS' geometry data is not handled by GeoSPARQL, just as GeoPSARQL does not delve into the internals of other Geometry formats such as WKT or GeoJSON.
-------------	--

Example 44. Empty DGGS Literal

An empty RDFS Literal of type **geo:dggsLiteral**, shall be interpreted as an empty **geo:Geometry**.

The following *ABNF* [IETF5234] syntax specification formally defines this literal:

```
dggsLiteral ::= iri-and-space dggs-geometry-data  
iri-and-whitespace = "<" IRI ">" LWSP / ""
```

The tokens used above are as per the DGGs *ABNF* above.

5.8.7.2. Property: `geo:asDGGs`

The property `geo:asDGGs` is defined to link a Geometry with its DGGs serialization.

Example 45. asDGGs Property

Implementations shall allow the RDF property `geo:asDGGs` to be used in SPARQL graph patterns.

```
geo:asDGGs  
  a rdf:Property, owl:DatatypeProperty ;  
  rdfs:subPropertyOf geo:hasSerialization ;  
  rdfs:isDefinedBy geo: ;  
  skos:prefLabel "as DGGs"@en ;  
  skos:definition "A DGGs serialization of a Geometry."@en ;  
  rdfs:domain geo:Geometry ;  
  rdfs:range geo:dggsLiteral ;  
  .
```

5.8.7.3. Function: `geof:asDGGs`

```
geof:asDGGs (geom: ogc:geomLiteral, specificDggsDatatype: xsd:anyURI): geo:DggsLiteral
```

```
geof:asDGGsResolution (geom: ogc:geomLiteral, specificDggsDatatype: xsd:anyURI,  
resolution: xsd:string): geo:dggsLiteral
```

The function `geof:asDGGs` converts `geom` to an equivalent DGGs representation, formulated according to the specific DGGs literal indicated by the IRI required to be present in the DGGs literal. This function returns an error if the DGGs needs additional parameters such as a resolution to return a result.

The function `geof:asDGGsResolution` converts `geom` to an equivalent DGGs representation, formulated according to the specific DGGs literal indicated by the IRI required to be present in the DGGs literal. The parameter `resolution` describes the resolution in the target DGGs. If the DGGs type does not require a resolution, the parameter `resolution` will be ignored.

Example 46. asDGGs Function

Implementations shall support `geof:asDGGs` and `geof:asDGGsResolution` as a SPARQL extension

function.

5.9. Non-topological Query Functions

This Requirements class defines SPARQL functions for performing non-topological spatial operations.

Example 47. Non-topological Query Functions (Simple Features)

Implementations shall support the functions `geof:boundary`, `geof:boundingCircle`, `geof:metricBuffer`, `geof:buffer`, `geof:metricWithinDistance`, `geof:withinDistance`, `geof:centroid`, `geof:convexHull`, `geof:concaveHull`, `geof:coordinateDimension`, `geof:difference`, `geof:dimension`, `geof:metricDistance`, `geof:distance`, `geof:envelope`, `geof:geometryType`, `geof:intersection`, `geof:isCollection`, `geof:is3D`, `geof:isEmpty`, `geof:isClosed`, `geof:isMeasured`, `geof:isRing`, `geof:isSimple`, `geof:spatialDimension`, `geof:symDifference`, `geof:transform`, `geof:transformCRS84` and `geof:union` as SPARQL extension functions, consistent with definitions of these functions in Simple Features [\[OGCSFACA\]](#) [\[ISO19125-1\]](#), for non-DGGS geometry literals.

Example 48. Non-topological Query Functions (Non Simple Features)

Implementations shall support the functions `geof:metricLength`, `geof:length`, `geof:metricPerimeter`, `geof:perimeter`, `geof:metricArea`, `geof:area`, `geof:endPoint`, `geof:geometryN`, `geof:easting`, `geof:northing`, `geof:maxX`, `geof:maxY`, `geof:maxZ`, `geof:minX`, `geof:minY`, `geof:minZ`, `geof:numGeometries`, `geof:numInteriorRing`, `geof:numPatches`, `geof:numPoints`, `geof:startPoint`, `geof:simplify`, `geof:X`, `geof:Y`, `geof:Z`, `geof:M` as SPARQL extension functions which are defined in this standard, for non-DGGS geometry literals.

Functions from this Requirements class are listed below, alphabetically.

5.9.1. Function notes

These notes apply to all of the following functions in this section.

An invocation of any of the following functions with invalid arguments produces an error. An invalid argument includes any of the following:

- An argument of an unexpected type
- An invalid geometry literal value
- A non-fitting geometry type for the given function
- A geometry literal from a spatial reference system that is incompatible with the spatial reference system used for calculations
- An invalid unit IRI

A more detailed description of expected inputs and expected outputs of the given functions is

shown in Annex B.

Unless otherwise stated in the function definition, the following behaviors should be followed by all SPARQL extension functions defined in the GeoSPARQL standard:

- Functions returning a new geometry literal should follow the literal format of the first geometry literal input parameter. If no geometry literal input parameter is present, a WKT literal shall be returned.
- Functions returning a new geometry literal should follow the SRS defined in the literal format of the first geometry literal input parameter. If no geometry literal input parameter is present, a geometry result should be returned in the CRS84 SRS.

For further discussion of the effects of errors during FILTER evaluation, consult Section 17^[8] of the SPARQL specification [Section 1.1.4](#).

Note that returning values instead of raising an error serves as an extension mechanism of SPARQL.

From Section 17.3.1^[9] of the SPARQL specification [Section 1.1.4](#):

SPARQL language extensions may provide additional associations between operators and operator functions; ... No additional operator may yield a result that replaces any result other The consequence of this rule is that SPARQL **FILTER** s will produce at least the same intermediate bindings after applying a **FILTER** as an unextended implementation.

This extension mechanism enables GeoSPARQL implementations to simultaneously support multiple geometry serializations. For example, a system that supports [WKT Literal](#) serializations may also support [GML Literal](#) serializations and consequently would not raise an error if it encounters multiple geometry datatypes while processing a given query.

NOTE

Several non-topological query functions use a unit of measure IRI. See the [Recommendation for units of measure](#). Also, the OGC has recommended units of measure vocabularies for use, see the OGC Definitions Server^[10].

5.9.2. Function: **geof:metricArea**

```
geof:metricArea (geom: ogc:geomLiteral): xsd:double
```

The function **geof:metricArea** returns the area of **geom** in square meters. Must return zero for all geometry types other than Polygon. This function is similar to **geof:area** but does not need a specification of measurement unit.

5.9.3. Function: **geof:area**

```
geof:area (geom: ogc:geomLiteral, units: xsd:anyURI): rdf:Resource
```

The function `geof:area` returns the area of `geom`. Must return zero for all geometry types other than Polygon. This function is similar to `geof:metricArea`, which does not need a specification of measurement unit.

NOTE | See the [Recommendation for units of measure](#).

5.9.4. Function: `geof:boundary`

```
geof:boundary (geom: ogc:geomLiteral): ogc:geomLiteral
```

The function `geof:boundary` returns the closure of the boundary of `geom`. Calculations are in the spatial reference system of `geom`.

5.9.5. Function: `geof:boundingCircle`

```
geof:boundingCircle (geom: ogc:geomLiteral): ogc:geomLiteral
```

The function `geof:boundingCircle` returns the minimum bounding circle around `geom`. Calculations are in the spatial reference system of `geom`.

5.9.6. Function: `geof:boundingPolygons`

```
geof:boundingPolygons (geom: ogc:geomLiteral, n: xsd:integer): ogc:geomLiteral
```

The function `geof:boundingPolygons` returns a collection of polygons which bound the n^{th} polygon of a given PolyhedralSurface `geom`.

5.9.7. Function: `geof:metricBuffer`

```
geof:metricBuffer (geom: ogc:geomLiteral,  
                  radius: xsd:double): ogc:geomLiteral
```

The function `geof:metricBuffer` returns a geometric object that represents all Points whose distance from `geom` is less than or equal to the `radius` measured in meters. Calculations are in the coordinate reference system of `geom`. This function is similar to `geof:buffer`, but does not need a specification of measurement unit.

5.9.8. Function: `geof:buffer`

```
geof:buffer (geom: ogc:geomLiteral,
```



```
radius: xsd:double,  
units: xsd:anyURI): ogc:geomLiteral
```

The function **geof:buffer** returns a geometric object that represents all Points whose distance from **geom** is less than or equal to the **radius** measured in **units**. Calculations are in the spatial reference system of **geom**. This function is similar to **geof:metricBuffer**, which does not need a specification of measurement unit.

NOTE See the [Recommendation for units of measure](#).

5.9.9. Function: geof:centroid

```
geof:centroid (geom: ogc:geomLiteral): ogc:geomLiteral
```

The function **geof:centroid** returns the mathematical centroid of **geom**. The centroid point does not have to be part of the surface it is derived from.

5.9.10. Function: geof:convexHull

```
geof:convexHull (geom: ogc:geomLiteral): ogc:geomLiteral
```

The function **geof:convexHull** returns a geometric object that represents all Points in the convex hull of **geom**. Calculations are in the spatial reference system of **geom**.

5.9.11. Function: geof:concaveHull

```
geof:concaveHull (geom: ogc:geomLiteral): ogc:geomLiteral
```

The function **geof:concaveHull** returns a geometric object that represents all Points in the concave hull of **geom**. Calculations are in the spatial reference system of **geom**. Various implementers use parameters to calculate a concave hull. As such, two implementations may return different results from their concave hull functions for the same geometry. Implementers should make clear any default values used to calculate a concave hull in their documentation.

5.9.12. Function: geof:coordinateDimension

```
geof:coordinateDimension (geom: ogc:geomLiteral): xsd:integer
```

The function **geof:coordinateDimension** returns the coordinate dimension of **geom**.

5.9.13. Function: geof:metricWithinDistance

```
geof:metricWithinDistance (geom1: ogc:geomLiteral, geom2: ogc:geomLiteral,
```

```
distance: xsd:double): xsd:boolean
```

Returns true if **geom2** is within a given distance of **geom1** in meters. Calculations are according to the units in the spatial reference system of **geom1**. The triple store implementation needs to take care of a conversion between meter and the unit of the spatial reference system.

5.9.14. Function: **geof:withinDistance**

```
geof:withinDistance (geom: ogc:geomLiteral, geom2: ogc:geomLiteral,  
                    distance: xsd:double, units: xsd:anyURI): xsd:boolean
```

Returns true if **geom2** is within a given distance of **geom1**. Calculations are in the spatial reference system of **geom1**. The triple store implementation needs to take care of a conversion between the unit given as a parameter and the unit of the spatial reference system.

5.9.15. Function: **geof:difference**

```
geof:difference (geom1: ogc:geomLiteral,  
               geom2: ogc:geomLiteral): ogc:geomLiteral
```

The function **geof:difference** returns a geometric object that represents all Points in the set difference of **geom1** with **geom2**. Calculations are in the spatial reference system of **geom1**.

5.9.16. Function: **geof:dimension**

```
geof:dimension (geom: ogc:geomLiteral): xsd:integer
```

The function **geof:dimensions** returns the dimension of **geom**. In non-homogeneous geometry collections, this will return the largest topological dimension of the contained objects.

5.9.17. Function: **geof:metricDistance**

```
geof:metricDistance (geom1: ogc:geomLiteral,  
                   geom2: ogc:geomLiteral): xsd:double
```

The function **geof:metricDistance** returns the shortest distance in meters between any two Points in the two geometric objects. Calculations are in the coordinate reference system of **geom1**. This function is similar to **geof:distance**, but does not need a specification of measurement unit.

5.9.18. Function: **geof:distance**

```
geof:distance (geom1: ogc:geomLiteral,  
             geom2: ogc:geomLiteral,
```

```
units: xsd:anyURI): xsd:double
```

The function **geof:distance** returns the shortest distance in **units** between any two Points in the two geometric objects. Calculations are in the spatial reference system of **geom1**. This function is similar to **geof:metricDistance**, which does not need a specification of measurement unit.

NOTE | See the [Recommendation for units of measure](#).

5.9.19. Function: geof:endPoint

```
geof:endPoint (geom: ogc:geomLiteral): ogc:geomLiteral
```

The function **geof:endPoint** returns the last point in the given **geom** in the literal format of **geom**.

5.9.20. Function: geof:envelope

```
geof:envelope (geom: ogc:geomLiteral): ogc:geomLiteral
```

The function **geof:envelope** returns the minimum bounding box - a rectangle - of **geom**. Calculations are in the spatial reference system of **geom**.

5.9.21. Function: geof:exteriorRing

```
geof:exteriorRing (geom: ogc:geomLiteral): ogc:geomLiteral
```

The function **geof:exteriorRing** returns the exterior interior ring of **geom** if the geometry represents a Polygon.

5.9.22. Function: geof:geometryN

```
geof:geometryN (geom: ogc:geomLiteral, geomindex: xsd:integer): ogc:geomLiteral
```

The function **geof:geometryN** returns the n^{th} geometry of **geom** if it is a GeometryCollection that is defined in a literal type (such as in the case of a **sf:GeometryCollection**) or **geom** if it is a Geometry. This function is not applicable to the type **geo:GeometryCollection**, as elements in **geo:GeometryCollection** are not guaranteed to be ordered.

5.9.23. Function: geof:geometryType

```
geof:geometryType (geom: ogc:geomLiteral): xsd:anyURI
```

The function **geof:geometryType** returns the URI of the subtype of Geometry of which this geometric

object is an member. No attempt to reconcile different geometry subtypes across all support literals need be made.

5.9.24. Function: **geof:getSRID**

```
geof:getSRID (geom: ogc:geomLiteral): xsd:anyURI
```

The function **geof:getSRID** returns the spatial reference system IRI for **geom**.

5.9.25. Function: **geof:interiorRingN**

```
geof:interiorRingN (geom: ogc:geomLiteral, ringindex: xsd:integer): ogc:geomLiteral
```

The function **geof:interiorRingN** returns the n^{th} interior ring of **geom** if the geometry represents a Polygon.

5.9.26. Function: **geof:intersection**

```
geof:intersection (geom1: ogc:geomLiteral,  
                  geom2: ogc:geomLiteral): ogc:geomLiteral
```

The function **geof:intersection** returns a geometric object that represents all Points in the intersection of **geom1** with **geom2**. Calculations are in the spatial reference system of **geom1**.

5.9.27. Function: **geof:is3D**

```
geof:is3D (geom: ogc:geomLiteral): xsd:boolean
```

The function **geof:is3D** Returns true if **geom** has z coordinate values.

5.9.28. Function: **geof:isCollection**

```
geof:isCollection (geom: ogc:geomLiteral): xsd:boolean
```

Returns true if the literal describing **geom** represents a GeometryCollection.

5.9.29. Function: **geof:isClosed**

```
geof:isClosed (geom: ogc:geomLiteral): xsd:boolean
```

The function **geof:isClosed** returns true if **geom** is a closed geometry, i.e. its start point equals its end point.

5.9.30. Function: `geof:isEmpty`

```
geof:isEmpty (geom: ogc:geomLiteral): xsd:boolean
```

The function `geof:isEmpty` returns true if `geom` is an empty geometry, i.e. contains no coordinates.

5.9.31. Function: `geof:isMeasured`

```
geof:isMeasured (geom: ogc:geomLiteral): xsd:boolean
```

The function `geof:isMeasured` returns true if `geom` has `m` coordinate values.

5.9.32. Function: `geof:isRing`

```
geof:isRing (geom: ogc:geomLiteral): xsd:boolean
```

The function `geof:isRing` Returns true if `geom` is a closed geometry (see `geof:isClosed`), and simple (see `geof:isSimple`).

5.9.33. Function: `geof:isSimple`

```
geof:isSimple (geom: ogc:geomLiteral): xsd:boolean
```

The function `geof:isSimple` returns true if `geom` is a simple geometry, i.e. has no anomalous geometric points, such as self intersection or self tangency.

5.9.34. Function: `geof:metricLength`

```
geof:metricLength (geom: ogc:geomLiteral): xsd:double
```

The function `geof:metricLength` returns the length of `geom` in meters. The longest length from any one dimension is returned. This is for example the length of a line from its beginning point to its endpoint or the length of the boundary of a polygon. This function is similar to `geof:length` but does not need a specification of measurement unit.

5.9.35. Function: `geof:length`

```
geof:length (geom: ogc:geomLiteral, units: xsd:anyURI): xsd:double
```

The function `geof:length` returns the length of `geom`. The longest length from any one dimension is returned. This function is similar to `geof:metricLength`, which does not need a specification of measurement unit.

NOTE See the [Recommendation for units of measure](#).

```
geof:easting (geom: ogc:geomLiteral): xsd:double
```

The function **geof:easting** returns the easting, e.g., the latitude of **geom** in the unit that is proposed by the coordinate reference system associated with the geometry. This function is defined only for such **geom** representing Points.

```
geof:northing (geom: ogc:geomLiteral): xsd:double
```

The function **geof:northing** returns the northing, e.g., the longitude of **geom** in the unit that is proposed by the coordinate reference system associated with the geometry. This function is defined only for such **geom** representing Points.

5.9.36. Function: **geof:maxX**

```
geof:maxX (geom: ogc:geomLiteral): xsd:double
```

The function **geof:maxX** returns the maximum X coordinate for **geom** using the SRS of **geom**.

5.9.37. Function: **geof:maxY**

```
geof:maxY (geom: ogc:geomLiteral): xsd:double
```

The function **geof:maxY** returns the maximum Y coordinate for **geom** using the SRS of **geom**.

5.9.38. Function: **geof:maxZ**

```
geof:maxZ (geom: ogc:geomLiteral): xsd:double
```

The function **geof:maxZ** returns the maximum Z coordinate for **geom** using the SRS of **geom**.

5.9.39. Function: **geof:minX**

```
geof:minX (geom: ogc:geomLiteral): xsd:double
```

The function **geof:minX** returns the minimum X coordinate for **geom** using the SRS of **geom**.

5.9.40. Function: **geof:minY**

```
geof:minY (geom: ogc:geomLiteral): xsd:double
```

The function **geof:minY** returns the minimum Y coordinate for **geom** using the SRS of **geom**.

5.9.41. Function: geof:minZ

```
geof:minZ (geom: ogc:geomLiteral): xsd:double
```

The function **geof:minZ** returns the minimum Z coordinate for **geom** using the SRS of **geom**.

5.9.42. Function: geof:numGeometries

```
geof:numGeometries (geom: ogc:geomLiteral): xsd:integer
```

The function **geof:numGeometries** returns the number of geometries of **geom**.

NOTE

This function returns 1 except in cases when it receives a collection type, such as an **sf:MultiPoint**, a **sf:MultiLineString**, a **sf:MultiPolygon**, or a **sf:GeometryCollection**. The function does not apply to **geo:GeometryCollection** instances, as members of **geo:GeometryCollection** instances are not ordered.

5.9.43. Function: geof:numPatches

```
geof:numPatches (geom: ogc:geomLiteral): xsd:integer
```

The function **geof:numPatches** returns the number of including polygons of **geom**.

5.9.44. Function: geof:numInteriorRing

```
geof:numInteriorRing (geom: ogc:geomLiteral): xsd:integer
```

The function **geof:numInteriorRing** Returns the number of interior rings of **geom** if the geometry represents a Polygon.

5.9.45. Function: geof:numPoints

```
geof:numPoints (geom: ogc:geomLiteral): xsd:integer
```

The function **geof:numPoints** returns the number of points of **geom**.

5.9.46. Function: **geof:patchN**

```
geof:patchN (geom: ogc:geomLiteral, index: xsd:integer): ogc:geomLiteral
```

The function **geof:patchN** returns the n^{th} polygon of **geom**, in the order of definition.

5.9.47. Function: **geof:perimeter**

```
geof:perimeter (geom: ogc:geomLiteral, unit: xsd:anyURI): xsd:double
```

The function **geof:perimeter** returns the perimeter of **geom** in the unit specified by the unit parameter for areal geometries. For non-area geometries the result is equivalent to **geof:hasLength**.

5.9.48. Function: **geof:metricPerimeter**

```
geof:metricPerimeter (geom: ogc:geomLiteral): xsd:double
```

The function **geof:metricPerimeter** returns the perimeter is similar to the function **geof:perimeter**, but always returns the result in meter.

5.9.49. Function: **geof:pointN**

```
geof:pointN (geom:ogc:geomLiteral, geomindex: xsd:integer): ogc:geomLiteral
```

The function **geof:pointN** Returns the n^{th} point in **geom**.

5.9.50. Function: **geof:pointOnSurface**

```
geof:pointOnSurface (geom: ogc:geomLiteral, n: xsd:integer): ogc:geomLiteral
```

The function **geof:pointOnSurface** returns a point which is guaranteed to be on the n^{th} surface of **geom**.

5.9.51. Function: **geof:spatialDimension**

```
geof:spatialDimension (geom: ogc:geomLiteral): xsd:integer
```

The function **geof:spatialDimension** returns the spatial dimension of **geom**.

5.9.52. Function: `geof:startPoint`

```
geof:startPoint (geom: ogc:geomLiteral): ogc:geomLiteral
```

The function `geof:startPoint` Returns the first point in the given `geom` in the literal format of `geom`.

5.9.53. Function: `geof:symDifference`

```
geof:symDifference (geom1: ogc:geomLiteral,  
                  geom2: ogc:geomLiteral): ogc:geomLiteral
```

The function `geof:symDifference` returns a geometric object that represents all Points in the set symmetric difference of `geom1` with `geom2`. Calculations are in the spatial reference system of `geom1`.

5.9.54. Function: `geof:simplify`

```
geof:simplify (geom: ogc:geomLiteral, tolerance: xsd:double): ogc:geomLiteral
```

The function `geof:simplify` simplifies `geom` by applying the Douglas-Peucker algorithm [DOUGLASPEUCKER].

5.9.55. Function: `geof:transform`

```
geof:transform (geom: ogc:geomLiteral, srsIRI: xsd:anyURI): ogc:geomLiteral
```

The function `geof:transform` converts `geom` to a spatial reference system defined by `srsIRI`. The function raises an error if a transformation is not mathematically possible.

NOTE

We recommend that implementers use the same literal type as a result of this function as the type of the input literal.

5.9.56. Function: `geof:transformCRS84`

```
geof:transformCRS84 (geom: ogc:geomLiteral): ogc:geomLiteral
```

The function `geof:transformCRS84` converts `geom` to the CRS84 coordinate system. The function raises an error if a transformation is not mathematically possible.

NOTE

We recommend that implementers use the same literal type as a result of this function as the type of the input literal.

5.9.57. Function: **geof:union**

```
geof:union (geom1: ogc:geomLiteral,  
            geom2: ogc:geomLiteral): ogc:geomLiteral
```

The function **geof:union** returns a geometric object that represents all Points in the union of **geom1** with **geom2**. Calculations are in the spatial reference system of **geom1**.

5.9.58. Function: **geof:X**

```
geof:X (geom: ogc:geomLiteral): xsd:double
```

The function **geof:X** returns the X coordinate if **geom** is a point geometry. The X coordinate is determined by the X axis definition of its SRS.

5.9.59. Function: **geof:Y**

```
geof:Y (geom: ogc:geomLiteral): xsd:double
```

The function **geof:Y** returns the Y coordinate if **geom** is a point geometry. The Y coordinate is determined by the Y axis definition of its SRS.

5.9.60. Function: **geof:Z**

```
geof:Z (geom: ogc:geomLiteral): xsd:double
```

The function **geof:Z** returns the Z coordinate if **geom** is a point geometry. The Z coordinate is determined by the Z axis definition of its SRS.

5.9.61. Function: **geof:M**

```
geof:M (geom: ogc:geomLiteral): xsd:double
```

The function **geof:M** returns the M coordinate if **geom** is a point geometry. The M coordinate is determined by the M coordinate definition of its SRS.

Example 49. SRID Function

Implementations shall support **geof:getSRID** as a SPARQL extension function.

5.9.62. Function: geof:getSRID

```
geof:getSRID (geom: ogc:geomLiteral): xsd:anyURI
```

The function `geof:getSRID` returns the spatial reference system IRI for `geom`.

5.10. Non-topological Query Functions - 3D Extension

This Requirements class defines SPARQL functions for performing non-topological spatial operations in 3D.

Example 50. Non-topological Query Functions - 3D

Implementations shall support the functions `geof:extrude` `geof:extrudeByLine` `geof:extrudeInTime` as SPARQL extension functions which are defined in this standard.

5.10.1. Function: geof:extrude

```
geof:extrude (geometry: ogc:geomLiteral,  
             height: xsd:double): geo:Feature
```

The function `geof:extrude` returns a spatial Feature with a 3D geometric object that represents all Points in the extruded form of 2D `geometry` where the `height` indicates the number added to Z coordinates of base `geometry` to arrive at Z coordinates of the top most `geometry` in the extruded form. In case of no Z coordinates being present in the base `geometry`, the `height` is added to 0 and is equal to Z coordinates of the top most `geometry` in the extruded form. Calculations are in the spatial reference system of `geometry`.

5.10.2. Function: geof:extrudeByLine

```
geof:extrudeByLine (geometry: ogc:geomLiteral,  
                   line-segment: ogc:geomLiteral): geo:Feature
```

The function `geof:extrudeByLine` returns a spatial Feature with a 3D geometric object that represents all Points in the extruded form of 2D `geometry` where the `line-segment` marks the starting and ending points of the extruded form in space. Calculations are in the spatial reference system of `geometry`.

5.10.3. Function: geof:extrudeInTime

```
geof:extrudeInTime (geometry: ogc:geomLiteral,  
                   line-segment: ogc:geomLiteral,  
                   start: xsd:dateTime,
```

```
end: xsd:dateTime): geo:Feature
```

The function `geof:extrudeInTime` returns a spatio-temporal Feature with a 3D geometric object that represents all Points in the extruded form of 2D `geometry` where the `line-segment` marks the starting and ending points of the extruded form in space and ProperInterval with `start` and `end` in time. Calculations are in the spatial reference system of `geometry`.

5.11. Spatial Aggregate Functions

This clause defines SPARQL functions for performing spatial aggregations of data.

Example 51. Spatial Aggregate Functions

Implementations shall support `geof:aggBoundingBox`, `geof:aggBoundingCircle`, `geof:aggCentroid`, `geof:aggCollect`, `geof:aggConcaveHull`, `geof:aggConvexHull` and `geof:aggUnion` as a SPARQL extension functions.

NOTE

This spec does not define a deterministic order on `geomLiterals` which act as inputs of Spatial Aggregate functions, as opposed to functions requiring a single parameter. This means in particular, that the coordinate reference system and the literal type which is to be used in the output of spatial aggregate functions can be derived from the first item in the array of `geomLiteral`, but GeoSPARQL does not mandate it. Users of GeoSPARQL may use the `geof:transform` function to convert results of spatial aggregate functions to their intended coordinate reference system.

5.11.1. Function: `geof:aggBoundingBox`

```
geof:aggBoundingBox (geom: ogc:geomLiteral[]): ogc:geomLiteral
```

The function `geof:aggBoundingBox` calculates a minimum bounding box - rectangle - of the set of given geometries.

5.11.2. Function: `geof:aggBoundingCircle`

```
geof:aggBoundingCircle (geom: ogc:geomLiteral[]): ogc:geomLiteral
```

The function `geof:aggBoundingCircle` calculates a minimum bounding circle of the set of given geometries.

5.11.3. Function: `geof:aggCentroid`

```
geof:aggCentroid (geom: ogc:geomLiteral[]): ogc:geomLiteral
```

The function **geof:aggCentroid** calculates the centroid of the set of given geometries.

5.11.4. Function: geof:aggCollect

```
geof:aggCollect (geom: ogc:geomLiteral[]): ogc:geomLiteral
```

The function **geof:aggCollect** creates a GeometryCollection literal out of a set of geometries.

5.11.5. Function: geof:aggConcaveHull

```
geof:aggConcaveHull (geom: ogc:geomLiteral[], targetPercent: xsd:double):  
ogc:geomLiteral
```

The function **geof:aggConcaveHull** calculates the concave hull of the set of given geometries.

5.11.6. Function: geof:aggConvexHull

```
geof:aggConvexHull (geom: ogc:geomLiteral[]): ogc:geomLiteral
```

The function **geof:aggConvexHull** calculates the convex hull of the set of given geometries.

NOTE This function is similar in name to **geof:convexHull** used to calculate the convex hull of just one geometry.

5.11.7. Function: geof:aggUnion

```
geof:aggUnion (geom: ogc:geomLiteral[]): ogc:geomLiteral
```

The function **geof:aggUnion** calculates the union of the set of given geometries.

NOTE This function is similar in name to **geof:union** used to calculate the union of just two geometries.

[1] <http://www.w3.org/2003/01/geo/>

[2] <http://purl.org/dc/terms/Location>

[3] <https://schema.org/GeoCoordinates>

[4] <https://schema.org/GeoShape>

[5] <https://www.w3.org/ns/locn>

[6] <https://www.w3.org/TR/vocab-dcat/#spatial-properties>

[7] <http://www.qudt.org>

[8] <https://www.w3.org/TR/sparql11-query/#expressions>

[9] <https://www.w3.org/TR/sparql11-query/#operatorExtensibility>

[10] <https://www.ogc.org/def-server>

Chapter 6. Geometry Topology Extension

This clause establishes the *Geometry Topology Extension* parameterized Requirements class with base IRI [/req/geometry-topology-extension](#), which defines a collection of topological query functions that operate on geometry literals. These Requirements are parameterized to give implementations flexibility in the topological relation families and geometry serializations that they choose to support. These Requirements have a single corresponding conformance class *Geometry Topology Extension*, with IRI [/conf/geometry-topology-extension](#).

The Dimensionally Extended Nine Intersection Model (DE-9IM) [\[DE-9IM\]](#) has been used to define the relation tested by the query functions introduced in this section. Each query function is associated with a defining DE-9IM intersection pattern. Possible pattern values are:

- **-1** (empty)
- **0, 1, 2, T** (true) = {0, 1, 2}
- **F** (false) = {-1}
- ***** (don't care) = {-1, 0, 1, 2}

In the following descriptions, the notation **X/Y** is used to denote applying a spatial relation to geometry types **X** and **Y** (i.e., **x relation y** where **x** is of type **X** and **y** is of type **Y**). The symbol **P** is used for 0-dimensional geometries (e.g., points). The symbol **L** is used for 1-dimensional geometries (e.g., lines), and the symbol **A** is used for 2-dimensional geometries (e.g., polygons). Consult the Simple Features specification [\[OGC06-103r4\]](#) [\[ISO19125-1\]](#) for a more detailed description of DE-9IM intersection patterns.

Example 52. Geometry Topology Extension

requirement

[/req/geometry-topology-extension/relate-query-function](#)

requirement

[/req/geometry-topology-extension/sf-query-functions](#)

requirement

[/req/geometry-topology-extension/eh-query-functions](#)

requirement

[/req/geometry-topology-extension/rcc8-query-functions](#)

6.1. Parameters

- **relation_family**: Specifies the set of topological spatial relations to support.
- **serialization**: Specifies the serialization standard to use for geometry literals.
- **version**: Specifies the version of the serialization format used.

6.2. Common Query Functions

Example 53. Relate Query Function

Implementations shall support `geof:relate` as a SPARQL extension function, consistent with the relate operator defined in Simple Features [\[OGC06-103r4\]](#) [\[ISO19125-1\]](#).

```
geof:relate (geom1: ogc:geomLiteral,  
            geom2: ogc:geomLiteral,  
            pattern-matrix: xsd:string): xsd:boolean
```

Returns `true` if the spatial relationship between `geom1` and `geom2` corresponds to one with acceptable values for the specified pattern-matrix. Otherwise, this function returns `false`. `pattern-matrix` represents a DE-9IM intersection pattern consisting of `T` (true) and `F` (false) values. The spatial reference system for `geom1` is used for spatial calculations.

6.3. Simple Features Relation Family

This clause establishes Requirements for the *Simple Features* relation family.

Example 54. Simple Features Query Functions

Implementations shall support `geof:sfEquals`, `geof:sfDisjoint`, `geof:sfIntersects`, `geof:sfTouches`, `geof:sfCrosses`, `geof:sfWithin`, `geof:sfContains` and `geof:sfOverlaps` as SPARQL extension functions, consistent with their corresponding DE-9IM intersection patterns, as defined by Simple Features [\[OGC06-103r4\]](#) [\[ISO19125-1\]](#).

Boolean query functions defined for the Simple Features relation family, along with their associated DE-9IM intersection patterns, are shown in [Simple Features Query Functions](#) below. Multi-row intersection patterns should be interpreted as a logical OR of each row. Each function accepts two arguments (`geom1` and `geom2`) of the geometry literal *serialization* type *specified* by serialization and version. Each function returns an `xsd:boolean` value of `true` if the specified relation exists between `geom1` and `geom2` and returns false otherwise. In each case, the spatial reference system of `geom1` is used for spatial calculations.

Simple Features Query Functions

Query Function	Defining DE-9IM Intersection Pattern
<code>geof:sfEquals(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean</code>	(TFFFTFFFT)
<code>geof:sfDisjoint(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean</code>	(FF*FF****)
<code>geof:sfIntersects(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean</code>	(FT***** F**T***** F***T****)
<code>geof:sfTouches(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean</code>	(FT***** F**T***** F***T****)

Query Function	Defining DE-9IM Intersection Pattern
<code>geof:sfCrosses(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean</code>	(T*T***T**) for P/L, P/A, L/A; (0*T***T**) for L/L
<code>geof:sfWithin(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean</code>	(T*F**F***)
<code>geof:sfContains(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean</code>	(T*****FF*)
<code>geof:sfOverlaps(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean</code>	(T*T***T**) for A/A, P/P; (1*T***T**) for L/L

6.4. Egenhofer Relation Family

This clause establishes Requirements for the *Egenhofer* relation family. Consult references [\[FORMAL\]](#) and [\[CATEG\]](#) for a more detailed discussion of *Egenhofer* relations.

Example 55. Egenhofer Query Functions

Implementations shall support `geof:ehEquals`, `geof:ehDisjoint`, `geof:ehMeet`, `geof:ehOverlap`, `geof:ehCovers`, `geof:ehCoveredBy`, `geof:ehInside` and `geof:ehContains` as SPARQL extension functions, consistent with their corresponding DE-9IM intersection patterns, as defined by Simple Features [\[OGC06-103r4\]](#) [\[ISO19125-1\]](#).

Boolean query functions defined for the *Egenhofer* relation family, along with their associated DE-9IM intersection patterns, are shown in [Egenhofer Query Functions](#) below. Multi-row intersection patterns should be interpreted as a logical OR of each row. Each function accepts two arguments (`geom1` and `geom2`) of the geometry literal serialization type specified by *serialization* and *version*. Each function returns an `xsd:boolean` value of `true` if the specified relation exists between `geom1` and `geom2` and returns `false` otherwise. In each case, the spatial reference system of `geom1` is used for spatial calculations.

Egenhofer Query Functions

Query Function	Defining DE-9IM Intersection Pattern
<code>geof:ehEquals(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean</code>	(TFFFTFFFT)
<code>geof:ehDisjoint(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean</code>	(FF*FF****)
<code>geof:ehMeet(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean</code>	(FT***** F**T***** F***T*****)
<code>geof:ehOverlap(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean</code>	(T*T***T**)
<code>geof:ehCovers(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean</code>	(T*TFT*FF*)
<code>geof:ehCoveredBy(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean</code>	(TFF*TFT**)
<code>geof:ehInside(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean</code>	(TFF*FFT**)

Query Function	Defining DE-9IM Intersection Pattern
<code>geof:ehContains(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean</code>	<code>(T*TFF*FF*)</code>

6.5. RCC8 Relation Family

This clause establishes Requirements for the *RCC8* relation family. Consult references [\[QUAL\]](#) and [\[LOGIC\]](#) for a more detailed discussion of *RCC8* relations.

Example 56. RCC8 Query Functions

Implementations shall support `geof:rcc8eq`, `geof:rcc8dc`, `geof:rcc8ec`, `geof:rcc8po`, `geof:rcc8tppi`, `geof:rcc8tpp`, `geof:rcc8ntpp` and `geof:rcc8ntppi` as SPARQL extension functions, consistent with their corresponding DE-9IM intersection patterns, as defined by Simple Features [\[OGC06-103r4\]](#) [\[ISO19125-1\]](#).

Boolean query functions defined for the *RCC8* relation family, along with their associated DE-9IM intersection patterns, are shown in [RCC8 Query Functions](#) below. Each function accepts two arguments (`geom1` and `geom2`) of the geometry literal serialization type specified by *serialization* and *version*. Each function returns an `xsd:boolean` value of `true` if the specified relation exists between `geom1` and `geom2` and returns `false` otherwise. In each case, the spatial reference system of `geom1` is used for spatial calculations.

RCC8 Query Functions

Query Function	Defining DE-9IM Intersection Pattern
<code>geof:rcc8eq(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean</code>	<code>(TFFFTFFFT)</code>
<code>geof:rcc8dc(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean</code>	<code>(FFTFTTTT)</code>
<code>geof:rcc8ec(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean</code>	<code>(FFTFTTTT)</code>
<code>geof:rcc8po(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean</code>	<code>(TTTTTTTT)</code>
<code>geof:rcc8tppi(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean</code>	<code>(TTTFTTFFT)</code>
<code>geof:rcc8tpp(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean</code>	<code>(TFFTFTTT)</code>
<code>geof:rcc8ntpp(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean</code>	<code>(TFFTFTTT)</code>
<code>geof:rcc8ntppi(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean</code>	<code>(TTTFTTFFT)</code>

6.6. RCC*-9 Relation Family

This clause establishes Requirements for the *RCC*-9* relation family. Consult references [\[RCC9\]](#) for a more detailed discussion of *RCC*-9* relations.

Example 57. RCC*-9 Query Functions

Implementations shall support `geof:rcc9eq`, `geof:rcc9dc`, `geof:rcc9cr`, `geof:rcc9ec`, `geof:rcc9po`, `geof:rcc9tppi`, `geof:rcc9tpp`, `geof:rcc9ntpp` and `geof:rcc9ntppi` as SPARQL extension functions, consistent with their corresponding DE-9IM intersection patterns, as defined by Simple Features [OGC06-103r4] [ISO19125-1].

Boolean query functions defined for the *RCC*-9* relation family, along with their associated DE-9IM intersection patterns, are shown in [RCC*-9 Query Functions](#) below. Each function accepts two arguments (`geom1` and `geom2`) of the geometry literal serialization type specified by *serialization* and *version*. Each function returns an `xsd:boolean` value of `true` if the specified relation exists between `geom1` and `geom2` and returns `false` otherwise. In each case, the spatial reference system of `geom1` is used for spatial calculations.

RCC*-9 Query Functions

Query Function	Defining DE-9IM Intersection Pattern
<code>geof:rcc9eq(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean</code>	<code>(TFFF*FFFT)</code>
<code>geof:rcc9cr(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean</code>	<code>(T*TFFTT**) v (TFT*F*TT*) v (TFTFFFFT) v (TTTTFFTTT) v (TFTTFTTFT)</code>
<code>geof:rcc9dc(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean</code>	<code>(FF*FF****)</code>
<code>geof:rcc9ec(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean</code>	<code>(F*TT**T**) v (FTT***T**) v (F*T*T*T**)</code>
<code>geof:rcc9po(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean</code>	<code>(TTTT**T**) v (T*T*T*T**)</code>
<code>geof:rcc9tppi(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean</code>	<code>(***T**FF*) v (****T*FF*) v ¬(TFFF*FFFT)</code>
<code>geof:rcc9tpp(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean</code>	<code>(*TF**F***) v (**F*TF***) v ¬(TFFF*FFFT)</code>
<code>geof:rcc9ntpp(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean</code>	<code>(*FF*FFT**)</code>
<code>geof:rcc9ntppi(geom1: ogc:geomLiteral, geom2: ogc:geomLiteral): xsd:boolean</code>	<code>(**TFF*FF*)</code>

Chapter 7. RDFS Entailment Extension

This clause establishes the *RDFS Entailment Extension* parameterized Requirements class with base IRI [/req/rdfs-entailment-extension](#), which defines a mechanism for matching implicitly-derived RDF triples in GeoSPARQL queries. This class is parameterized to give implementations flexibility in the topological relation families and geometry types that they choose to support. These Requirements have a single corresponding conformance class *RDFS Entailment Extension*, with IRI [/conf/rdfs-entailment-extension](#).

7.1. Parameters

- **relation_family**: Specifies the set of topological spatial relations to support.
- **serialization**: Specifies the serialization standard to use for geometry literals.
- **version**: Specifies the version of the serialization format used.

7.2. Common Requirements

The basic mechanism for supporting RDFS entailment has been defined by the W3C SPARQL 1.1 RDFS Entailment Regime [\[SPARQLENT\]](#).

Example 58. Basic Graph Pattern

Basic graph pattern matching shall use the semantics defined by the RDFS Entailment Regime [\[SPARQLENT\]](#).

7.3. WKT Serialization

This section establishes the requirements for representing geometry data in RDF based on WKT as defined by Simple Features [\[OGCSFACA\]](#) [\[ISO19125-1\]](#).

7.3.1. Geometry Class Hierarchy

The Simple Features specification presents a geometry class hierarchy. It is straightforward to represent this class hierarchy in RDFS and OWL by constructing IRIs for geometry classes using the following pattern: <http://www.opengis.net/ont/sf#{geometry class}> and by asserting appropriate `rdfs:subClassOf` statements. The *Simple Features Vocabulary* resource within GeoSPARQL 1.1 (sibling resource to this specification) does this. The following list gives the class hierarchy with each indented item being a subclass of the item in the line above. The class hierarchy starts with GeoSPARQL's `geo:Geometry` class of which `sf:Geometry` is a subclass:

```
geo:Geometry
  sf:Geometry
    sf:Curve
      sf:LineString
        sf:Line
```

```

        sf:LinearRing
sf:GeometryCollection
    sf:MultiCurve
        sf:MultiLineString
    sf:MultiPoint
    sf:MultiSurface
        sf:MultiPolygon
sf:Point
sf:Surface
    sf:Polygon
        sf:Envelope
        sf:Triangle
    sf:PolyhedralSurface
        sf:TIN

```

The following example RDF snippet below encodes the *Simple Features vocabulary* Polygon class:

```

sf:Polygon
  a rdfs:Class, owl:Class ;
  rdfs:isDefinedBy <http://www.opengis.net/ont/sf> ;
  skos:prefLabel "Polygon"@en ;
  rdfs:subClassOf sf:Surface ;
  skos:definition "A planar surface defined by 1 exterior boundary and 0 or
                  more interior boundaries"@en ;
.

```

Example 59. WKT Geometry Types

Implementations shall support graph patterns involving terms from an RDFS/OWL class hierarchy of geometry types consistent with the one in the specified version of Simple Features [\[OGCSFACA\] \[ISO19125-1\]](#).

7.4. GML Serialization

This section establishes Requirements for representing geometry data in RDF based on GML as defined by Geography Markup Language Encoding Standard [\[OGC07-036\]](#).

7.4.1. Geometry Class Hierarchy

An RDF/OWL class hierarchy can be generated from the GML schema that implements **GM_Object** by constructing IRIs for geometry classes using the following pattern: <http://www.opengis.net/ont/gml#{GML Element}> and by asserting appropriate `rdfs:subClassOf` statements.

The example RDF snippet below encodes the Polygon class from GML 3.2.

```

gml:Polygon

```

```
a rdfs:Class, owl:Class ;  
skos:prefLabel "Polygon"@en ;  
rdfs:subClassOf gml:SurfacePatch ;  
skos:definition "A planar surface defined by 1 exterior boundary and 0 or  
more interior boundaries."@en ;  
.
```

Example 60. GML Geometry Types

Implementations shall support graph patterns involving terms from an RDFS/OWL class hierarchy of geometry types consistent with the GML schema that implements *GM_Object* using the specified *version* of GML [\[OGC07-036\]](#).

Appendix A: Abstract Test Suite

A.0. Overview

This Annex lists tests for the Conformance Classes defined in the main body sections of this Specification with links to their Requirements and test purpose method and type. Conformance classes may be used to signify the compatibility of a given implementation to parts of the GeoSPARQL standard. They may be stated as part of a SPARQL 1.1 Service Description [\[SPARQLSERVDESC\]](#).

A.1. Conformance Class: Core

Example 61. Core

```
target
  /req/core

abstract-test
  /conf/core/sparql-protocol

abstract-test
  /conf/core/spatial-object-class

abstract-test
  /conf/core/feature-class

abstract-test
  /conf/core/spatial-object-collection-class

abstract-test
  /conf/core/feature-collection-class

abstract-test
  /conf/core/spatial-object-properties
```

A.1.1. SPARQL

```
target
  /req/core/sparql-protocol

test-purpose
  Check conformance with this requirement

test-method
  Verify that the implementation accepts SPARQL queries and returns the correct results in
```

the correct format, according to the SPARQL Query Language for RDF, the SPARQL Protocol for RDF and SPARQL Query Results XML Format W3C specifications.

test-method-type

Capabilities

reference

[\[sparql-protocol\]](#)

A.1.2. RDF Classes & Properties

target

/req/core/spatial-object-class

test-purpose

Check conformance with this requirement

test-method

Verify that queries involving `geo:SpatialObject` return the correct result on a test dataset.

test-method-type

Capabilities

reference

[Section 3.2.1](#)

target

/req/core/feature-class

test-purpose

Check conformance with this requirement

test-method

Verify that queries involving `geo:Feature` return the correct result on a test dataset.

test-method-type

Capabilities

reference

[Section 3.2.2](#)

target

/req/core/spatial-object-collection-class

test-purpose

Check conformance with this requirement

test-method

Verify that queries involving `geo:SpatialObjectCollection` return the correct result on a test dataset.

test-method-type

Capabilities

reference

[Section 3.2.3](#)

target

/req/core/feature-collection-class

test-purpose

Check conformance with this requirement

test-method

Verify that queries involving `geo:FeatureCollection` return the correct result on a test dataset.

test-method-type

Capabilities

reference

[Section 3.2.4](#)

target

/req/core/spatial-object-properties

test-purpose

Check conformance with this requirement

test-method

Verify that queries involving these following properties return the correct result for a test dataset: `geo:hasSize`, `geo:hasMetricSize`, `geo:hasLength`, `geo:hasMetricLength`, `geo:hasPerimeterLength`, `geo:hasMetricPerimeterLength`, `geo:hasArea`, `geo:hasMetricArea`, `geo:hasVolume` and `geo:hasMetricVolume`

test-method-type

Capabilities

A.2. Conformance Class: Topology Vocabulary Extension

Example 62. Topology Vocabulary Extension

target

/req/topology-vocab-extension

abstract-test

/conf/topology-vocab-extension/sf-spatial-relations

abstract-test

/conf/topology-vocab-extension/eh-spatial-relations

abstract-test

/conf/topology-vocab-extension/rcc8-spatial-relations

A.2.1. Simple Features Relation Family

target

/req/topology-vocab-extension/sf-spatial-relations

test-purpose

Check conformance with this requirement

test-method

Verify that queries involving the following properties return the correct result for a test dataset: `geo:sfEquals`, `geo:sfDisjoint`, `geo:sfIntersects`, `geo:sfTouches`, `geo:sfCrosses`, `geo:sfWithin`, `geo:sfContains` and `geo:sfOverlaps`

test-method-type

Capabilities

reference

[Simple Features Topological Relations](#)

A.2.2. Egenhofer Relation Family

target

/req/topology-vocab-extension/eh-spatial-relations

test-purpose

Check conformance with this requirement

test-method

Verify that queries involving the following properties return the correct result for a test dataset: `geo:ehEquals`, `geo:ehDisjoint`, `geo:ehMeet`, `geo:ehOverlap`, `geo:ehCovers`, `geo:ehCoveredBy`, `geo:ehInside` and `geo:ehContains`

test-method-type

Capabilities

reference

[Egenhofer Topological Relations](#)

A.2.3. RCC8 Relation Family

target

/req/topology-vocab-extension/rcc8-spatial-relations

test-purpose

Check conformance with this requirement

test-method

Verify that queries involving the following properties return the correct result for a test dataset: `geo:rcc8eq`, `geo:rcc8dc`, `geo:rcc8ec`, `geo:rcc8po`, `geo:rcc8tppi`, `geo:rcc8tpp`, `geo:rcc8ntpp`, `geo:rcc8ntppi`

test-method-type

Capabilities

reference

[RCC8 Topological Relations](#)

A.3. Conformance Class: Geometry Extension

Example 63. Geometry Extension

target

/req/geometry-extension

abstract-test

/conf/geometry-extension/geometry-class

abstract-test

/conf/geometry-extension/geometry-collection-class

abstract-test

/conf/geometry-extension/feature-properties

abstract-test

/conf/geometry-extension/geometry-properties

abstract-test

/conf/geometry-extension/query-functions

abstract-test

/conf/geometry-extension/srid-function

abstract-test

/conf/geometry-extension/sa-functions

abstract-test

/conf/geometry-extension/wkt-literal

abstract-test

/conf/geometry-extension/wkt-literal-default-srs

abstract-test

/conf/geometry-extension/wkt-axis-order

abstract-test

/conf/geometry-extension/wkt-literal-empty

abstract-test

/conf/geometry-extension/geometry-as-wkt-literal

abstract-test

/conf/geometry-extension/asWKT-function

abstract-test

/conf/geometry-extension/gml-literal

abstract-test

/conf/geometry-extension/gml-literal-empty

abstract-test

/conf/geometry-extension/gml-profile

abstract-test

/conf/geometry-extension/geometry-as-gml-literal

abstract-test

/conf/geometry-extension/asGML-function

abstract-test

/conf/geometry-extension/geojson-literal

abstract-test

/conf/geometry-extension/geojson-literal-srs

abstract-test

/conf/geometry-extension/geojson-literal-empty

abstract-test

/conf/geometry-extension/geometry-as-geojson-literal

abstract-test

/conf/geometry-extension/asGeoJSON-function

abstract-test

/conf/geometry-extension/kml-literal

abstract-test

/conf/geometry-extension/kml-literal-srs

abstract-test

/conf/geometry-extension/kml-literal-empty

abstract-test

/conf/geometry-extension/geometry-as-kml-literal

abstract-test

/conf/geometry-extension/asKML-function

This Conformance Class applies to non-DGGS geometries. See [DGGS Conformance Class: Geometry Extension - DGGS](#) for DGGS geometries.

A.3.1. Tests for all Serializations except DGGS

target

/req/geometry-extension/geometry-class

test-purpose

Check conformance with this requirement

test-method

Verify that queries involving `geo:Geometry` return the correct result on a test dataset

test-method-type

Capabilities

reference

[Section 5.6.1](#)

target

/req/geometry-extension/geometry-collection-class

test-purpose

Check conformance with this requirement

test-method

Verify that queries involving [Geometry Collection](#) return the correct result on a test dataset

test-method-type

Capabilities

reference

[Section 5.6.2](#)

target

/req/geometry-extension/feature-properties

test-purpose

Check conformance with this requirement

test-method

Verify that queries involving the following properties return the correct result for a test dataset: [geo:hasGeometry](#), [geo:hasDefaultGeometry](#), [geo:hasLength](#), [geo:hasArea](#), [geo:hasVolume](#), [geo:hasCentroid](#), [geo:hasBoundingBox](#) and [geo:hasSpatialResolution](#)

test-method-type

Capabilities

reference

[Section 3.4](#)

target

/req/geometry-extension/geometry-properties

test-purpose

Check conformance with this requirement

test-method

Verify that queries involving these properties return the correct result for a test dataset:

geo:dimension, geo:coordinateDimension, geo:spatialDimension, geo:hasSpatialResolution, geo:hasMetricSpatialResolution, geo:hasSpatialAccuracy, geo:hasMetricSpatialAccuracy, geo:isEmpty, geo:isSimple and geo:hasSerialization

test-method-type

Capabilities

reference

[Section 5.7](#)

target

/req/geometry-extension/query-functions

test-purpose

Check conformance with this requirement

test-method

Verify that a set of SPARQL queries involving each of the following functions returns the correct result for a test dataset when using the specified serialization and version: geof:boundary geof:boundingCircle, geof:metricBuffer, geof:buffer, geof:centroid, geof:convexHull, geof:concaveHull, geof:coordinateDimension, geof:difference, geof:dimension, geof:metricDistance, geof:distance, geof:envelope, geof:geometryType, geof:intersection, geof:is3D, geof:isEmpty, geof:isMeasured, geof:isSimple, geof:spatialDimension, geof:symDifference, geof:transform and geof:union.

test-method-type

Capabilities reference: [Section 5.9](#)

target

/req/geometry-extension/srid-function

test-purpose

Check conformance with this requirement

test-method

Verify that a SPARQL query involving the [get SRID](#) function returns the correct result for a test dataset when using the specified serialization and version.

test-method-type

Capabilities

reference

[Section 5.9.24](#)

target

/req/geometry-extension/sa-functions

test-purpose

Check conformance with this requirement

test-method

Verify that a set of SPARQL queries involving each of the following functions returns the correct result for a test dataset: `geof:aggBoundingBox`, `geof:aggBoundingBoxCircle`, `geof:aggCentroid`, `geof:aggConcaveHull`, `geof:aggConvexHull` and `geof:aggUnion`

test-method-type

Capabilities

reference

[Section 5.11](#)

A.3.2. WKT Serialization

target

/req/geometry-extension/wkt-literal

test-purpose

Check conformance with this requirement

test-method

Verify that queries involving [WKT Literal](#) values return the correct result for a test dataset.

test-method-type

Capabilities

reference

[Section 5.8.1.1](#)

target

/req/geometry-extension/wkt-literal-default-srs

test-purpose

Check conformance with this requirement

test-method

Verify that queries involving [WKT Literal](#) values without an explicit encoded SRS IRI return the correct result for a test dataset.

test-method-type

Capabilities

reference

[Example 18](#)

target

/req/geometry-extension/wkt-axis-order

test-purpose

Check conformance with this requirement

test-method

Verify that queries involving [WKT Literal](#) values return the correct result for a test dataset.

test-method-type

Capabilities

reference

[Example 19](#)

target

/req/geometry-extension/wkt-literal-empty

test-purpose

Check conformance with this requirement

test-method

Verify that queries involving empty [WKT Literal](#) values return the correct result for a test dataset.

test-method-type

Capabilities

reference

[Example 20](#)

target

/req/geometry-extension/geometry-as-wkt-literal

test-purpose

Check conformance with this requirement

test-method

Verify that queries involving the `geo:asWKT` property return the correct result for a test dataset.

test-method-type

Capabilities

reference

[Section 5.8.1.2](#)

target

/req/geometry-extension/asWKT-function

test-purpose

Check conformance with this requirement

test-method

Verify that a set of SPARQL queries involving the `geof:asWKT` function returns the correct result for a test dataset when using the specified serialization and version.

test-method-type

Capabilities

reference

[Section 5.8.1.3](#)

A.3.3. GML Serialization

target

/req/geometry-extension/gml-literal

test-purpose

Check conformance with this requirement

test-method

Verify that queries involving `geo:gmlLiteral` values return the correct result for a test dataset.

test-method-type

Capabilities

reference

[Section 5.8.3.1](#)

target

/req/geometry-extension/gml-literal-empty

test-purpose

Check conformance with this requirement

test-method

Verify that queries involving empty **geo:gmlLiteral** values return the correct result for a test dataset.

test-method-type

Capabilities

reference

[Example 24](#)

target

/req/geometry-extension/gml-profile

test-purpose

Check conformance with this requirement

test-method

Examine the implementation's documentation to verify that the supported GML profiles are documented.

test-method-type

Capabilities

reference

[Example 25](#)

target

/req/geometry-extension/geometry-as-gml-literal

test-purpose

Check conformance with this requirement

test-method

Verify that queries involving the **geo:asGML** property return the correct result for a test dataset.

test-method-type

Capabilities

reference

[Example 26](#)

target

/req/geometry-extension/asGML-function

test-purpose

Check conformance with this requirement

test-method

Verify that a set of SPARQL queries involving the `geof:asGML` function returns the correct result for a test dataset when using the specified serialization and version.

test-method-type

Capabilities

reference

[Section 5.8.3.3](#)

A.3.4. GeoJSON Serialization

target

/req/geometry-extension/geojson-literal

test-purpose

Check conformance with this requirement

test-method

Verify that queries involving `geo:geoJSONLiteral` values return the correct result for a test dataset.

test-method-type

Capabilities

reference

[Section 5.8.4.1](#)

target

/req/geometry-extension/geojson-literal-srs

test-purpose

Check conformance with this requirement

test-method

Verify that queries involving `geo:geoJSONLiteral` values without an explicit encoded SRS IRI return the correct result for a test dataset.

test-method-type

Capabilities

reference

[Example 29](#)

target

/req/geometry-extension/geojson-literal-empty

test-purpose

Check conformance with this requirement

test-method

Verify that queries involving empty `geo:geoJSONLiteral` values return the correct result for a test dataset.

test-method-type

Capabilities

reference

[Example 30](#)

target

/req/geometry-extension/geometry-as-geojson-literal

test-purpose

Check conformance with this requirement

test-method

Verify that queries involving the `geo:asGeoJSON` property return the correct result for a test dataset.

test-method-type

Capabilities

reference

[Example 31](#)

target

/req/geometry-extension/asGeoJSON-function

test-purpose

Check conformance with this requirement

test-method

Verify that a set of SPARQL queries involving the `geof:asGeoJSON` function returns the correct result for a test dataset when using the specified serialization and version.

test-method-type

Capabilities

reference

[Section 5.8.4.3](#)

A.3.5. KML Serialization

target

/req/geometry-extension/kml-literal

test-purpose

Check conformance with this requirement

test-method

Verify that queries involving `geo:kmlLiteral` values return the correct result for a test dataset.

test-method-type

Capabilities

reference

[Section 5.8.5.1](#)

target

/req/geometry-extension/kml-literal-srs

test-purpose

Check conformance with this requirement

test-method

Verify that queries involving `geo:kmlLiteral` values without an explicit encoded SRS IRI return the correct result for a test dataset.

test-method-type

Capabilities

reference

[Example 34](#)

target

/req/geometry-extension/kml-literal-empty

test-purpose

Check conformance with this requirement

test-method

Verify that queries involving empty `geo:kmlLiteral` values return the correct result for a test dataset.

test-method-type

Capabilities

reference

[Example 35](#)

target

/req/geometry-extension/geometry-as-kml-literal

test-purpose

Check conformance with this requirement

test-method

Verify that queries involving the `geo:asKML` property return the correct result for a test dataset.

test-method-type

Capabilities

reference

[Example 36](#)

target

/req/geometry-extension/asKML-function

test-purpose

Check conformance with this requirement

test-method

Verify that a set of SPARQL queries involving the `geof:asKML` function returns the correct result for a test dataset when using the specified serialization and version.

test-method-type

Capabilities

reference

[Section 5.8.5.3](#)

DGGS Conformance Class: Geometry Extension - DGGS

This conformance Class applies only to DGGS geometries. See [Section A.3](#) for other geometries.

Example 64. Geometry Extension DGGS

target

/req/geometry-extension-dggs

abstract-test

/conf/geometry-extension-dggs/query-functions

abstract-test

/conf/geometry-extension-dggs/query-functions-non-sf

abstract-test

/conf/geometry-extension-dggs/srid-function

abstract-test

/conf/geometry-extension-dggs/sa-functions

abstract-test

/conf/geometry-extension-dggs/dggs-literal

abstract-test

/conf/geometry-extension-dggs/dggs-literal-empty

abstract-test

/conf/geometry-extension-dggs/geometry-as-dggs-literal

abstract-test

/conf/geometry-extension-dggs/asDGGS-function

Tests for DGGS Serializations

target

/req/geometry-extension-dggs/query-functions

test-purpose

Check conformance with this requirement

test-method

Verify that implementations support the functions of Requirement <http://www.opengis.net/spec/geosparql/1.1/req/geometry-extension/query-functions> for DGGS geometry literals as SPARQL extension functions, in a manner which is consistent with definitions of these functions in Simple Features [OGCSFACA] [ISO19125-1], for non-DGGS geometry literals.test-method-type:: Capabilities

target

/req/geometry-extension-dggs/query-functions-non-sf

test-purpose

Check conformance with this requirement

test-method

Verify that implementations support the functions of Requirement <http://www.opengis.net/spec/geosparql/1.1/req/geometry-extension/query-functions-non-sf> for DGGS geometry literals as SPARQL extension functions which are defined in this standard, for non-DGGS geometry literals.

target

/req/geometry-extension-dggs/srid-function

test-purpose

Check conformance with this requirement

test-method

Verify that Implementations shall support geof:getSRID as a SPARQL extension function for DGGS geometry literals.

reference

[Section 5.9.24](#)

target

/req/geometry-extension-dggs/sa-functions

test-purpose

Check conformance with this requirement

test-method

Verify that implementations support the functions of Requirement A.3.1.8 /conf/geometry-extension/sa-functions as SPARQL extension functions which are defined in this standard, for DGGs geometry literals, in a manner which is consistent with definitions of these functions in Simple Features [\[OGCSFACA\]](#) [\[ISO19125-1\]](#).

test-method-type

Capabilities

reference

[\[_spatial_aggregate_functions\]](#)

DGGS Serialization

target

/req/geometry-extension-dggs/dggs-literal

test-purpose

Check conformance with this requirement

test-method

Verify that queries involving empty `geo:dggsLiteral` values return the correct result for a test dataset.

test-method-type

Capabilities

reference

[Section 5.8.7.1](#)

target

/req/geometry-extension-dggs/dggs-literal-empty

test-purpose

Check conformance with this requirement

test-method

Verify that queries involving empty `geo:dggsLiteral` values return the correct result for a test dataset.

test-method-type

Capabilities

reference

[Example 44](#)

target

/req/geometry-extension-dggs/geometry-as-dggs-literal

test-purpose

Check conformance with this requirement

test-method

Verify that queries involving the **geo:asDggs** property return the correct result for a test dataset.

test-method-type

Capabilities

reference

[Example 45](#)

target

/req/geometry-extension-dggs/asDggs-function

test-purpose

Check conformance with this requirement

test-method

Verify that a set of SPARQL queries involving the **geof:asDggs** function returns the correct result for a test dataset when using the specified serialization and version.

test-method-type

Capabilities

reference

[Section 5.8.7.3](#)

A.4. Conformance Class: Geometry Topology Extension

Example 65. Geometry Topology Extension

target

/req/geometry-topology-extension

abstract-test

/conf/geometry-topology-extension/relate-query-function

abstract-test

/conf/geometry-topology-extension/sf-query-functions

abstract-test

/conf/geometry-topology-extension/eh-query-functions

abstract-test

/conf/geometry-topology-extension/rcc8-query-functions

A.4.1. Tests for all relation families

target

/req/geometry-topology-extension/relate-query-function

test-purpose

Check conformance with this requirement

test-method

Verify that a set of SPARQL queries involving the `geof:relate` function returns the correct result for a test dataset when using the specified serialization and version.

test-method-type

Capabilities

reference

[\[Function: geof:relate\]](#)

A.4.2. Simple Features Relation Family

target

/req/geometry-topology-extension/sf-query-functions

test-purpose

Check conformance with this requirement

test-method

Verify that a set of SPARQL queries involving each of the following functions returns the correct result for a test dataset when using the specified serialization and version: `geof:sfEquals`, `geof:sfDisjoint`, `geof:sfIntersects`, `geof:sfTouches`, `geof:sfCrosses`, `geof:sfWithin`, `geof:sfContains`, `geof:sfOverlaps`.

test-method-type

Capabilities

reference

[Section 6.3](#)

A.4.3. Egenhofer Relation Family

target

/req/geometry-topology-extension/eh-query-functions

test-purpose

Check conformance with this requirement

test-method

Verify that a set of SPARQL queries involving each of the following functions returns the correct result for a test dataset when using the specified serialization and version: `geof:ehEquals`, `geof:ehDisjoint`, `geof:ehMeet`, `geof:ehOverlap`, `geof:ehCovers`, `geof:ehCoveredBy`, `geof:ehInside`, `geof:ehContains`.

test-method-type

Capabilities

reference

[Section 6.4](#)

A.4.4. RCC8 Relation Family

target

/req/geometry-topology-extension/rcc8-query-functions

test-purpose

Check conformance with this requirement

test-method

Verify that a set of SPARQL queries involving each of the following functions returns the correct result for a test dataset when using the specified serialization and version: `geof:rcc8eq`, `geof:rcc8dc`, `geof:rcc8ec`, `geof:rcc8po`, `geof:rcc8tpi`, `geof:rcc8tp`, `geof:rcc8ntpp`, `geof:rcc8ntppi`.

test-method-type

Capabilities

reference

[Section 6.5](#)

A.5. Conformance Class: RDFS Entailment Extension

target

/req/rdfs-entailment-extension

abstract-test

/conf/rdfs-entailment-extension/bgp-rdfs-ent

abstract-test

/conf/rdfs-entailment-extension/wkt-geometry-types

abstract-test

/conf/rdfs-entailment-extension/gml-geometry-types

A.5.1. Tests for all implementations

target

/req/rdfs-entailment-extension/bgp-rdfs-ent

test-purpose

Check conformance with this requirement

test-method

Verify that a set of SPARQL queries involving entailed RDF triples returns the correct result for a test dataset using the specified serialization, version and relation_family.

test-method-type

Capabilities

reference

[Clause 7](#)

A.5.2. WKT Serialization

target

/req/rdfs-entailment-extension/wkt-geometry-types

test-purpose

Check conformance with this requirement

test-method

Verify that a set of SPARQL queries involving WKT Geometry types returns the correct result for a test dataset using the specified version of Simple Features.

test-method-type

Capabilities

reference

[Section 7.3](#)

A.5.3. GML Serialization

target

/req/rdfs-entailment-extension/gml-geometry-types

test-purpose

Check conformance with this requirement

test-method

Verify that a set of SPARQL queries involving GML Geometry types returns the correct result for a test dataset using the specified version of GML.

test-method-type

Capabilities

reference

[Section 7.4](#)

A.6. Conformance Class: Query Rewrite Extension

Example 67. Query Rewrite Extension

target

/req/query-rewrite-extension

abstract-test

/conf/query-rewrite-extension/sf-query-rewrite

abstract-test

/conf/query-rewrite-extension/eh-query-rewrite

abstract-test

/conf/query-rewrite-extension/rcc8-query-rewrite

A.6.1. Simple Features Relation Family

target

/req/query-rewrite-extension/sf-query-rewrite

test-purpose

Check conformance with this requirement

test-method

Verify that queries involving the following query transformation rules return the correct result for a test dataset when using the specified serialization and version: `geor:sfEquals`, `geor:sfDisjoint`, `geor:sfIntersects`, `geor:sfTouches`, `geor:sfCrosses`, `geor:sfWithin`, `geor:sfContains` and `geor:sfOverlaps`.

test-method-type

Capabilities

reference

[\[query_rewrite_sf\]](#)

A.6.2. Egenhofer Relation Family

target

/req/query-rewrite-extension/eh-query-rewrite

test-purpose

Check conformance with this requirement

test-method

Verify that queries involving the following query transformation rules return the correct result for a test dataset when using the specified serialization and version: `geor:ehEquals`, `geor:ehDisjoint`, `geor:ehMeet`, `geor:ehOverlap`, `geor:ehCovers`, `geor:ehCoveredBy`, `geor:ehInside`, `geor:ehContains`.

test-method-type

Capabilities

reference

[\[query_rewrite_eh\]](#)

A.6.3. RCC8 Relation Family

target

/req/query-rewrite-extension/rcc8-query-rewrite

test-purpose

Check conformance with this requirement

test-method

Verify that queries involving the following query transformation rules return the correct result for a test dataset when using the specified serialization and version: `geor:rcc8eq`, `geor:rcc8dc`, `geor:rcc8ec`, `geor:rcc8po`, `geor:rcc8tppi`, `geor:rcc8tpp`, `geor:rcc8ntpp`, `geor:rcc8ntppi`.

test-method-type

Capabilities

reference

[\[query_rewrite_rcc8\]](#)

Appendix B: Functions Summary

B.0. Overview

This annex summarizes all the functions defined in GeoSPARQL, providing descriptions of their parameters and return types.

The value `ogc:geomLiteral` indicates any one of the specific geometry serializations datatypes defined in this Specification, for example `geo:wktLiteral`.

The geometry subtypes - Polygon, Point, CellList etc. - are the *Simple Features specification* [OGCSFACA] [ISO19125-1] or DGGs types, as implemented by the various geometry serialization specifications referenced here. See [Section 5.8](#) for the individual specification references.

B.1. Functions Summary Table

GeoSPARQL Functions Summary

Simple Features Functions				
Function	Input Datatypes	Input Subtypes	Output Datatype	Output Subtype
sfContains	2x ogc:geomLiteral	1x Polygon, 1x Geometry	xsd:boolean	
sfCrosses	2x ogc:geomLiteral	1x Point or LineString, 1 x LineString or Polygon	xsd:boolean	
sfDisjoint	2x ogc:geomLiteral	2x Geometry	xsd:boolean	
sfEquals	2x ogc:geomLiteral	2x Geometry	xsd:boolean	
sfIntersects	2x ogc:geomLiteral	2x Polygon	xsd:boolean	
sfOverlaps	2x ogc:geomLiteral	2x Point or 2x LineString or 2x Polygon	xsd:boolean	
sfTouches	2x ogc:geomLiteral	2x Geometry but not Point	xsd:boolean	
sfWithin	2x ogc:geomLiteral	1x Geometry, 1x Polygon	xsd:boolean	
Egenhofer Functions				
Function	Input Datatypes	Input Subtypes	Output Datatype	Output Subtype

ehContains	2x ogc:geomLiteral	1x Polygon, 1x Geometry	xsd:boolean	
ehCoveredBy	2x ogc:geomLiteral	1x Polygon, 1x Geometry	xsd:boolean	
ehCovers	2x ogc:geomLiteral	1x Polygon, 1x Geometry	xsd:boolean	
ehDisjoint	2x ogc:geomLiteral	2x Geometry	xsd:boolean	
ehEquals	2x ogc:geomLiteral	2x Geometry	xsd:boolean	
ehMeet	2x ogc:geomLiteral	2x Geometry but not Point	xsd:boolean	
ehOverlap	2x ogc:geomLiteral	2x Geometry	xsd:boolean	
ehInside	2x ogc:geomLiteral	2x Geometry	xsd:boolean	
Region Connection Calculus Functions				
Function	Input Datatypes	Input Subtypes	Output Datatype	Output Subtype
rcc8dcc	2x ogc:geomLiteral	2x Polygon	xsd:boolean	
rcc8ecc	2x ogc:geomLiteral	2x Polygon	xsd:boolean	
rcc8eq	2x ogc:geomLiteral	2x Polygon	xsd:boolean	
rcc8ntpp	2x ogc:geomLiteral	2x Polygon	xsd:boolean	
rcc8ntppi	2x ogc:geomLiteral	2x Polygon	xsd:boolean	
rcc8po	2x ogc:geomLiteral	2x Polygon	xsd:boolean	
rcc8tp	2x ogc:geomLiteral	2x Polygon	xsd:boolean	
rcc8tpi	2x ogc:geomLiteral	2x Polygon	xsd:boolean	
Spatial Aggregate Functions				
Function	Input Datatypes	Input Subtypes	Output Datatype	Output Subtype
aggBoundingBox	1 or more ogc:geomLiteral		ogc:geomLiteral	square Polygon (not DGGS), CellList (DGGS)

aggBoundingCircle	1 or more ogc:geomLiteral		ogc:geomLiteral	Polygon (not DGGS) CellList (DGGS)
aggCentroid	1 or more ogc:geomLiteral		ogc:geomLiteral	Point (not DGGS), Cell (DGGS)
aggConcaveHull	1 or more ogc:geomLiteral		ogc:geomLiteral	Polygon (not DGGS), CellList (DGGS)
aggConvexHull	1 or more ogc:geomLiteral		ogc:geomLiteral	Polygon (not DGGS), CellList (DGGS)
aggUnion	1 or more ogc:geomLiteral		ogc:geomLiteral	Polygon (not DGGS), CellList (DGGS)
Non-topological Query Functions				
Function	Input Datatypes	Input Subtypes	Output Datatype	Output Subtype
metricArea	1x ogc:geomLiteral	Polygon	xsd:double	
area	1x ogc:geomLiteral	Polygon	xsd:double	
boundary	1x ogc:geomLiteral	Geometry	ogc:geomLiteral	LineString (not DGGS), OrderedCellList (DGGS)
buffer	1x ogc:geomLiteral, 1x xsd:double , 1x xsd:anyURI	any	ogc:geomLiteral	(Multi)Polygon (not DGGS), CellList (DGGS)
convexHull	1x ogc:geomLiteral	Geometry	ogc:geomLiteral	LineString (not DGGS)
coordinateDimension	1x ogc:geomLiteral	Geometry	xsd:integer	

difference	2x ogc:geomLiteral	2x Geometry	ogc:geomLiteral	(Multi)Polygon (not DGGS), CellList (DGGS)
easting	1x ogc:geomLiteral	Point	xsd:double	
dimension	1x ogc:geomLiteral	Geometry	xsd:double	
metricDistance	2x ogc:geomLiteral, 1x xsd:anyURI	2x Geometry	xsd:double	
distance	2x ogc:geomLiteral, 1x xsd:anyURI	2x Geometry	rdfs:Resource	
envelope	1x ogc:geomLiteral, 1x xsd:anyURI	Geometry	ogc:geomLiteral	(Multi)Polygon (not DGGS), CellList (DGGS)
geometryN	1x ogc:geomLiteral	GeometryCollection (not DGGS)	xsd:double	
geometryType	1x ogc:geomLiteral	Geometry	xsd:anyURI	
getSRID	1x ogc:geomLiteral	Geometry	xsd:anyURI	
intersection	2x ogc:geomLiteral	2x Geometry	ogc:geomLiteral	Polygon (not DGGS), CellList (DGGS)
is3D	1x ogc:geomLiteral	Geometry	xsd:boolean	
isEmpty	1x ogc:geomLiteral	Geometry	xsd:boolean	
isMeasured	1x ogc:geomLiteral	Geometry	xsd:boolean	
isSimple	1x ogc:geomLiteral	Geometry	xsd:boolean	
metricLength	1x ogc:geomLiteral	Geometry	xsd:double	

length	1x ogc:geomLiteral	Geometry	rdfs:Resource	
numGeometries	1x ogc:geomLiteral	Geometry (not DGGs)	xsd:double	
northing	1x ogc:geomLiteral	Point	xsd:double	
metricPerimeter	1x ogc:geomLiteral	Geometry	xsd:double	
perimeter	1x ogc:geomLiteral	Geometry	rdfs:Resource	
spatialDimension	1x ogc:geomLiteral	Geometry	xsd:integer	
symDifference	2x ogc:geomLiteral	2x Geometry	ogc:geomLiteral	(Multi)Polygon (not DGGs), CellList DGGs)
transform	1x ogc:geomLiteral, 1x xsd:anyURI	Geometry	ogc:geomLiteral	Geometry
union	2x ogc:geomLiteral	2x Geometry	ogc:geomLiteral	Polygon (not DGGs), CellList (DGGs)
Non-topological Query Functions - 3D Extension				
Function	Input Datatypes	Input Subtypes	Output Datatype	Output Subtype
extrude	2x ogc:geomLiteral	1x Polygon, 1 x LineString	geo:Feature	
extrudeToHeight	1x ogc:geomLiteral, 1x xsd:double	1x Polygon	geo:Feature	
extrudeInTime	2x ogc:geomLiteral, 2x xsd:dateTime	1x Polygon, 1 x LineString	geo:Feature	
Serialization Functions				
Function	Input Datatypes	Input Subtypes	Output Datatype	Output Subtype
asDGGs	1x ogc:geomLiteral	Geometry	geo:dggLiteral	
asGeoJSON	1x ogc:geomLiteral	Geometry	geo:geoJSONLiteral	

asGML	1x ogc:geomLiteral, 1x xsd:string	Geometry	geo:gmlLiteral	
asGeoCode	1x ogc:geomLiteral	Geometry	geo:geocodeLiteral	
asKML	1x ogc:geomLiteral	Geometry	geo:kmlLiteral	
asWKT	1x ogc:geomLiteral	Geometry	geo:wktLiteral	
Extent Functions				
Function	Input Datatypes	Input Subtypes	Output Datatype	Output Subtype
getSRID	1x ogc:geomLiteral	Geometry	xsd:anyURI	
maxX	1x ogc:geomLiteral	Geometry	xsd:double	
maxY	1x ogc:geomLiteral	Geometry	xsd:double	
maxZ	1x ogc:geomLiteral	Geometry	xsd:double	
minX	1x ogc:geomLiteral	Geometry	xsd:double	
minY	1x ogc:geomLiteral	Geometry	xsd:double	
minZ	1x ogc:geomLiteral	Geometry	xsd:double	
Other Functions				
Function	Input Datatypes	Input Subtypes	Output Datatype	Output Subtype
relate	2x ogc:geomLiteral		xsd:string	

B.2. GeoSPARQL to SFA Functions Mapping

The following table indicates which GeoSPARQL non-topological query functions map to Simple Features Access ([[OGCSFACA](#)] [[ISO19125-1](#)]) functions and in which GeoSPARQL version the functions are defined.

Where the Simple Features Access function has the same name as the GeoSPARQL function, 'x' is recorded.

GeoSPARQL To SFA Mappings

GeoSPARQL Function	in 1.0	in 1.1	SFA
metricArea		x	Area
area		x	Area
			AsBinary
asWKT*	x	x	AsText
boundary	x	x	Boundary
buffer	x	x	Buffer
			Centroid
convexHull	x	x	ConvexHull
coordinateDimension		x	
difference	x	x	Difference
dimension		x	Dimension
metricDistance		x	Distance
distance	x	x	Distance
			EndPoint
envelope	x	x	Envelope
geometryN		x	GeometryN
geometryType		x	GeometryType
getSRID	x	x	SRID
			InteriorRingN
intersection	x	x	Intersection
is3D		x	
			IsClosed
isEmpty		x	IsEmpty
isMeasured		x	
			IsRing
isSimple		x	IsSimple
metricLength		x	Length
length		x	Length
maxX		x	
maxY		x	
maxZ		x	
minX		x	
minY		x	

GeoSPARQL Function	in 1.0	in 1.1	SFA
minZ		x	
numGeometries		x	NumGeometries
			NumInteriorRing
			NumPoints
perimeterLength		x	
perimeter		x	
			PointN
			PointOnSurface
spatialDimension		x	
			StartPoint
symDifference	x	x	SymDifference
transform		x	
union	x	x	Union
			X
			Y

* GeoSPARQL's **asWKT** is only a partial implementation of **asText** since **asWKT** only returns WKT, not textual geometry literal data in general.

Appendix C: GeoSPARQL Examples

C.0. Overview

This Annex provides examples of the GeoSPARQL ontology and functions. In addition to these, extended examples are provided separately by the GeoSPARQL 1.1 profile. See the [GeoSPARQL Standard structure](#) for the link to those examples.

C.1. RDF Examples

This Section illustrates GeoSPARQL ontology modelling with extended examples.

C.1.1. Classes

SpatialObject

The **SpatialObject** class is defined in [Section 3.2.1](#).

Basic use

Basic use (as per the example in the class definition)

```
eg:x
  a geo:SpatialObject ;
  skos:prefLabel "Object X";
.
```

NOTE

It is unlikely that users of GeoSPARQL will create many instances of **geo:SpatialObject** as its two more concrete subclasses, **geo:Feature** & **geo:Geometry**, are more directly relatable to real-world phenomena and use.

Size Properties

The "size" properties - **geo:hasSize**, **geo:hasMetricSize**, **geo:hasLength**, **geo:hasMetricLength**, **geo:hasPerimeterLength**, **geo:hasMetricPerimeterLength**, **geo:hasArea**, **geo:hasMetricArea**, **geo:hasVolume** and **geo:hasMetricVolume** - are all applicable to instances of **geo:SpatialObject** although, as per the note in the section above, they are likely to be used with **geo:Feature** & **geo:Geometry** instances.

```
@prefix qudt: <http://qudt.org/schema/qudt/> .
@prefix unit: <http://qudt.org/vocab/unit/> .

eg:moreton-island
  a geo:SpatialObject ;

  skos:prefLabel "Moreton Island" ;
  rdfs:seeAlso "https://en.wikipedia.org/wiki/Moreton_Island"^^xsd:anyURI ;
```

```

geo:hasPerimeterLength [
  qudt:numericValue "92.367"^^xsd:float ;
  qudt:unit unit:KiloM ;
];
geo:hasMetricPerimeterLength "92367"^^xsd:double ;
.

```

Here a spatial object, Moreton Island, has the distance of its coastline given with two properties: `geo:hasPerimeterLength` & `geo:hasMetricPerimeterLength`. The object for the first is a Blank Node with a QUDT value property of 92.367 and a QUDT unit property of `unit:KiloM` (kilometre). The object for the second is the literal `92367` (a double) which is, by the property's definition, a number of metres.

The use of the *Quantities, Units, Dimensions and Types (QUDT)* ontology^[1] and its `qudt:numericValue` & `qudt:unit` is just one of many possible ways to convey the value of `geo:hasPerimeterLength` and any subproperty of `geo:hasSize`.

Feature

The `Feature` class is defined in [Section 3.2.2](#).

Basic use

```

eg:x
  a geo:Feature ;
  skos:prefLabel "Feature X" ;
.

```

Here a `Feature` is declared and given a preferred label.

A `Feature` related to a `Geometry` and a `GeometryCollection`

```

eg:x
  a geo:Feature ;
  skos:prefLabel "Feature X" ;
  geo:hasGeometry [
    geo:asWKT "MULTIPOLYGON (((149.06016 -35.23610, 149.06062 -35.23604, ... ,
149.06016 -35.23610)))"^^geo:wktLiteral ;
  ] ;
.

```

Here a `geo:Feature` is declared, given a preferred label and a `Geometry` for that `geo:Feature` is indicated with the use of `geo:hasGeometry`. The `Geometry` indicated is described using a *Well-Known Text* literal value, indicated by the property `geo:asWKT` and the literal type `geo:wktLiteral`.

```

eg:x
  a geo:Feature ;
  skos:prefLabel "Feature X" ;

```

```

    geo:hasGeometry [
      a geo:GeometryCollection ;
      rdfs:member [
        geo:asWKT "POLYGON ((149.06016 -35.23610, 149.06062 -35.23604, ... ,
149.06016 -35.23610))"^^geo:wktLiteral ;
      ] ,
      [
        geo:asWKT "POINT (149.061 -35.23607)"^^geo:wktLiteral ;
      ] ;
    ] ;
.

```

Here a `geo:Feature` is declared and linked to a `geo:GeometryCollection` in order to bundle multiple `geo:Geometry` instances together. Using this mechanism, you could add additional properties to the `geo:GeometryCollection` to detail useful things about the set of Geometries. This is an alternative to the pattern given in `Feature with two different Geometry instances indicated` example below.

Feature with Geometry and size (area)

```

eg:x
  a geo:Feature ;
  skos:prefLabel "Feature X" ;
  geo:hasGeometry [
    geo:asWKT "MULTIPOLYGON (((149.06016 -35.23610, 149.06062 -35.23604, ... ,
149.06016 -35.23610)))"^^geo:wktLiteral ;
  ] ;
  geo:hasMetricArea "8.9E4"^^xsd:double ;
.

```

This example and the example below (B 1.1.2.4) show the same `geo:Feature`, but with a different specification of its area. This example shows the recommended way to express size: by using a subproperty of `geo:hasMetricSize` (in this case, [Section 3.3.8](#)). These subproperties have fixed units based on meter (the unit of distance in the International System of Units).

Feature with Geometry and non-metric size

```

@prefix qudt: <http://qudt.org/schema/qudt/> .

eg:x
  a geo:Feature ;
  skos:prefLabel "Feature X";
  geo:hasGeometry [
    geo:asWKT "MULTIPOLYGON (((149.06016 -35.23610, 149.06062 -35.23604, ... ,
149.06016 -35.23610)))"^^geo:wktLiteral ;
  ] ;
  geo:hasArea [
    qudt:numericValue "2.2E5"^^xsd:double ;
    qudt:unit <http://qudt.org/vocab/unit/AC> ; # international acre
  ] ;
.

```

```
] ;
```

```
.
```

Here a **geo:Feature** is described as per the previous example but its area is expressed in non-metric units: the acre.

Feature with two different Geometry instances indicated

```
eg:x
  a geo:Feature ;
  skos:prefLabel "Feature X";
  geo:hasGeometry [
    rdfs:label "Official boundary" ;
    rdfs:comment "Official boundary from the Department of Xxx" ;
    geo:asWKT "MULTIPOLYGON (((149.06016 -35.23610, 149.06062 -35.23604, ... ,
149.06016 -35.23610)))"^^geo:wktLiteral ;
  ] ,
  [
    rdfs:label "Unofficial boundary" ;
    rdfs:comment "Unofficial boundary as actually used by everyone" ;
    geo:asWKT "MULTIPOLYGON (((149.06016 -35.23610, 149.06062 -35.23604, ... ,
149.06016 -35.23610)))"^^geo:wktLiteral ;
  ] ;
.
```

In this example, **Feature X** has two different **Geometry** instances indicated with their difference explained in annotation properties. No GeoSPARQL ontology properties are used to indicate a difference in these **Geometry** instances thus machine use of this **Feature** would not be easily able to differentiate them.

Feature with two different Geometry instances with different property values

```
eg:x
  a geo:Feature ;
  skos:prefLabel "Feature X";
  geo:hasGeometry [
    geo:hasMetricSpatialResolution "100"^^xsd:double ;
    geo:asWKT "MULTIPOLYGON (((149.0601 -35.2361, 149.0606 -35.2360, ... ,
149.0601 -35.2361)))"^^geo:wktLiteral ;
  ] ,
  [
    geo:hasMetricSpatialResolution "5"^^xsd:double ;
    geo:asWKT "MULTIPOLYGON (((149.06016 -35.23610, 149.06062 -35.23604, ... ,
149.06016 -35.23610)))"^^geo:wktLiteral ;
  ] ;
.
```

In this example, **Feature X** has two different **Geometry** instances indicated with different spatial

resolutions. Machine use of this **Feature** would be able to differentiate the two **Geometry** instances based on this use of **geo:hasMetricSpatialResolution**.

Feature with non-metric size

```
@prefix dbp: <http://dbpedia.org/resource/> .
@prefix qudt: <http://qudt.org/schema/qudt/> .

ex:Seleucia_Artemita
  a geo:Feature ;
  skos:prefLabel "The route from Seleucia to Artemita"@en ;
  geo:hasLength [
    qudt:unit ex:Schoenus ;
    qudt:value "15"^^xsd:integer ;
  ]
.
ex:Schoenus
  a qudt:Unit;
  skos:exactMatch dbp:Schoenus;
.
```

In this example it is not possible to convert the length of the feature to meters, because the historical length unit does not have a known precise conversion factor.

Feature with two different types of **Geometry instances**

```
eg:x
  a geo:Feature ;
  skos:prefLabel "Feature X";
  geo:hasGeometry [
    geo:asWKT "POLYGON ((149.06016 -35.23610, 149.060620 -35.236043, ... ,
149.06016 -35.23610))"^^geo:wktLiteral ;
  ] ;
  geo:hasCentroid [
    geo:asWKT "POINT (149.06017784 -35.23612321)"^^geo:WktLiteral ;
  ] ;
.
```

Here a **Feature** instance has two geometries, one indicated with the general property **hasGeometry** and a second indicated with the specialized property **hasCentroid** which suggests the role that the indicated geometry plays. Note that while **hasGeometry** may indicate any type of **Geometry**, **hasCentroid** should only be used to indicate a point geometry. It may be informally inferred that the polygonal geometry is the **Feature** instance's boundary.

Feature with multiple sizes

```
ex:lake-x
  a geo:Feature ;
```

```

skos:prefLabel "Lake X" ;
eg:hasFeatureCategory <http://example.com/cat/lake> ;
geo:hasMetricArea "9.26E4"^^xsd:double ;
geo:hasMetricVolume "6E5"^^xsd:double ;
.

```

This example shows a **Feature** instance with area and volume declared. A categorization of the Feature is given through the use of the **eg:hasFeatureCategory** dummy property which, along with the Feature's preferred label, indicate that this Feature is a lake. Having both an area and a volume makes sense for a lake.

Geometry

The **Geometry** class is defined in [Section 5.6.1](#).

Basic Use

```

eg:y a geo:Geometry ;
    skos:prefLabel "Geometry Y";
.

```

Here a **Geometry** is declared and given a preferred label.

From GeoSPARQL 1.0 use, the most commonly observed use of a **Geometry** is in relation to a **Feature** as per the first example in [Basic use](#) and often the **Geometry** instance's class type is indirectly declared by the use of **hasGeometry** on the **Feature** instance indicating a Blank Node, given the range value for **hasGeometry**. However, it is entirely possible to declare **Geometry** instances without any **Feature** instances. The next basic example declares a **Geometry** instance with an absolute URI and data.

```

<https://example.com/geometry/y>
  a geo:Geometry ;
  skos:prefLabel "Geometry Y";
  geo:asWKT "MULTIPOLYGON (((149.06016 -35.23610, 149.060620 -35.236043, ... ,
149.06016 -35.23610)))"^^geo:wktLiteral ;
.

```

Here the **Geometry** instance has data in WKT form and, since no CRS is declared, WGS84 is the assumed, default, CRS. This Geometry might have been declared with a named nod, i.e. not with a Blank Node, so that it can be referred to from multiple Features.

A **Geometry** with multiple serializations

```

eg:x
  a geo:Feature ;
  skos:prefLabel "Feature X";
  geo:hasGeometry [

```

```

    geo:asWKT "<http://www.opengis.net/def/crs/EPSSG/0/4326> MULTIPOLYGON (((-
35.23610 149.06016, -35.236043 149.060620, ... , -35.23610
149.06016)))"^^geo:wktLiteral ;
    geo:asDGGS "<https://w3id.org/dggs/auspdx> CELLLIST ((R1234 R1235 R1236 ...
R1256))"^^geo:dggsLiteral ;
  ] ;
.

```

Here a single **Geometry**, linked to a **Feature** instance, is expressed using two different serializations: Well-known Text and the DGGS with the AusPIX DGGS indicated by its IRI.

Geometry with scalar spatial property

```

eg:x
  a geo:Feature ;
  skos:prefLabel "Feature X";
  geo:hasGeometry eg:x-geo ;
.

eg:x-geo
  a geo:Geometry ;
  geo:asWKT "MULTIPOLYGON (((149.06016 -35.23610, 149.060620 -35.236043, ... ,
149.06016 -35.23610)))"^^geo:wktLiteral ;
  geo:hasMetricArea "8.7E4"^^xsd:double;
.

```

This example shows a **Feature**, **eg:x**, with a **Geometry**, **eg:x-geo**, which has both a serialization (WKT) indicated with the predicate **geo:asWKT** and a scalar area indicated with the predicate **geo:hasMetricArea**. While it is entirely possible that scalar areas can be calculated from polygons, it may be efficient to store a pre-calculated scalar area in addition to the polygon. Perhaps the polygon is large and detailed and a one-time calculation with results stored is efficient for repeated use.

This use of a scalar spatial measurement property with a **Geometry**, here **geo:hasMetricArea**, is possible since the domain of such properties is **geo:SpatialObject**, the superclass of both **geo:Feature** and **geo:Geometry**.

SpatialObjectCollection

geo:SpatialObjectCollection isn't really intended to be implemented - it's essentially an abstract class - therefore no examples of its use are given. See the following two sections for examples of the concrete **geo:FeatureCollection** & **geo:GeometryCollection** classes.

FeatureCollection

This example shows a **FeatureCollection** instance containing 3 **Feature** instances and a **hasBoundingBox** predicate instance indicating a **Geometry** - the bounding box of all the **Features** within the **Feature Collection**.

```

ex:fc-x
  a geo:FeatureCollection ;
  dcterms:title "Feature Collection X" ;
  rdfs:member
    ex:feature-something ,
    ex:feature-other ,
    ex:feature-another ;
  geo:hasBoundingBox [
    geo:asWKT "<http://www.opengis.net/def/crs/EPSG/0/4326> POLYGON ((-35.236
149.060, ... , -35.236 149.060)))"^^geo:wktLiteral ;
  ] ;
.

```

All the GeoSPARQL collection classes are unordered since they are subclasses of the generic `rdfs:Container`, however implementers should consider that there are many ways to order the members of a `FeatureCollection` such as the `Feature` instance labels, their areas, geometries or any other property.

GeometryCollection

This example shows a `GeometryCollection` instance containing 3 `Geometry` instances.

```

ex:gc-x
  a geo:GeometryCollection ;
  dcterms:title "Geometry Collection X" ;
  rdfs:member
    ex:geometry-shape ,
    ex:geometry-othershape ,
    ex:geometry-anothershape ;
.

```

As per `FeatureCollection`, the `GeometryCollection` itself doesn't impose any ordering on its member `Geometry` instances, however there are many ways to order them, based on their own properties.

Simple Features classes

Most of the geometry serializations used in GeoSPARQL define the geometry type - point, polygon etc. *within* the literal, e.g. WKT can encode `POLYGON()` or `'POINT()'`, however the *Simple Features Vocabulary* resource within GeoSPARQL 1.1 contains specialised Geometry RDF classes such as `sf:Polygon`, `sf:PolyhedralSurface` and others.

It may be appropriate to use these specialized forms of Geometry in circumstances when geometry type differentiation is required within RDF and not withing specialized literal handling. This is the case when type differentiation must occur within plain SPARQL, not GeoSPARQL.

The following example shows a `Feature` instance with two `Geometry` instances where the *Simple Features Vocabulary* classes are used to indicate the Geometry type:


```

ex:x
  a geo:Feature ;
  rdfs:label "Feature X" ;
  geo:hasGeometry [
    a sf:Point ;
    geo:asWKT "POINT(...)" ;
    rdfs:comment "A point geometry for Feature X, possibly a centroid though not
declared one" ;
  ] ;
  geo:hasGeometry [
    a sf:Polygon ;
    geo:asWKT "POLYGON(...)" ;
    rdfs:comment "A polygon geometry for Feature X" ;
  ] ;

```

There are several GeoSPARQL properties that suggest they could be used with particular *Simple Features Vocabulary* geometry types, for instance, `geo:hasCentroid` indicates it could be used with a `sf:Point` and `geo:hasBoundingBox` indicates use with an `sf:Envelope`.

C.1.2. Properties

C.1.2.1. Spatial Object Properties

See the section [Basic use](#) above.

Feature Properties

This example shows a `geo:Feature` instance with each of the properties defined in [Section 3.4](#) used, except for the properties `geo:hasMetricSize` and `geo:hasSize`, that are intended to be used through their subproperties and `geo:hasMetricPerimeterLength` and `geo:hasPerimeterLength` which are exemplified in [Size Properties](#).

```

@prefix qudt: <http://qudt.org/schema/qudt/> .

eg:x
  a geo:Feature ;
  skos:preferredLabel "Feature X" ;
  geo:hasGeometry [
    geo:asWKT "<http://www.opengis.net/def/crs/EPSG/0/4326> POLYGON ((-35.23610
149.06016, ... , -35.23610 149.06016)))"^^geo:wktLiteral ;
  ] ;
  geo:hasDefaultGeometry [
    geo:asWKT "<http://www.opengis.net/def/crs/EPSG/0/4326> POLYGON ((-35.2361
149.0601, ... , -35.2361 149.0601)))"^^geo:wktLiteral ;
  ] ;
  geo:hasMetricLength "355"^^xsd:double ;
  geo:hasLength [
    qudt:numericValue 355 ;
    qudt:unit <http://qudt.org/vocab/unit/M> ; # meter

```

```

] ;
geo:hasMetricArea "8.7E4"^^xsd:double ;
geo:hasArea [
    qudt:numericValue 8.7 ;
    qudt:unit <http://qudt.org/vocab/unit/HA> ; # hectare
] ;
geo:hasMetricVolume "624432"^^xsd:double ;
geo:hasVolume [
    qudt:numericValue 624432 ;
    qudt:unit <http://qudt.org/vocab/unit/M3> ; # cubic meter
] ;
geo:hasCentroid [
    geo:asWKT "POINT (149.06017 -35.23612)"^^geo:wktLiteral ;
] ;
geo:hasBoundingBox [
    geo:asWKT "<http://www.opengis.net/def/crs/EPSG/0/4326> POLYGON ((149.060
-35.236, ... , 149.060 -35.236)))"^^geo:wktLiteral ;
] ;
geo:hasMetricSpatialResolution "5"^^xsd:double ;
geo:hasSpatialResolution [
    qudt:numericValue 5 ;
    qudt:unit <http://qudt.org/vocab/unit/M> ; # meter
] ;
.

```

The properties defined for this example's **Feature** instance are vaguely aligned in that the values are not real but are not unrealistic either. It is outside the scope of GeoSPARQL to validate **Feature** instances' property values.

Note that this **Feature** has a 2D **Geometry** and yet a property indicating a scalar volume: **geo:hasVolume**. Used in this way, the scalar property is indicating information that cannot be calculated from other information about the **Feature** such as its geometry. Perhaps a volume for the feature has been estimated or measured in such a way that a 3D geometry was not created.

Geometry Properties

This example shows a **Geometry** instance, a Blank Node, declared in relation to a **Feature** instance, with each of the properties defined in [Section 5.7](#) used.

```

@prefix qudt: <http://qudt.org/schema/qudt/> .
@prefix unit: <http://qudt.org/vocab/unit/> .

eg:x
  a geo:Feature ;
  geo:hasGeometry [
    skos:prefLabel "Geometry Y" ;
    geo:dimension 2 ;
    geo:coordinateDimension 2 ;
    geo:spatialDimension 2 ;
    geo:isEmpty false ;

```

```

    geo:isSimple true ;
    geo:hasSerialization "<http://www.opengis.net/def/crs/EPSG/0/4326> POLYGON ((-
35.236 149.060, ... , -35.236 149.060)))"^^geo:wktLiteral ;
    geo:hasSpatialAccuracy [
      qudt:numericValue "30"^^xsd:float ;
      qudt:unit unit:CentiM ; # centimetres
    ] ;
    geo:hasMetricSpatialAccuracy "0.3"^^xsd:double ;
  ] ;
.

```

In this example, each of the properties defined for a **Geometry** instance has realistic values. For example, the **is empty** property is set to **false** since the **Geometry** contains information.

Geometry Serializations

C.1.2.2. Literals

This example shows a **Geometry** instance for a **Feature** instance which is represented in all supported literal GeoSPARQL serializations. The geometry values given are real geometry values and approximate **Moreton Island** in Queensland, Australia.

Note that the concrete DGGS serialization used is for example purposes only as it is not formally defined in GeoSPARQL.

```

eg:x
  a geo:Feature ;
  geo:hasGeometry [
    geo:asWKT ""<http://www.opengis.net/def/crs/EPSG/0/4326>
      POLYGON ((
        -27.0621757 153.3610112,
        -27.1990606 153.3658177,
        -27.3406573 153.421436,
        -27.3607835 153.4269292,
        -27.3315078 153.4434087,
        -27.2913403 153.4183848,
        -27.2039578 153.4189391,
        -27.0267166 153.4673476,
        -27.0621757 153.3610112
      ))""^^geo:wktLiteral ;

    geo:asGML ""<gml:Polygon
      srsName="http://www.opengis.net/def/crs/EPSG/0/4326">
      <gml:exterior>
        <gml:LinearRing>
          <gml:posList>
            -27.0621757 153.3610112
            -27.1990606 153.3658177
            -27.3406573 153.421436
            -27.3607835 153.4269292

```

```

-27.3315078 153.4434087
-27.2913403 153.4183848
-27.2039578 153.4189391
-27.0267166 153.4673476
-27.0621757 153.3610112
    </gml:posList>
  </gml:LinearRing>
</gml:exterior>
</gml:Polygon>""^^go:gmlLiteral ;

geo:asKML ""<Polygon>
  <outerBoundaryIs>
    <LinearRing>
      <coordinates>
        153.3610112,-27.0621757
        153.3658177,-27.1990606
        153.421436,-27.3406573
        153.4269292,-27.3607835
        153.4434087,-27.3315078
        153.4183848,-27.2913403
        153.4189391,-27.2039578
        153.4673476,-27.0267166
        153.3610112,-27.0621757
      </coordinates>
    </LinearRing>
  </outerBoundaryIs>
</Polygon>""^^go:kmlLiteral ;

geo:asGeoJSON ""{
  "type": "Polygon",
  "coordinates": [[
    [153.3610112, -27.0621757],
    [153.3658177, -27.1990606],
    [153.421436, -27.3406573],
    [153.4269292, -27.3607835],
    [153.4434087, -27.3315078],
    [153.4183848, -27.2913403],
    [153.4189391, -27.2039578],
    [153.4673476, -27.0267166],
    [153.3610112, -27.0621757]
  ]]
}""^^geo:geoJSONLiteral ;

geo:asDGGs ""<https://w3id.org/dggs/auspex> CELLLIST ((R8346031 R8346034
R8346037
R83460058 R83460065 R83460068 R83460072 R83460073 R83460074 R83460075
R83460076
R83460077 R83460078 R83460080 R83460081 R83460082 R83460083 R83460084
R83460085
R83460086 R83460087 R83460088 R83460302 R83460305 R83460308 R83460320
R83460321

```

R83460323 R83460324 R83460326 R83460327 R83460332 R83460335 R83460338
R83460350
R83460353 R83460356 R83460362 R83460365 R83460380 R83460610 R83460611
R83460612
R83460613 R83460614 R83460615 R83460617 R83460618 R83460641 R83460642
R83460644
R83460645 R83460648 R83460672 R83460686 R83463020 R83463021 R834600487
R834600488
R834600557 R834600558 R834600564 R834600565 R834600566 R834600567
R834600568
R834600571 R834600572 R834600573 R834600574 R834600575 R834600576
R834600577
R834600578 R834600628 R834600705 R834600706 R834600707 R834600708
R834600712
R834600713 R834600714 R834600715 R834600716 R834600717 R834600718
R834601334
R834601335 R834601336 R834601337 R834601338 R834601360 R834601361
R834601363
R834601364 R834601366 R834601367 R834601600 R834601601 R834601603
R834601606
R834601630 R834601633 R834603220 R834603221 R834603223 R834603224
R834603226
R834603227 R834603250 R834603251 R834603253 R834603256 R834603280
R834603283
R834603510 R834603511 R834603512 R834603513 R834603514 R834603515
R834603516
R834603517 R834603540 R834603541 R834603543 R834603544 R834603546
R834603547
R834603570 R834603573 R834603576 R834603681 R834603682 R834603684
R834603685
R834603687 R834603688 R834603810 R834603830 R834603831 R834603832
R834603833
R834603834 R834603835 R834603836 R834603837 R834603860 R834603861
R834603863
R834603864 R834603866 R834603867 R834606021 R834606022 R834606024
R834606025
R834606028 R834606052 R834606055 R834606160 R834606161 R834606162
R834606164
R834606165 R834606167 R834606168 R834606200 R834606203 R834606206
R834606230
R834606233 R834606236 R834606260 R834606263 R834606266 R834606401
R834606402
R834606405 R834606408 R834606432 R834606471 R834606472 R834606474
R834606475
R834606477 R834606478 R834606500 R834606503 R834606506 R834606530
R834606533
R834606536 R834606560 R834606563 R834606566 R834606712 R834606715
R834606718
R834606750 R834606751 R834606752 R834606753 R834606754 R834606755
R834606757
R834606758 R834606781 R834606782 R834606784 R834606785 R834606788

```

R834606800
  R834606803 R834606806 R834606807 R834606830 R834606831 R834606833
R834606834
  R834606835 R834606836 R834606837 R834606838 R834606870 R834606873
R834606874
  R834606876 R834606877 R834630122 R834630125 R834630226 R834630230
R834630231
  R834630232 R834630234 R834630235 R834630237 R834630238 R834630240
R834630241
  R834630242 R834630243 R834630244 R834630245 R834630246 R834630247
R834630261
  R834630262 R834630264 R834630265 R834630268 R834630270 R834630271
R834630273
  R834630276 R834630502))""^geo:dggsLiteral ;
  ] ;
.

```

C.1.2.3. Distributions

This example shows a **Geometry** instance linked to a serialization of it in the [Industry_Foundation_Classes](#) according to [ISO 10303-21](#) (not a format directly supported with a GeoSPARQL predicate) stored in a local file:

```

ex:mygeom
  a geo:Geometry ;
  geo:hasSerialization [
    a dcat:Distribution ;
    dcterms:conformsTo "https://www.iso.org/standard/63141.html"^xsd:anyURI ;
    dcat:byteSize 714356 ;
    dcat:accessURL "file:///some/local/path/mygeom.ifc"^xsd:anyURI .
  ] ;
.

```

Example SPARQL Queries & Rules

This Section provides example data and then illustrates the use of GeoSPARQL functions and the application of rules with that data.

Example Data

The following RDF data (Turtle format) encodes application-specific spatial data. The resulting spatial data is illustrated in the figure below. The RDF statements define the feature class **my:PlaceOfInterest**, and two properties are created for associating geometries with features: **my:hasExactGeometry** and **my:hasPointGeometry**. **my:hasExactGeometry** is designated as the default geometry for the **my:PlaceOfInterest** feature class.

All the following examples use the parameter values `relation_family = Simple Features`, `serialization = WKT`, and `version = 1.0`.

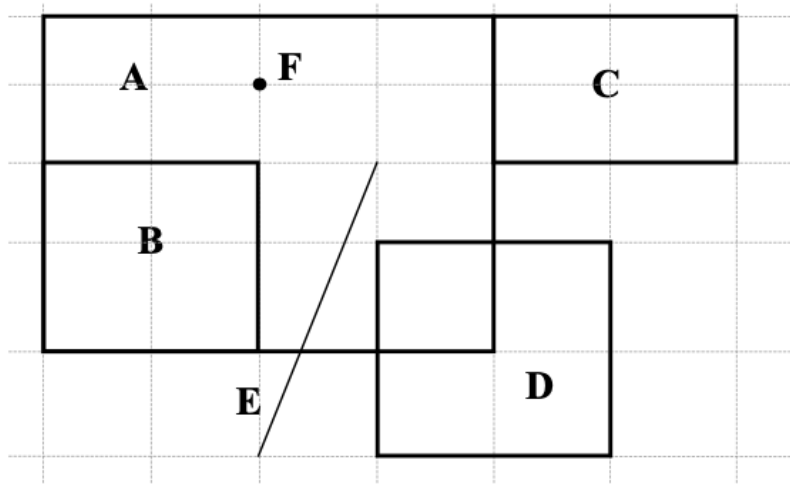


Illustration of spatial data

```
@prefix geo: <http://www.opengis.net/ont/geosparql#> .
@prefix my: <http://example.org/ApplicationSchema#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix sf: <http://www.opengis.net/ont/sf#> .
```

```
my:PlaceOfInterest a rdfs:Class ;
    rdfs:subClassOf geo:Feature .
```

```
my:A a my:PlaceOfInterest ;
    my:hasExactGeometry my:AExactGeom ;
    my:hasPointGeometry my:APointGeom .
```

```
my:B a my:PlaceOfInterest ;
    my:hasExactGeometry my:BExactGeom ;
    my:hasPointGeometry my:BPointGeom .
```

```
my:C a my:PlaceOfInterest ;
    my:hasExactGeometry my:CExactGeom ;
    my:hasPointGeometry my:CPointGeom .
```

```
my:D a my:PlaceOfInterest ;
    my:hasExactGeometry my:DExactGeom ;
    my:hasPointGeometry my:DPointGeom .
```

```
my:E a my:PlaceOfInterest ;
    my:hasExactGeometry my:EExactGeom .
```

```
my:F a my:PlaceOfInterest ;
    my:hasExactGeometry my:FExactGeom .
```

```
my:hasExactGeometry a rdf:Property ;
    rdfs:subPropertyOf geo:hasDefaultGeometry,
        geo:hasGeometry .
```

```
my:hasPointGeometry a rdf:Property ;
```

```
rdfs:subPropertyOf geo:hasGeometry .
```

```
my:AEExactGeom a sf:Polygon ;
  geo:asWKT "''<http://www.opengis.net/def/crs/OGC/1.3/CRS84>
    Polygon((-83.6 34.1, -83.2 34.1, -83.2 34.5,
    -83.6 34.5, -83.6 34.1))''^^geo:wktLiteral.

my:APointGeom a sf:Point ;
  geo:asWKT "''<http://www.opengis.net/def/crs/OGC/1.3/CRS84>
    Point(-83.4 34.3)''^^geo:wktLiteral.

my:BExactGeom a sf:Polygon ;
  geo:asWKT "''<http://www.opengis.net/def/crs/OGC/1.3/CRS84>
    Polygon((-83.6 34.1, -83.4 34.1, -83.4 34.3,
    -83.6 34.3, -83.6 34.1))''^^geo:wktLiteral.

my:BPointGeom a sf:Point ;
  geo:asWKT "''<http://www.opengis.net/def/crs/OGC/1.3/CRS84>
    Point(-83.5 34.2)''^^geo:wktLiteral.

my:CEExactGeom a sf:Polygon ;
  geo:asWKT "''<http://www.opengis.net/def/crs/OGC/1.3/CRS84>
    Polygon((-83.2 34.3, -83.0 34.3, -83.0 34.5,
    -83.2 34.5, -83.2 34.3))''^^geo:wktLiteral.

my:CPointGeom a sf:Point ;
  geo:asWKT "''<http://www.opengis.net/def/crs/OGC/1.3/CRS84>
    Point(-83.1 34.4)''^^geo:wktLiteral.

my:DEExactGeom a sf:Polygon ;
  geo:asWKT "''<http://www.opengis.net/def/crs/OGC/1.3/CRS84>
    Polygon((-83.3 34.0, -83.1 34.0, -83.1 34.2,
    -83.3 34.2, -83.3 34.0))''^^geo:wktLiteral.

my:DPointGeom a sf:Point ;
  geo:asWKT "''<http://www.opengis.net/def/crs/OGC/1.3/CRS84>
    Point(-83.2 34.1)''^^geo:wktLiteral.

my:EExactGeom a sf:LineString ;
  geo:asWKT "''<http://www.opengis.net/def/crs/OGC/1.3/CRS84>
    LineString(-83.4 34.0, -83.3 34.3)''^^geo:wktLiteral.

my:FExactGeom a sf:Point ;
  geo:asWKT "''<http://www.opengis.net/def/crs/OGC/1.3/CRS84>
    Point(-83.4 34.4)''^^geo:wktLiteral.
```

Example Queries

This Section illustrates the use of GeoSPARQL functions through a series of example queries.

All features that a given feature contains

Find all features that feature `my:A` contains, where spatial calculations are based on `my:hasExactGeometry`.

```
PREFIX my: <http://example.org/ApplicationSchema#>
PREFIX geo: <http://www.opengis.net/ont/geosparql#>
PREFIX geof: <http://www.opengis.net/def/function/geosparql/>

SELECT ?f
WHERE {
  my:A my:hasExactGeometry ?aGeom .
  ?aGeom geo:asWKT ?aWKT .
  ?f my:hasExactGeometry ?fGeom .
  ?fGeom geo:asWKT ?fWKT .

  FILTER (
    geof:sfContains(?aWKT, ?fWKT) &&
    !sameTerm(?aGeom, ?fGeom)
  )
}
```

Result:

?f
<code>my:B</code>
<code>my:F</code>

All features within bounding box

Find all features that are within a transient bounding box geometry, where spatial calculations are based on `my:hasPointGeometry`.

```
PREFIX my: <http://example.org/ApplicationSchema#>
PREFIX geo: <http://www.opengis.net/ont/geosparql#>
PREFIX geof: <http://www.opengis.net/def/function/geosparql/>

SELECT ?f
WHERE {
  ?f my:hasPointGeometry ?fGeom .
  ?fGeom geo:asWKT ?fWKT .
  FILTER (
    geof:sfWithin(
      ?fWKT,
      "<http://www.opengis.net/def/crs/OGC/1.3/CRS84>
      Polygon ((-83.4 34.0, -83.1 34.0,
                -83.1 34.2, -83.4 34.2,
                -83.4 34.0))"^^geo:wktLiteral
    )
  )
}
```

```
)  
}
```

Result:

?f
my:D

All features touching the union of two features

Find all features that touch the union of feature my:A and feature my:D, where computations are based on my:hasExactGeometry.

```
PREFIX my: <http://example.org/ApplicationSchema#>  
PREFIX geo: <http://www.opengis.net/ont/geosparql#>  
PREFIX geof: <http://www.opengis.net/def/function/geosparql/>
```

```
SELECT ?f  
WHERE {  
  ?f my:hasExactGeometry ?fGeom .  
  ?fGeom geo:asWKT ?fWKT .  
  my:A my:hasExactGeometry ?aGeom .  
  ?aGeom geo:asWKT ?aWKT .  
  my:D my:hasExactGeometry ?dGeom .  
  ?dGeom geo:asWKT ?dWKT .  
  FILTER (  
    geof:sfTouches(  
      ?fWKT,  
      geof:union(?aWKT, ?dWKT)  
    )  
  )  
}
```

Result:

?f
my:C

Three closest features to a feature

Find the 3 closest features to feature my:C, where computations are based on my:hasExactGeometry.

```
PREFIX uom: <http://www.opengis.net/def/uom/OGC/1.0/>  
PREFIX my: <http://example.org/ApplicationSchema#>  
PREFIX geo: <http://www.opengis.net/ont/geosparql#>  
PREFIX geof: <http://www.opengis.net/def/geosparql/function>
```

```
SELECT ?f
```

```

WHERE {
  my:C my:hasExactGeometry ?cGeom .
  ?cGeom geo:asWKT ?cWKT .
  ?f my:hasExactGeometry ?fGeom .
  ?fGeom geo:asWKT ?fWKT .
  FILTER (?fGeom != ?cGeom)
}
ORDER BY ASC (geof:distance(?cWKT, ?fWKT, uom:metre))
LIMIT 3

```

Result:

?f
my:A
my:D
my:E

Maximum and minimum coordinates of a set of geometries

Find the maximum and minimum coordinates of a given set of geometries.

```

PREFIX geo: <http://www.opengis.net/ont/geosparql#>
PREFIX geof: <http://www.opengis.net/def/function/geosparql/>

SELECT ?minX ?minY ?minZ ?maxX ?maxY ?maxZ
WHERE {
  BIND ("<http://www.opengis.net/def/crs/OGC/1.3/CRS84>
        Polygon Z((-83.4 34.0 0, -83.1 34.0 1,
                  -83.1 34.2 1, -83.4 34.2 1,
                  -83.4 34.0 0))"^^geo:wktLiteral) AS ?testgeom)
  BIND(geof:minX(?testgeom) AS ?minX)
  BIND(geof:maxX(?testgeom) AS ?maxX)
  BIND(geof:minY(?testgeom) AS ?minY)
  BIND(geof:maxY(?testgeom) AS ?maxY)
  BIND(geof:maxZ(?testgeom) AS ?maxZ)
  BIND(geof:minZ(?testgeom) AS ?minZ)
}

```

Result:

?minX	?minY	?minZ	?maxX	?maxY	?maxZ
-83.4	34.0	0	-83.1	34.2	1

Example Easting and Northing Functions

Find the easting and northing values of the given geometries.

```

PREFIX geo: <http://www.opengis.net/ont/geosparql#>
PREFIX geof: <http://www.opengis.net/def/function/geosparql/>

SELECT ?easting_geom1 ?easting_geom2 ?easting_geom3 ?northing_geom1 ?northing_geom2
?northing_geom3
WHERE {
  BIND ("<http://www.opengis.net/def/crs/OGC/1.3/CRS84>
        Point (33.95 -83.38)"^^geo:wktLiteral) AS ?testgeom1)
  BIND ("<http://www.opengis.net/def/crs/EPSG/0/4326>
        Point (33.95 -83.38)"^^geo:wktLiteral) AS ?testgeom2)
  BIND ("<http://www.opengis.net/def/crs/EPSG/0/3857>
        POINT (3779296.71243164 -18178745.6206012)"^^geo:wktLiteral) AS
?testgeom2)
  BIND(geof:easting(?testgeom1) AS ?easting_geom1)
  BIND(geof:easting(?testgeom2) AS ?easting_geom2)
  BIND(geof:easting(?testgeom3) AS ?easting_geom3)
  BIND(geof:northing(?testgeom1) AS ?northing_geom1)
  BIND(geof:northing(?testgeom2) AS ?northing_geom2)
  BIND(geof:northing(?testgeom3) AS ?northing_geom3)
}

```

Result:

?easting_geom1	?easting_geom2	?easting_geom3	?northing_geom1	?northing_geom2	?northing_geom3
33.95	-83.38	3779296.71243164	-83.38	33.95	-18178745.6206012

Example Rule Application

This section illustrates the query transformation strategy for implementing GeoSPARQL rules.

All features or geometries overlapping with another feature

*Find all features or geometries that overlap feature **my:A**.*

Original Query:

```

PREFIX geo: <http://www.opengis.net/ont/geosparql#>

SELECT ?f
WHERE { ?f geo:sfOverlaps my:A }

```

Transformed Query (application of transformation rule **geo:sfOverlaps**):

```

PREFIX my: <http://example.org/ApplicationSchema#>
PREFIX geo: <http://www.opengis.net/ont/geosparql#>

```

```
PREFIX geof: <http://www.opengis.net/def/function/geosparql/>
```

```
SELECT ?f
WHERE {
  { # check for asserted statement
    ?f geo:sfOverlaps my:A }
  UNION
  { # feature = feature
    ?f geo:hasDefaultGeometry ?fGeom .
    ?fGeom geo:asWKT ?fSerial .
    my:A geo:hasDefaultGeometry ?aGeom .
    ?aGeom geo:asWKT ?aSerial .
    FILTER (geof:sfOverlaps(?fSerial, ?aSerial))
  }
  UNION
  { # feature = geometry
    ?f geo:hasDefaultGeometry ?fGeom .
    ?fGeom geo:asWKT ?fSerial .
    my:A geo:asWKT ?aSerial .
    FILTER (geof:sfOverlaps(?fSerial, ?aSerial))
  }
  UNION
  { # geometry = feature
    ?f geo:asWKT ?fSerial .
    my:A geo:hasDefaultGeometry ?aGeom .
    ?aGeom geo:asWKT ?aSerial .
    FILTER (geof:sfOverlaps(?fSerial, ?aSerial))
  }
  UNION
  { # geometry = geometry
    ?f geo:asWKT ?fSerial .
    my:A geo:asWKT ?aSerial .
    FILTER (geof:sfOverlaps(?fSerial, ?aSerial))
  }
}
```

Result:

?f
my:D
my:DExactGeom
my:E
my:EExactGeom

Example Geometry Serialization Conversion Functions

For the geometry literal values in [C.1.2.4 Geometry Serializations](#):

Application of the function `geof:asWKT` to the GML, KML, GeoJSON, GeoCode, and DGGs literals should return WKT literal and similarly for each of the other conversion methods, `geof:asGML`,

geof:asKML, geof:asGeoJSON & geof:asDGGs.

[1] <http://www.qudt.org>

Appendix D: CQL / GeoSPARQL Mapping

F.0. Overview

This annex presents a mapping between the Common Query Language(CQL) [CQLDEF] and GeoSPARQL as well as generic SPARQL Section 1.1.4. This is likely of relevance to the delivery of GeoSPARQL data via systems such as the OGC's Web Feature Service [WFS] and OGC API Features [OGCSFACA] which implement CQL.

D.1. Accessing spatial Features in a SPARQL endpoint

Spatial *Features* accessed via SPARQL endpoints [SPARQLPROT] are, as defined in the GeoSPARQL standard, instances of the OWL class `geo:Feature` or of subclasses of it. They may have one or more `geo:hasGeometry` properties indicating `geo:Geometry` instances and other properties related to the Feature. They may also be grouped into `geo:FeatureCollection` instances where `geo:FeatureCollection` is a new class in GeoSPARQL 1.1, specifically for the description of collections of `geo:Feature` instances.

The following example SPARQL query retrieves all Features within the Feature Collection with the IRI `ex:x` within a given SPARQL endpoint.

```
PREFIX ex: <http://example.com/>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT ?fcollection ?item ?rel ?val ?geom
WHERE {
  ex:x rdfs:member ?item .
  ?item rdfs:subClassOf* geo:Feature .
}
```

GeoSPARQL's `geo:FeatureCollection` definition requires that `geo:Feature` instances are to be linked to the Collection by use of the `rdf:member` property. No inverse property is defined.

NOTE

Some CQL-implementing systems, such as OGC API, have fixed notions of Feature Collections and require that Features be members of exactly one Feature Collection. There is no such restriction in GeoSPARQL: Features may be members of one or more Feature Collections.

An extension to the above can retrieve any Geometry serializations for the Features within Feature Collection `ex:x`:

```
PREFIX ex: <http://example.com/>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT ?fcollection ?item ?rel ?val ?geom
WHERE {
```

```

ex:x rdfs:member ?item .
?item rdfs:subClassOf* geo:Feature .

OPTIONAL {
  ?item geo:hasGeometry/geo:hasSerialization ?geom
}
}

```

Some additional concerns for GeoSPARQL / CQL or OGC API Features Feature Collections mappings are:

- APIs may need more information about the `geo:FeatureCollection` instance for correct handling, in particular, an identifier and perhaps a label. If the back-end data store also contains information for the `geo:FeatureCollection` instance then this may be queried for. If not, the API might need to create such data
 - One particular scenario observed is that OGC APIs require token-like identifiers for Feature Collections and GeoSPARQL IRIs, or their parts, may not be able to be used for such. In these cases, the RDF property `dcterms:identifier` may be used to store appropriate token-like identifiers
- Perhaps only data in a certain namespace is of interest. The solution is to apply FILTER expressions to the SPARQL query

D.2. Mappings from CQL2 statements to GeoSPARQL queries

This section presents lists of equivalences between Common Query Language (CQL2) [\[CQLDEF\]](#) statements and GeoSPARQL statements.

D.2.1. Query Parameters

Several query parameters may be given as parameters to the HTTP request of CD-implementing systems, such as the OGC API Features service. These parameters have an influence on the SPARQL query to be executed for the retrieval of a FeatureCollection to be exposed using an OGC API Features service.

CQL To GeoSPARQL Mappings: Query Parameters

Query Parameter	Example	SPARQL Expression	Example	Comment
limit	limit=5	LIMIT	LIMIT 5	
offset	offset=10	OFFSET	OFFSET 10	

Query Parameter	Example	SPARQL Expression	Example	Comment
bbox	bbox=160.6,-55.95,-170,-25.89	<code>FILTER(geo:sfIntersects())</code>	<code>FILTER(geo:sfIntersects(?geom, "POLYGON((160.6 -55.95,160.6 -25.89, -170 -25.89, -170 -55.95, 160.6 -55.95))"^^geo:wktLiteral))</code>	WKT does not define a type boundingbox, therefore a bbox is converted to a Polygon
datetime	datetime=2018-02-12T23%3A20%3A52Z	-	-	GeoSPARQL doesn't detail temporal aspects of data. Filtering data using RDF temporal properties may be achieved using basic SPARQL queries and also OWL TIME [TIME]

D.2.2. Literal Values

CQL2 defines literal values for a variety of datatypes. The following table shows the equivalences of these values in RDF which may be used in any GeoSPARQL query.

CQL To GeoSPARQL Mappings: Literal Values

CQL2 literal	Examples	(Geo)SPARQL literal	Examples
String	"This is a string"	<code>xsd:string</code>	"This is a string"^^xsd:string
Number	-100 3.14159	<code>xsd:int</code> , <code>xsd:integer</code> , <code>xsd:double</code>	"-100"^^xsd:integer "3.14159"^^xsd:double
Boolean	true false	<code>xsd:boolean</code>	"true"^^xsd:boolean "false"^^xsd:boolean
Spatial Geometry (WKT)	POINT(1 1)	WKT Literal	"POINT(1 1)"^^geo:wktLiteral
Spatial Geometry (JSON)	{"type": "Point", "coordinates": [1,1]}	GeoJSON Literal	"{"type": "Point", "coordinates": [1,1]}"^^geo:geoJSONLiteral
Temporal Literal	1969-07-20 1969-07-20T20:17:40Z	<code>xsd:date</code> , <code>xsd:dateTime</code> , <code>xsd:dateTimeStamp</code>	"1969-07-20"^^xsd:date "1969-07-20T20:17:40Z"^^xsd:dateTime

D.2.3. Property references

CQL2 allows the referencing of properties in a Feature Collection it is targeting for filtering. A property reference is converted to a triple pattern as shown in the following example. A SPARQL variable `?item` is assumed to represent the Feature Collection.

CQL To GeoSPARQL Mappings: Property references

Property Reference	Triple pattern
name="OGC"	<code>?item my:name "OGC"^^xsd:string</code>
number=5	<code>?item my:number "5"^^xsd:integer</code>
number>5	<code>?item my:number ?number . FILTER(?number>5)</code>

D.2.4. Comparison Predicates

CQL2 defines comparison predicates to compare two scalar expressions. A comparison predicate is converted to a triple pattern as shown in the following example. A SPARQL variable `?item` is assumed to represent the Feature Collection.

CQL To GeoSPARQL Mappings: Comparison Predicates

Comparison predicate	Triple pattern	Comment
name="OGC"	<code>?item my:name "OGC"^^xsd:string</code>	Equality statements can be converted to a triple pattern
number=5	<code>?item my:number "5"^^xsd:integer</code>	
number>5	<code>?item my:number ?number . FILTER(?number>5)</code>	Arithmetic comparisons (<,>,>=,<=) are converted to filter expressions
number BETWEEN 5 AND 10	<code>?item my:number ?number . FILTER(?number>=5 && ?number<=10)</code>	BETWEEN statements are converted to arithmetic expressions
name IN ("OGC","W3C")	<code>?item my:name IN ("OGC", "W3C")</code>	IN statements may also be expressed using SPARQL VALUES statements
name IS NOT NULL	<code>EXISTS {?item my:name ?name }</code>	NOT NULL statements are converted to EXIST statements
name LIKE "OGC."	<code>?item my:name ?name . FILTER(regex(?name, "OGC.", "i"))</code>	LIKE statements are converted to SPARQL regex filters
INTERSECTS(geometry1, geometry2)	<code>FILTER(geof:sfIntersects(?geometry1,?geometry2))</code>	The INTERSECTS filter statement is converted to a GeoSPARQL FILTER statement

There is no direct GeoSPARQL equivalent to a CRS-based CQL filter, however certain GeoSPARQL geometry literals have explicit CRS/SRS information that may be filtered using SPARQL `REGEX`

operators.

D.2.5. Spatial Operators

GeoSPARQL includes equivalents of many CQL2 filter functions as can be seen in the table below.

CQL To GeoSPARQL Mappings: Spatial Operators

CQL2 Filter Expression	GeoSPARQL Filter Function
CONTAINS(geometry1,geometry2)	FILTER(geof:sfContains(?geometry1,?geometry2))
CROSSES(geometry1,geometry2)	FILTER(geof:sfCrosses(?geometry1,?geometry2))
DISJOINT(geometry1,geometry2)	FILTER(geof:sfDisjoint(?geometry1,?geometry2))
EQUALS(geometry1,geometry2)	FILTER(geof:sfEquals(?geometry1,?geometry2))
INTERSECTS(geometry1,geometry2)	FILTER(geof:sfIntersects(?geometry1,?geometry2))
OVERLAPS(geometry1,geometry2)	FILTER(geof:sfOverlaps(?geometry1,?geometry2))
TOUCHES(geometry1,geometry2)	FILTER(geof:sfTouches(?geometry1,?geometry2))
WITHIN(geometry1,geometry2)	FILTER(geof:sfWithin(?geometry1,?geometry2))

D.2.6. Temporal Operators

Temporal operators are not part of the GeoSPARQL standard.

CQL To GeoSPARQL Mappings: Temporal Operators

CQL2 Filter Expression	GeoSPARQL Filter Function
beginTime AFTER 1969-07-16T13:32:00Z	N/A
beginTime BEFORE 1969-07-16T13:32:00Z	N/A
beginTime BEGINS 1969-07-16T13:32:00Z	N/A
beginTime BEGUNBY 1969-07-16T13:32:00Z	N/A
beginTime DURING 1969-07-16T13:32:00Z	N/A
beginTime ENDEDBY 1969-07-16T13:32:00Z	N/A
beginTime ENDS 1969-07-16T13:32:00Z	N/A
beginTime MEETS 1969-07-16T13:32:00Z	N/A
beginTime METBY 1969-07-16T13:32:00Z	N/A
beginTime OVERLAPPEDBY 1969-07-16T13:32:00Z	N/A
beginTime TCONTAINS 1969-07-16T13:32:00Z	N/A
beginTime TEQUALS 1969-07-16T13:32:00Z	N/A

CQL2 Filter Expression	GeoSPARQL Filter Function
beginTime TOVERLAPS 1969-07-16T13:32:00Z	N/A

As noted above in Section [F.2.1 Query Parameters](#), temporal filtering of RDF data via SPARQL queries is possible with standard SPARQL functions to compare date values (`xsd:date`, `xsd:dateTime` and `xsd:dateTimeStamp` literals) and OWL TIME [\[TIME\]](#) may be used to assert temporal relations between objects.

D.3. Mappings from Simple Features for SQL

The following table maps the functions and properties from Simple Features for SQL [\[OGCSFACA\]](#) [\[ISO19125-1\]](#) to GeoSPARQL.

CQL To GeoSPARQL Mappings: Simple Features for SQL

Simple Features for SQL	GeoSPARQL Equivalent	Since GeoSPARQL	Related Property Available	Since GeoSPARQL
2.1.1.1 Basic Methods on Geometry				
Dimension(): Double	<code>geof:dimension</code>	-	<code>geo:dimension</code>	1.0
GeometryType(): Integer	Class of geometry instance	1.0	N/A	-
SRID(): Integer	<code>geof:getSRID</code>	1.0	N/A	-
Envelope(): Geometry	<code>geof:envelope</code>	1.0	<code>geo:hasBoundingBox</code>	1.1
AsText(): String	<code>geof:asWKT</code>	1.1	<code>geo:asWKT</code>	1.0
AsBinary(): Binary	N/A	-	N/A	-
IsEmpty(): Integer	<code>geof:isEmpty</code>	-	<code>geo:isEmpty</code>	1.0
IsSimple(): Integer	<code>geof:isEmpty</code>	-	<code>geo:isSimple</code>	1.0
Boundary(): Geometry	<code>geof:boundary</code>	1.0	N/A	-
2.1.1.2 Spatial Relations				
Equals(anotherGeometry): Integer	<code>geof:sfEquals</code>	1.0	<code>geo:sfEquals</code>	1.0
Disjoint(anotherGeometry): Integer	<code>geof:sfDisjoint</code>	1.0	<code>geo:sfDisjoint</code>	1.0

Simple Features for SQL	GeoSPARQL Equivalent	Since GeoSPARQL	Related Property Available	Since GeoSPARQL
Intersects(another Geometry: Geometry): Integer	<code>geof:sfIntersects</code>	1.0	<code>geo:sfIntersects</code>	1.0
Touches(another Geometry: Geometry): Integer	<code>geof:sfTouches</code>	1.0	<code>geo:sfTouches</code>	1.0
Crosses(another Geometry: Geometry): Integer	<code>geof:sfCrosses</code>	1.0	<code>geo:sfCrosses</code>	1.0
Within(another Geometry: Geometry): Integer	<code>geof:sfWithin</code>	1.0	<code>geo:sfWithin</code>	1.0
Contains(another Geometry: Geometry): Integer	<code>geof:sfContains</code>	1.0	<code>geo:sfContains</code>	1.0
Overlaps(another Geometry: Geometry): Integer	<code>geof:sfOverlaps</code>	1.0	<code>geo:sfOverlaps</code>	1.0
Relate(another Geometry: Geometry, IntersectionPatternMatrix: String): Integer	<code>geof:relate</code>	1.0	N/A	-
2.1.1.3 Spatial Analysis				
Buffer(distance: Double): Geometry	<code>geof:buffer</code>	1.0	N/A	-
ConvexHull(): Geometry	<code>geof:convexHull</code>	1.0	N/A	-
Intersection(another Geometry: Geometry): Geometry	<code>geof:intersection</code>	1.0	N/A	-
Union(another Geometry: Geometry): Geometry	<code>geof:union</code>	1.0	N/A	-

Simple Features for SQL	GeoSPARQL Equivalent	Since GeoSPARQL	Related Property Available	Since GeoSPARQL
Difference(anotherGeometry): Geometry	geof:difference	1.0	N/A	-
SymDifference(anotherGeometry): Geometry	geof:symDifference	1.0	N/A	-
2.1.2.1 GeometryCollection				
NumGeometries(): Integer	geof:numGeometries	-	N/A	-
GeometryN(N: Integer): Geometry	geof:geometryN	-	N/A	-
2.1.3.1 Point				
X(): Double	N/A	-	N/A	-
Y(): Double	N/A	-	N/A	-
Z(): Double (not in the SQL spec, but a logical extension)	N/A	-	N/A	-
M(): Double (not in the SQL spec, but a logical extension)	N/A	-	N/A	-
2.1.5.1 Curve				
Length(): Double	geof:length	-	geo:hasLength	1.1
StartPoint(): Point	N/A	-	N/A	-
EndPoint(): Point	N/A	-	N/A	-
IsClosed(): Integer	N/A	-	N/A	-
IsRing(): Integer	N/A	-	N/A	-
2.1.6.1 LineString				
NumPoints(): Integer	N/A	-	N/A	-
PointN(N: Integer): Point	N/A	-	N/A	-
2.1.7.1 MultiCurve				
IsClosed(): Integer	N/A	-	N/A	-
Length(): Double	geof:length	-	geo:hasLength	1.1

Simple Features for SQL	GeoSPARQL Equivalent	Since GeoSPARQL	Related Property Available	Since GeoSPARQL
2.1.9.1 Surface				
Area(): Double	geof:area	-	geo:hasArea	1.1
Centroid(): Point	geof:centroid	1.1	geo:hasCentroid	1.1
PointOnSurface(): Point	N/A	-	N/A	-
2.1.10.1 Polygon				
ExteriorRing(): LineString	N/A	-	N/A	-
NumInteriorRing(): Integer	N/A	-	N/A	-
InteriorRingN(N: Integer): LineString	N/A	-	N/A	-
2.1.11.1 MultiSurface				
Area(): Double	geof:area	-	geo:hasArea	1.1
Centroid(): Point	geof:centroid	1.1	geo:hasCentroid	1.1
PointOnSurface(): Point	N/A	-	N/A	-

Bibliography